数据结构与算法分析-7

人工智能与自动化学院 谭毅华

yhtan@hust.edu.cn 87556301

上章内容回顾

- 树是一种非线性结构：一对多、递归定义等特点
- 二叉树具有相关性质：i层最多结点数（ 2^{i-1} ）、满二叉树与完全二叉树的性质
- 二叉树的遍历方法及其实现
  - 前序、后序与中序遍历
  - 中序的非递归实现（栈的构建及其操作）
  - 二叉树的线索化算法（中序线索化）
  - 二叉排序树的查找、插入与删除
- 二叉树的编码应用
  - 赫夫曼树的构建
  - 赫夫曼树结点的表示
  - 赫夫曼树的编码算法

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

7. 图

- 7.1 图的定义和术语
- 7.2 图的存储结构
- 7.3 图的遍历
- 7.4生成树
- 7.5拓扑排序
- 7.6关键路径
- 7.7最短路径

華中科技大學 人工智能与自动化学院
A-ambitious 积极进取 U-unique 锐意创新
T-tolerant 包容并蓄 O-open 开放交流

7.1 图的定义和术语

- 图(Graph)——图G是由两个集合V(G)和E(G)组成的,记为G=(V,E)。其中：

  V(G)是顶点的非空有限集

  E(G)是边的有限集合，边是顶点的无序对或有序对

- 有向图——有向图G是由两个集合V(G)和E(G)组成的

  其中：V(G)是顶点的非空有限集

  E(G)是有向边（也称弧）的有限集合，弧是顶点的有序对，记为<v,w>，v,w是顶点，v为弧尾，w为弧头

- 无向图——无向图G是由两个集合V(G)和E(G)组成的

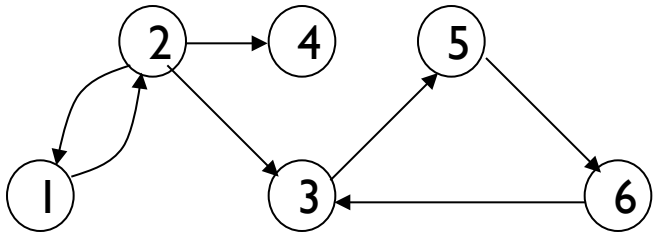
  其中：V(G)是顶点的非空有限集

  E(G)是边的有限集合，边是顶点的无序对，记为（v,w）或（w,v），并且（v,w)=(w,v)

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

图的例子

例1

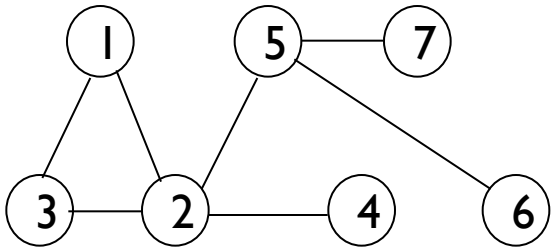


G1

图G1中：V(G1)={1,2,3,4,5,6}

E(G1)={<1,2>, <2,1>, <2,3>, <2,4>, <3,5>, <5,6>, <6,3>}

例2



G2

图G2中：V(G2)={1,2,3,4,5,6,7}

E(G1)={(1,2), (1,3), (2,3), (2,4),(2,5), (5,6), (5,7)}

華中科技大學 人工智能与自动化学院

图的定义和术语

- 有向完备图——n个顶点的有向图最大边数是n(n-1)
- 无向完备图——n个顶点的无向图最大边数是n(n-1)/2
- 权——与图的边或弧相关的数叫权
- 网——带权的图叫网
- 子图——如果图G(V,E)和图G‘(V’,E‘),满足：
  - V’⊆V
  - E’⊆E　　则称G‘为G的子图
- 顶点的度
  - 无向图中，顶点的度为与每个顶点相连的边数
  - 有向图中，顶点的度分成入度与出度
    - 入度：以该顶点为头的弧的数目
    - 出度：以该顶点为尾的弧的数目

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

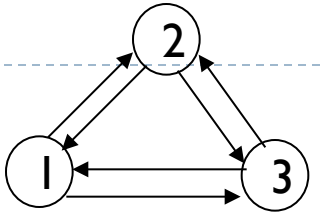
图的定义和术语

- 路径：路径是顶点的序列$V=\{V_{i0},V_{i1},\ldots\ldots V_{in}\}$，满足$(V_{ij-1},V_{ij})\in E$ 或 $<V_{ij-1},V_{ij}>\in E,(1<j\leq n)$
- 路径长度：沿路径边的数目或沿路径各边权值之和
- 回路：第一个顶点和最后一个顶点相同的路径叫~
- 简单路径：序列中顶点不重复出现的路径叫~
- 简单回路：除了第一个顶点和最后一个顶点外，其余顶点不重复出现的回路叫~
- 连通：从顶点V到顶点W有一条路径，则说V和W是连通的
- 连通图：图中任意两个顶点都是连通的叫~
- 连通分量：非连通图的每一个连通部分叫~
- 强连通图：有向图中，如果对每一对V_i，$V_j\in V$，$V_i\neq V_j$，从V_i到V_j和从V_j到V_i都存在路径，则称G是~

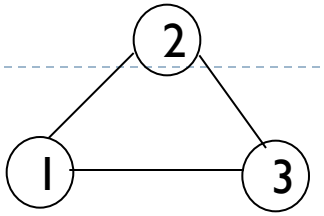
華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

示例

例

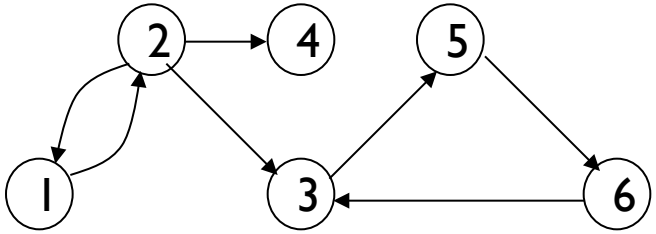


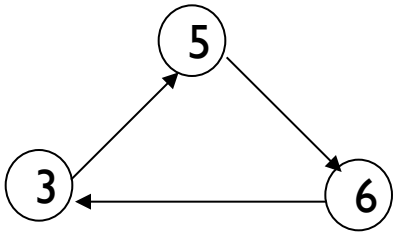
有向完备图



无向完备图

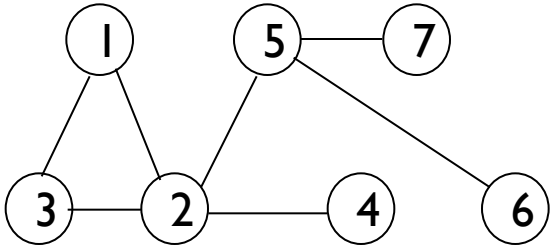
例





例

图与子图

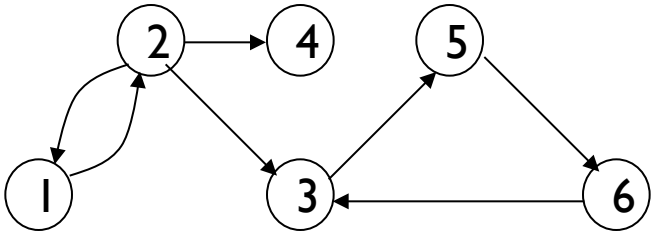


G2

顶点5的度：3

顶点2的度：4

例



G1

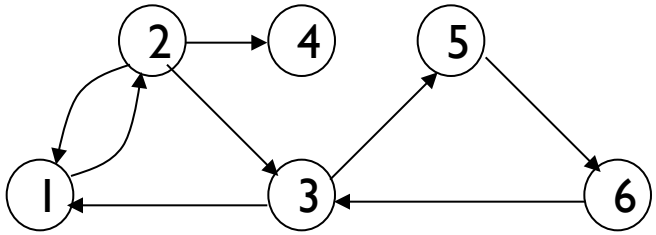
顶点2入度：1　出度：3

顶点4入度：1　出度：0

華中科技大学
人工智能与自动化学院
A-ambitious 积极进取 U-unique 锐意创新
T-tolerant 包容并蓄 O-open 开放交流

示例

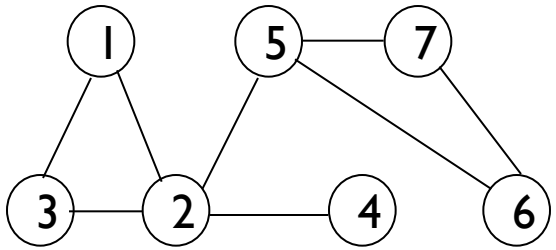
例



G1

路径：1，2，3，5，6，3
路径长度：5
简单路径：1，2，3，5
回路：1，2，3，5，6，3，1
简单回路：3，5，6，3

例

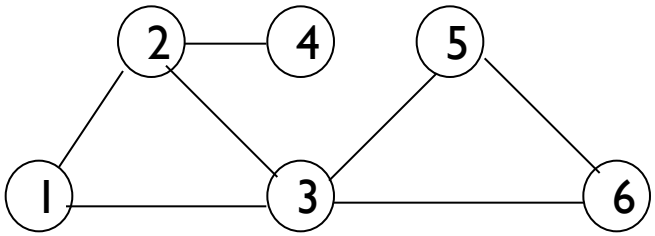


G2

路径：1，2，5，7，6，5，2，3
路径长度：7
简单路径：1，2，5，7，6
回路：1，2，5，7，6，5，2，1
简单回路：1，2，3，1

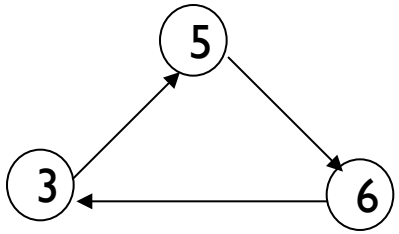
示例

例



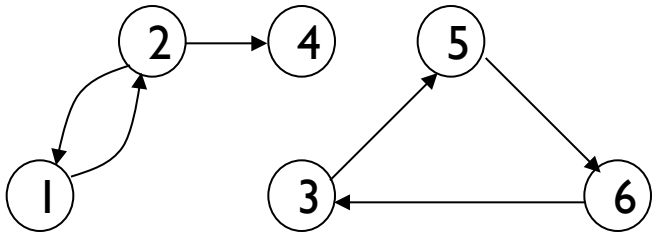
连通图

例



强连通图

例



非连通图
连通分量

華中科技大學
人工智能与自动化学院

图的抽象数

ADT Graph {

数据对象 V:V 是具有相同特性的

数据关系 R:

R = {VR}

VR = {<v,w> | v,w∈V 且

谓词 P(v,w)

基本操作 P:

CreateGraph(&G,V,VR);

初始条件:V 是图的顶点集,V

操作结果:按 V 和 VR 的定义

DestroyGraph(&G);

初始条件:图 G 存在。

操作结果:销毁图 G。

LocateVex(G, u);

初始条件:图 G 存在,u 和 G

操作结果:若 G 中存在顶点 u

GetVex(G, v);

初始条件:图 G 存在,v 是 G

操作结果:返回 v 的值。

PutVex(&G, v, value);

初始条件:图 G 存在,v 是 G

操作结果:对 v 赋值 value。

FirstAdjVex(G, v);

初始条件:图 G 存在,v 是 G 中某个顶点。

操作结果:返回 v 的第一个邻接顶点。若顶点在 G 中没有邻接顶点,则返回"空"。

NextAdjVex(G, v, w);

初始条件:图 G 存在,v 是 G 中某个顶点,w 是 v 的邻接顶点。

操作结果:返回 v 的(相对于 w 的)下一个邻接顶点。若 w 是 v 的最后一个邻接点,则返回"空"。

InsertVex(&G, v);

初始条件:图 G 存在,v 和图中顶点有相同特征。

操作结果:在图 G 中增添新顶点 v。

DeleteVex(&G, v);

初始条件:图 G 存在,v 是 G 中某个顶点。

操作结果:删除 G 中顶点 v 及其相关的弧。

InsertArc(&G, v, w);

初始条件:图 G 存在,v 和 w 是 G 中两个顶点。

操作结果:在 G 中增添弧<v,w>,若 G 是无向的,则还增添对称弧<w,v>。

DeleteArc(&G, v, w);

初始条件:图 G 存在,v 和 w 是 G 中两个顶点。

操作结果:在 G 中删除弧<v,w>,若 G 是无向的,则还删除对称弧<w,v>。

DFSTraverse(G, Visit());

初始条件:图 G 存在,Visit 是顶点的应用函数。

操作结果:对图进行深度优先遍历。在遍历过程中对每个顶点调用函数 Visit 一次且仅一次。一旦 visit()失败,则操作失败。

BFSTraverse(G, Visit());

初始条件:图 G 存在,Visit 是顶点的应用函数。

操作结果:对图进行广度优先遍历。在遍历过程中对每个顶点调用函数 Visit 一次且仅一次。一旦 visit()失败,则操作失败。

}ADT Graph

7.2 图的存储结构

- 数组表示
- 邻接表
- 十字链表
- 邻接多重表

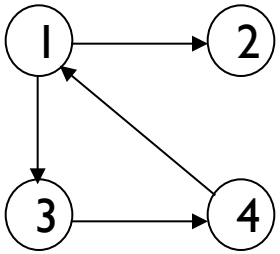
華中科技大学 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

数组表示-邻接矩阵

- 邻接矩阵——表示顶点间相联关系的矩阵
- 定义：设G=(V,E)是有n≥1个顶点的图，G的邻接矩阵A是具有以下性质的n阶方阵

$$A[i,j]=\begin{cases}1, & (v_i,v_j) \text{ 或 } <v_i,v_j>\in E(G)\\ 0, & \text{其它}\end{cases}$$

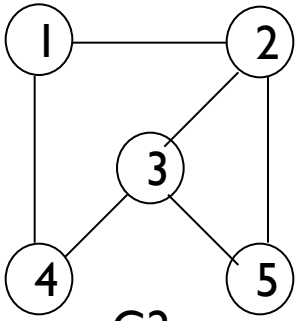
例



G1

$$\begin{array}{c|cccc} & ① & ② & ③ & ④ \\ \hline ① & 0 & 1 & 1 & 0 \\ ② & 0 & 0 & 0 & 0 \\ ③ & 0 & 0 & 0 & 1 \\ ④ & 1 & 0 & 0 & 0 \end{array}$$

例



G2

$$\begin{array}{c|ccccc} & ① & ② & ③ & ④ & ⑤ \\ \hline ① & 0 & 1 & 0 & 1 & 0 \\ ② & 1 & 0 & 1 & 0 & 1 \\ ③ & 0 & 1 & 0 & 1 & 1 \\ ④ & 1 & 0 & 1 & 0 & 0 \\ ⑤ & 0 & 1 & 1 & 0 & 0 \end{array}$$

華中科技大學 人工智能与自动化学院

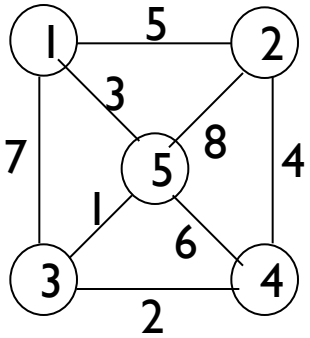
邻接矩阵表示的特点

- 无向图的邻接矩阵对称，可压缩存储；有n个顶点的无向图需存储空间为n(n+1)/2
- 有向图邻接矩阵不一定对称；有n个顶点的有向图需存储空间为n^2
- 无向图中顶点Vi的度TD(Vi)是邻接矩阵A中第i行元素之和
- 有向图中：
  - 顶点Vi的出度是A中第i行元素之和
  - 顶点Vi的入度是A中第i列元素之和
- 网络的邻接矩阵可定义为：

$$A[i,j]=\begin{cases}\omega_{ij}, & (v_i,v_j) \quad <v_i,v_j>\in E(G)\\ 0, & \text{其它}\end{cases}$$

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

例



$$
\begin{array}{c|ccccc}
 & ① & ② & ③ & ④ & ⑤ \\
\hline
① & 0 & 5 & 7 & 0 & 3 \\
② & 5 & 0 & 0 & 4 & 8 \\
③ & 7 & 0 & 0 & 2 & 1 \\
④ & 0 & 4 & 2 & 0 & 6 \\
⑤ & 3 & 8 & 1 & 6 & 0
\end{array}
$$

華中科技大學
人工智能与自动化学院
A-ambitious 积极进取 U-unique 锐意创新
T-tolerant 包容并蓄 O-open 开放交流

邻接矩阵-存储结构表示

```
//------ 图的数组(邻接矩阵)存储表示 ------
#define INFINITY          INT_MAX          // 最大值∞
#define MAX_VERTEX_NUM    20               // 最大顶点个数
typedef enum {DG, DN, UDG, UDN} GraphKind;  //{有向图,有向网,无向图,无向网}
typedef struct ArcCell {
   VRType      adj;   // VRType 是顶点关系类型。对无权图,用 1 或 0
                      // 表示相邻否;对带权图,则为权值类型。
   InfoType   *info;  // 该弧相关信息的指针
}ArcCell,  AdjMatrix[MAX_VERTEX_NUM][MAX_VERTEX_NUM];
typedef struct {
   VertexType  vexs[MAX_VERTEX_NUM];       // 顶点向量
   AdjMatrix   arcs;                       // 邻接矩阵
   int         vexnum, arcnum;             // 图的当前顶点数和弧数
   GraphKind   kind;                       // 图的种类标志
}MGraph;
```


華中科技大学 人工智能与自动化学院

邻接矩阵-图的构造算法

构造无向图

```
Status CreateUDN(MGraph &G) {
    // 采用数组(邻接矩阵)表示法,构造无向网 G。
    scanf(&G.vexnum, &G.arcnum, &IncInfo);      // IncInfo 为 0 则各弧不含其他信息
    for (i=0; i<G.vexnum; ++i)  scanf (&G.vexs[i]);  // 构造顶点向量
    for (i=0; i<G.vexnum; ++i)                  // 初始化邻接矩阵
       for (j=0; j<G.vexnum; ++j)  G.arcs[i][j] = {INFINITY, NULL};   //{adj,info}
    for (k=0; k<G.arcnum; ++k) {                // 构造邻接矩阵
       scanf(&v1, &v2, &w);                     // 输入一条边依附的顶点及权值
       i = LocateVex(G, v1);  j = LocateVex(G, v2);  // 确定 v1 和 v2 在 G 中位置
       G.arcs[i][j].adj = w;                    // 弧<v1,v2>的权值
       if (IncInfo) Input( *G.arcs[i][j].info);   // 若弧含有相关信息,则输入
       G.arcs[j][i] = G.arcs[i][j];             // 置<v1,v2>的对称弧<v2,v1>
    }
    return OK;
}// CreateUDN
```

算法 7.2

華中科技大学 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

链式存储-邻接表

- 实现：为图中每个顶点建立一个单链表，第i个单链表中的结点表示依附于顶点V_i的边（有向图中指以V_i为尾的弧）

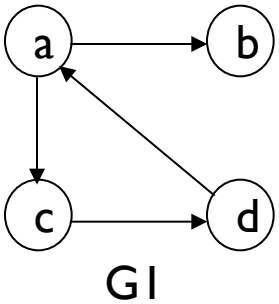
```
#define MAX_VERTEX_NUM 20
typedef   struct   ArcNode {
    int  adjvex;      //邻接点域，存放与Vi邻接的点在表头数组中的位置
    struct  ArcNode  *nextarc;  //链域，指示下一条边或弧
    InfoType *info;
}ArcNode;
typedef   struct VNode {   //表头接点
   VertexType  data;      //存放顶点信息
   ArcNode  *firstarc;   //指示第一个邻接点
}VNode,AdjList[MAX_VERTEX_NUM];
typedef struct {
   AdjList vertices;
    int vexnum, arcnum;
    int kind;    //图的种类标识
} ALGraph;
```

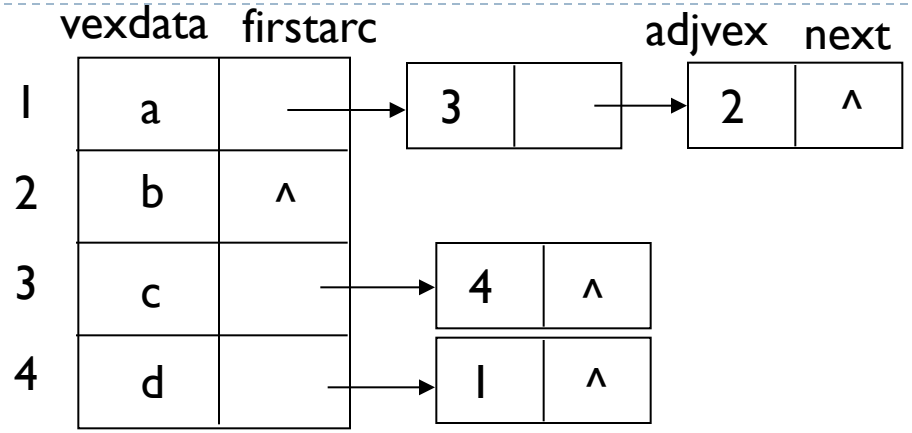
adjvex	nextarc info

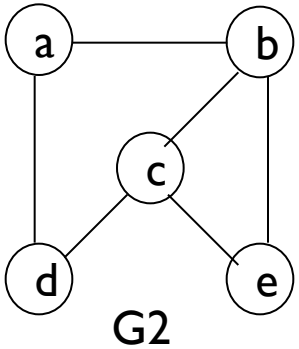
vexdata	firstarc

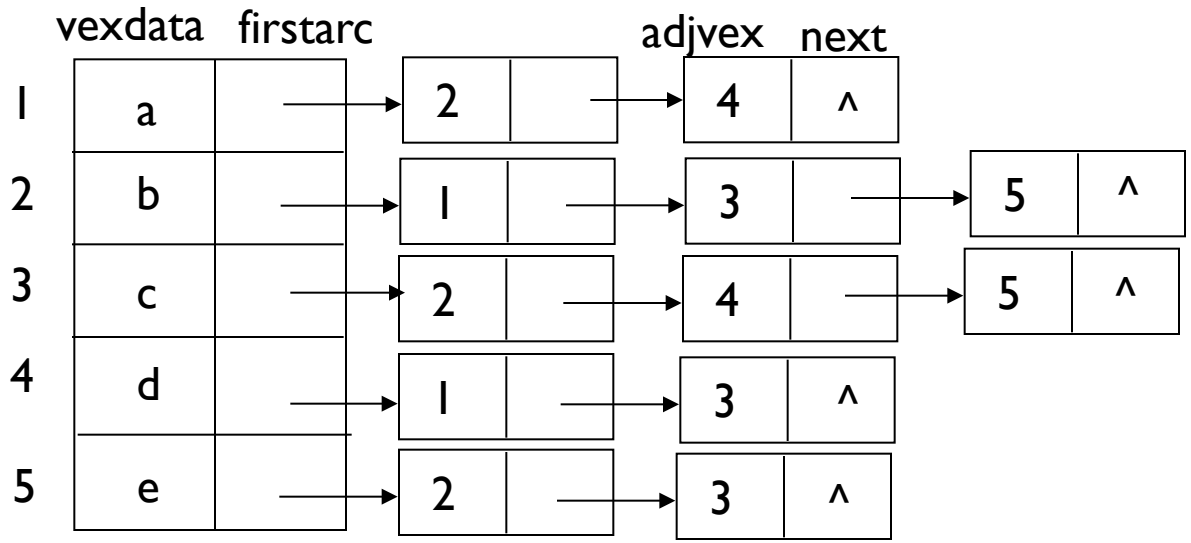
華中科技大學 人工智能与自动化学院

邻接表示例


a
b
c
d
G1


vexdata
firstarc
adjvex
next
1
a
3
2
^
2
b
^
3
c
4
^
4
d
1
^

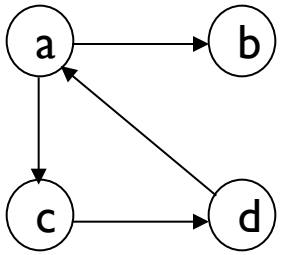

a
b
c
d
e
G2


vexdata
firstarc
adjvex
next
1
a
2
4
^
2
b
1
3
5
^
3
c
2
4
5
^
4
d
1
3
^
5
e
2
3
^

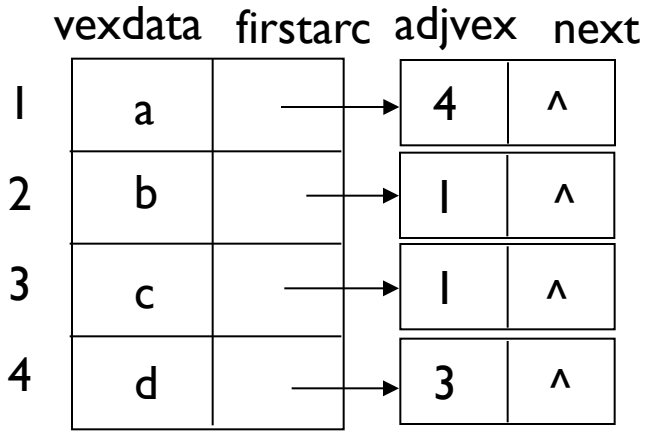
邻接表特点

- 无向图中顶点V_i的度为第i个单链表中的结点数
- 有向图中
  - 顶点V_i的出度为第i个单链表中的结点个数
  - 顶点V_i的入度为整个单链表中邻接点域值是i的结点个数
  - 逆邻接表：有向图中对每个结点建立以V_i为头的弧的单链表

例



G1



華中科技大學
人工智能与自动化学院

有向图的十字链表表示法

```
#define MAX_VERTEX_NUM 20

typedef  struct  ArcBox {     //弧结点
    int  tailvex, headvex;      //弧尾、弧头在表头数组中位置
    struct ArcBox  *hlink, *tlink；//分别指向弧头、弧尾相同的下一条弧
    InfoType *info;
}ArcBox;
```

tailvex	headvex	hlink	tlink	info

```
typedef  struct  VexNode {     //顶点结点
   VertexType data; //存与顶点有关信息
   ArcBox *firstin, *firstout；//分别指向该顶点第一条入弧和出弧
}VexNode;
```

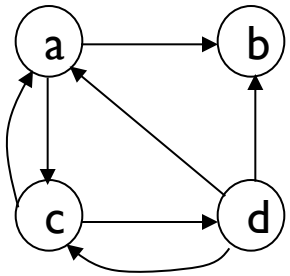
data	firstin	firstout

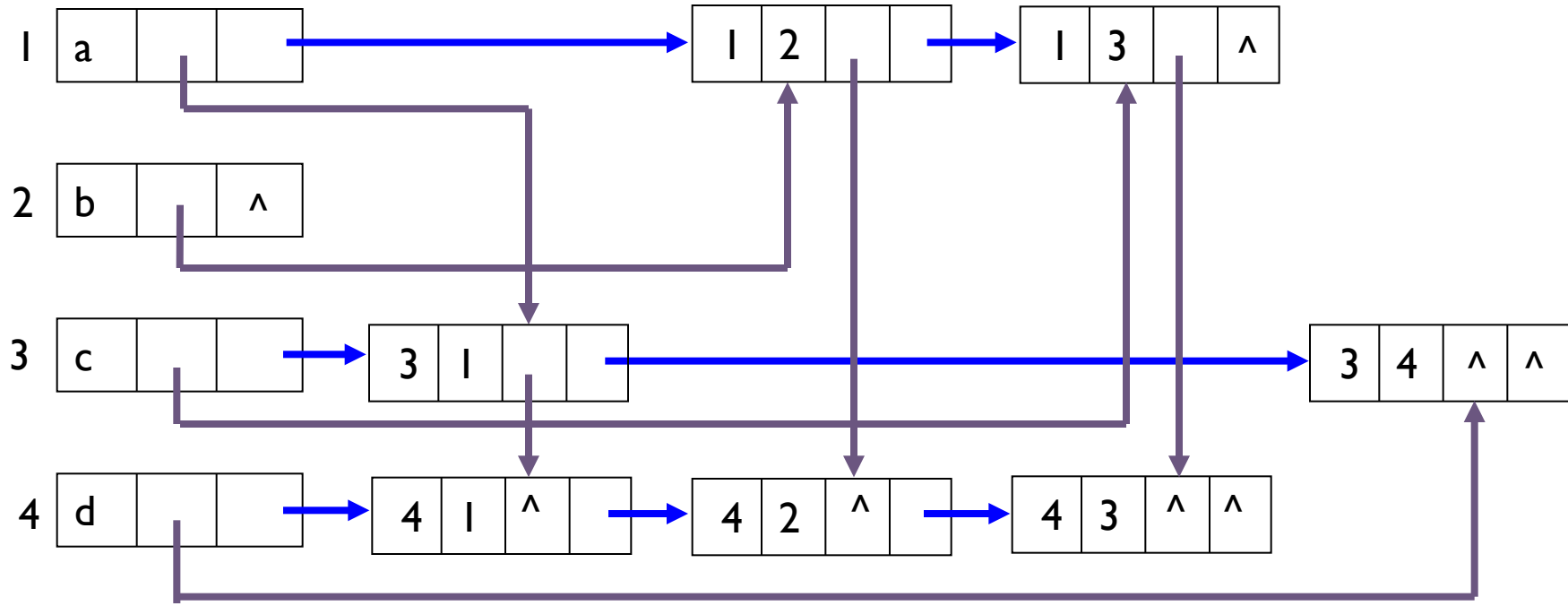
```
typedef struct {
   VexNode xlist[MAX_VERTEX_NUM];
   int vexnum, arcnum;
} OLGraph;
```


華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

有向图的十字链表表示法

示例





建立十字链表算法代码

```
Status CreateDG(OLGraph &G) {
 //采用十字链表存储表示,构造有向图G(G.kind =DG)
 scanf(&G.vexnum,&G.arcnum,&IncInfo); // IncInfo为零，则各弧不含其它信息
 for(i=0; i<G.vexnum;++i){          //构造表头向量
      scanf(&G.xlist[i].data);        //输入顶点值
      G.xlist[i].firstin = NULL; G.xlist[i].firstout = NULL; //初始化指针
 }
 for(k=0;k<G.arcnum;++k){//输入各弧并构造十字链表
    scanf(&v1,&v2);          //输入一条弧的起点与终点
    i=LocateVex(G,v1); j=LocateVex(G,v2);  //确定v1和v2在G中的位置
     p=(ArcBox*) malloc(sizeof(ArcBox));   //假定有足够空间
     *p= {i,j,G.xlist[j].firstin,G.xlist[i].firstout,NULL}  //对弧结点赋值
     //{tailvex,headvex,hlink,tlink,info}
     G.xlist[j].firstin = G.xlist[i].firstout =p ;  //完成在入弧和出弧链头的插入
     if (Incinfo) Input(*p->info);                //若弧含有相关信息，则输入
   }
}//CreateDG
```


華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

无向图的邻接多重表表示法

```
#define MAX_VERTEX_NUM 20
#define enum {unvisited, visited} VisitIf;
typedef  struct  EBox {        //边结点
   VisitIf  mark;    //标志域
   int  ivex, jvex;  //该边依附的两个顶点在表头数组中位置
   struct  EBox  *ilink, *jlink; //分别指向依附于ivex和jvex的下一条边
   InfoType *info;
}EBox;
```

mark	ivex	ilink	jvex	jlink

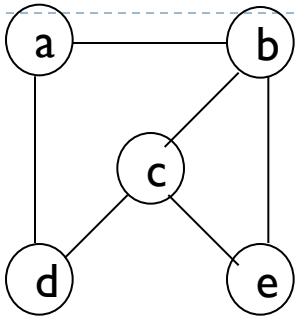
```
typedef  struct  VexNode {   //顶点结点
   VertexType data;  //存与顶点有关的信息
   EBox  *firstedge; //指向第一条依附于该顶点的边
}VexBox;
```

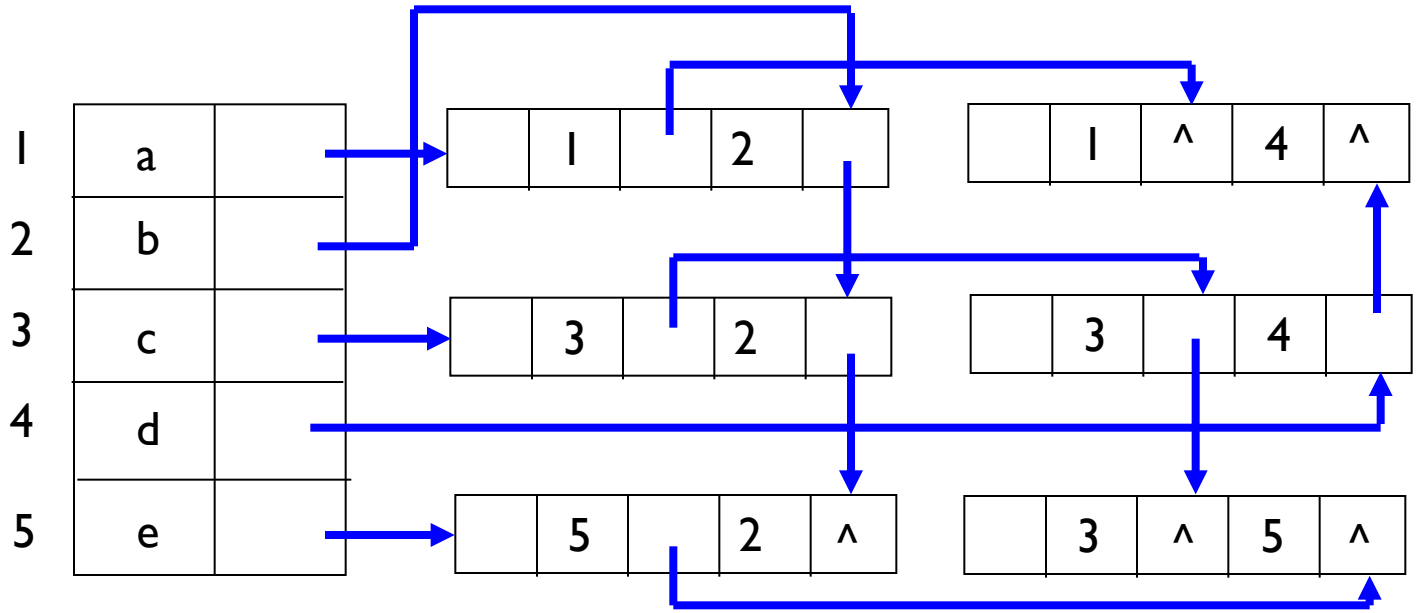
data	firstedge

```
typedef struct {
   VexBox adjmulist[MAX_VERTEX_NUM];
   int vexnum, edgenum;
} AMLGraph;
```


華中科技大学 人工智能与自动化学院

示例





華中科技大學
人工智能与自动化学院

小结

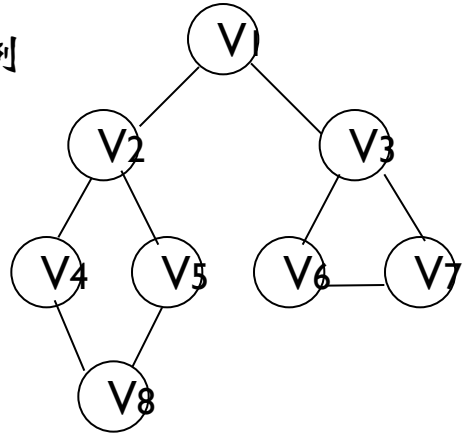
- 图的表示类型
  - 邻接矩阵表示（其中无向图为对称矩阵）
  - 邻接表表示：类似稀疏矩阵的行链表
  - 有向图的十字链表：与稀疏矩阵构建十字链表时相比，其插入无特殊情况（即分为头结点或中间结点）
  - 邻接多重表：无向图的链式存储结构，解决邻接表中一条边的两个结点位于两个不同链表的问题

華中科技大学 人工智能与自动化学院
A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

7.3 图的遍历

- 深度优先遍历(DFS)
  - 从图的某一顶点V0出发，访问此顶点；然后依次从V0的未被访问的邻接点出发，深度优先遍历图，直至图中所有和V0相通的顶点都被访问到；若此时图中尚有顶点未被访问，则另选图中一个未被访问的顶点作起点，重复上述过程，直至图中所有顶点都被访问为止

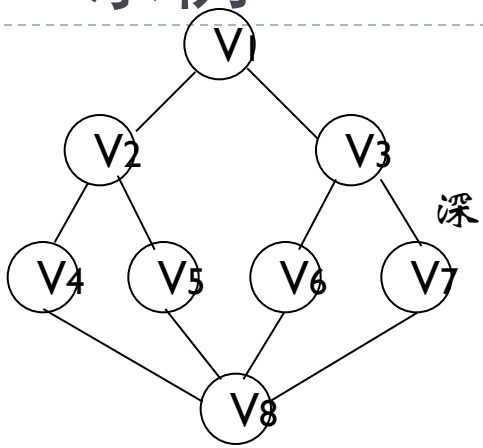
例



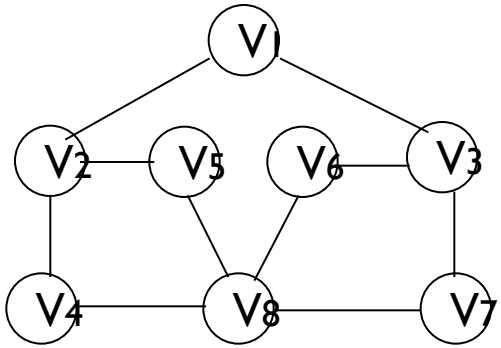
深度遍历：V1⇒V2 ⇒V4 ⇒V8 ⇒V5 ⇒V3 ⇒V6 ⇒V7

華中科技大學 人工智能与自动化学院

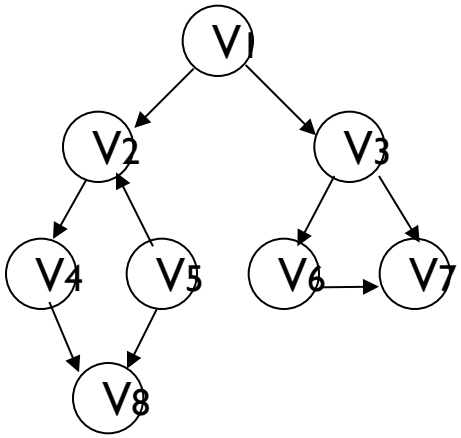
7.3 示例



深度遍历：V1⇒V2 ⇒V4 ⇒V8 ⇒V5 ⇒V6 ⇒V3 ⇒V7



深度遍历：V1⇒V2 ⇒V4 ⇒V8 ⇒V5 ⇒V6 ⇒V3 ⇒V7

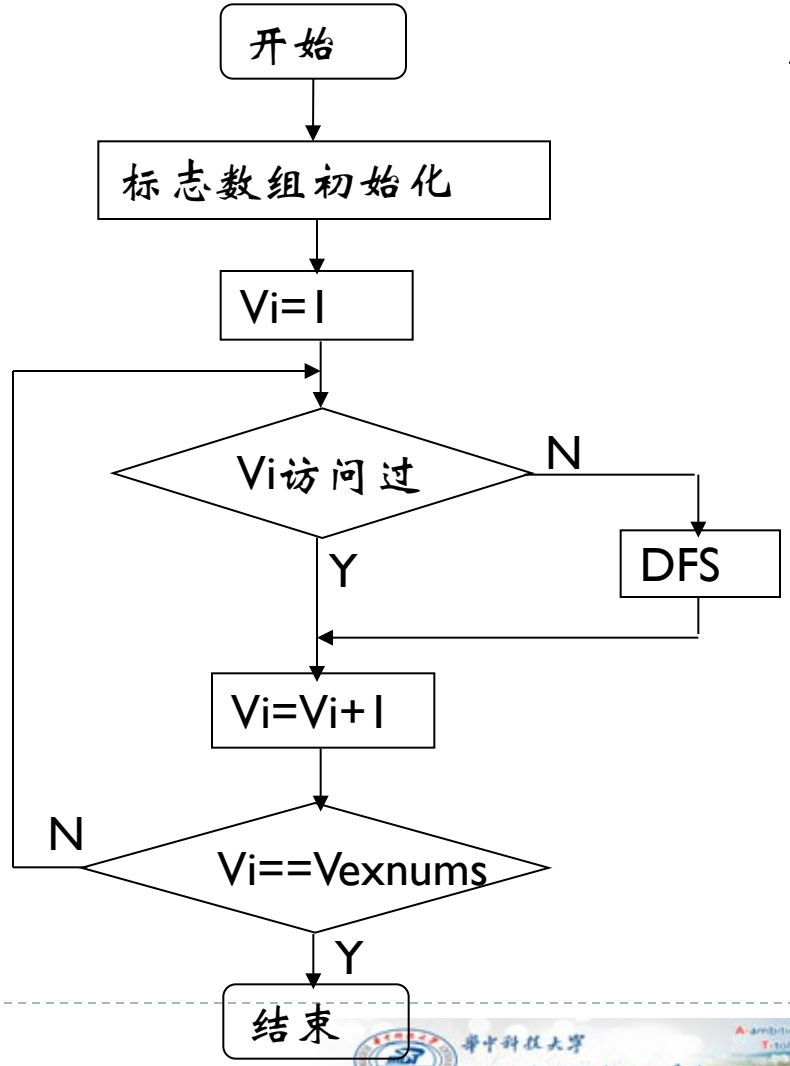


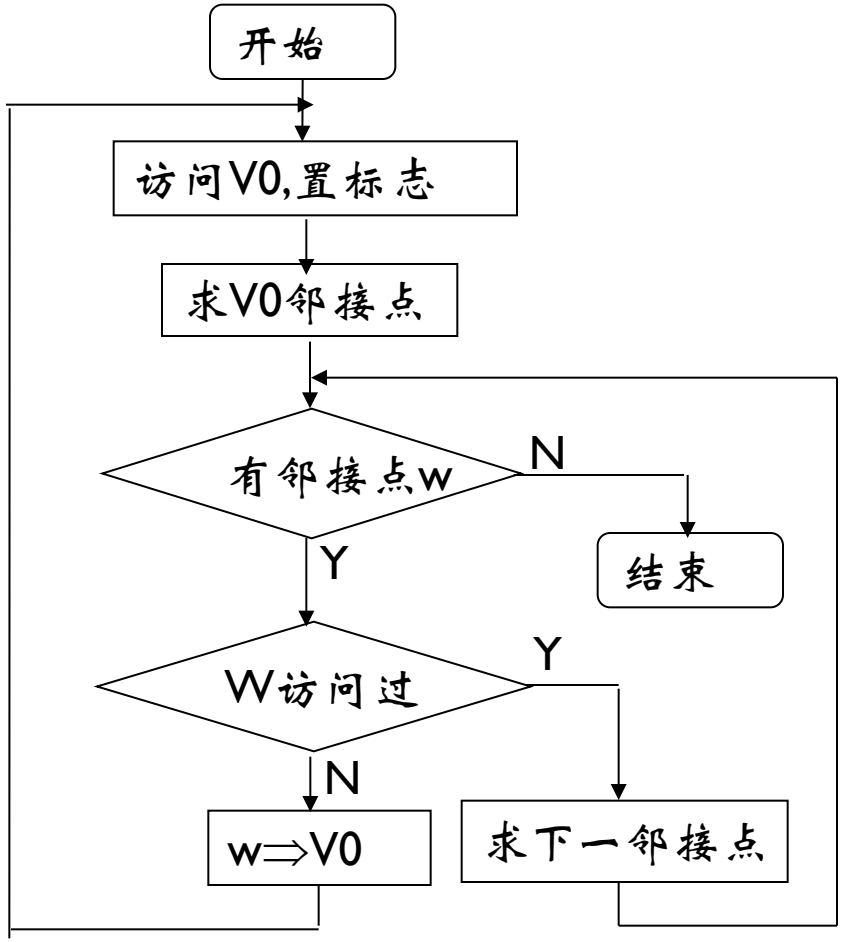
深度遍历：V1⇒V2 ⇒V4 ⇒V8 ⇒V3 ⇒V6 ⇒V7 ⇒V5

華中科技大學 人工智能与自动化学院

深度优先遍历算法

- 递归算法





DFS

華中科技大學 人工智能与自动化学院

深度优先遍历算法代码

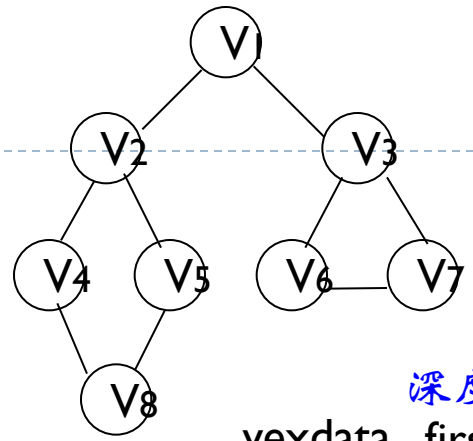
```
Boolean Visited[MAX]; //访问标志数组
Status (*VisitFunc)(int v);

void DFSTraverse(Graph G, Status (*Visit) (int v)) {//对图G作深度优先遍历
    VisitFunc = Visit;          // 使用全局变量，DFS不必设函数指针参数
    for(v=0; v<G.vexnum;++v) Visited[v] = FALSE;        //访问标志数组初始化
    for(v=0; v<G.vexnum;++v)                            //
     if (!Visited[v]) DFS(G,v);                         //对尚未访问的顶点调用DFS
 }
 void DFS(Graph G, int v) {
    // 从第v个顶点出发递归地深度优先遍历图G
     visited[v] = TRUE; VisitedFunc(v);    //访问第v个顶点
     for (w = FirstAdjVex(G,v); w>=0; w = NextAdjVex(G,v,w))
        if (!visited[w]) DFS(G,w);          //对v尚未访问的邻接顶点w递归调用DFS
   }
}
```

若顶点数为n,边数为e，则

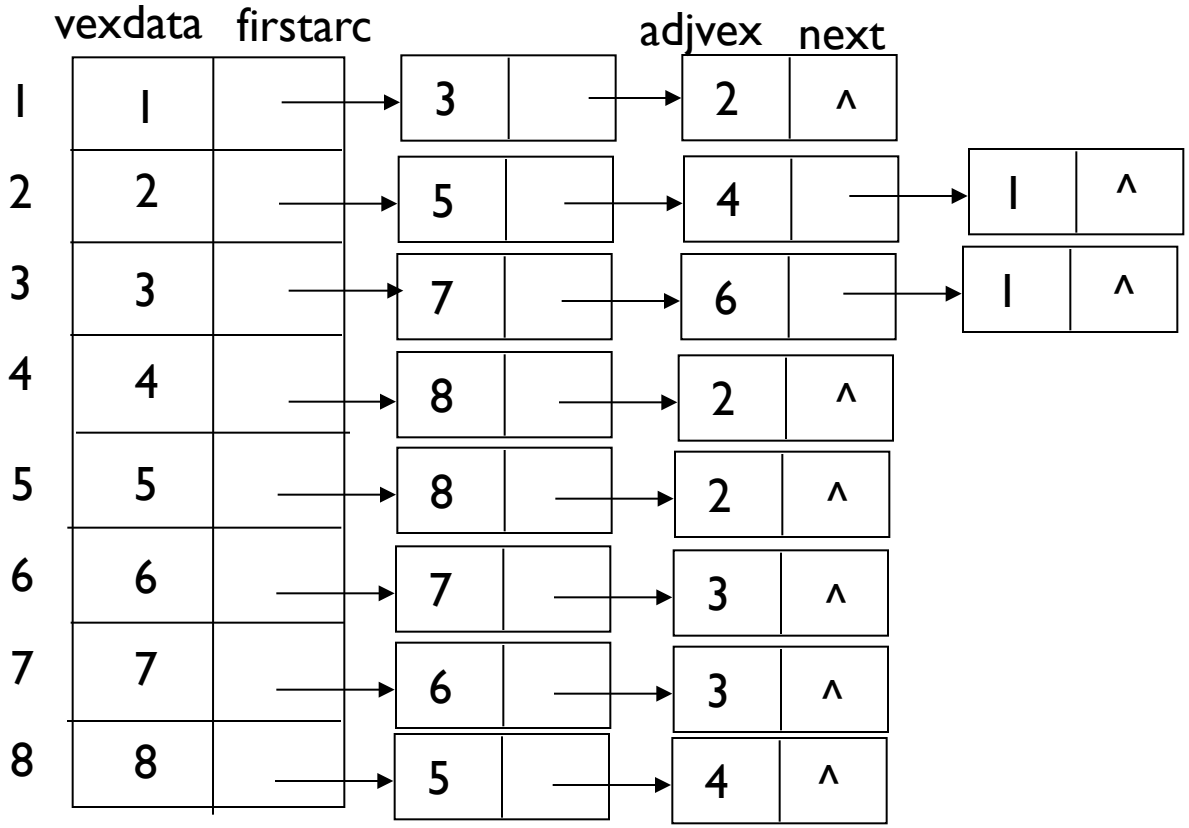
- 若使用邻接矩阵，则查找每个顶点的时间为$O(n^2)$
- 若使用邻接表， 则找邻接点的时间为O(e)，故时间复杂度为$O(n+e)$

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

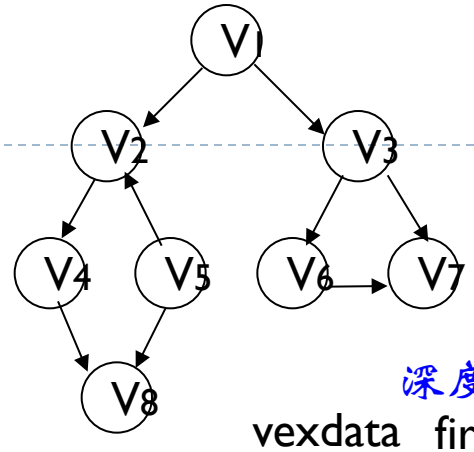
例



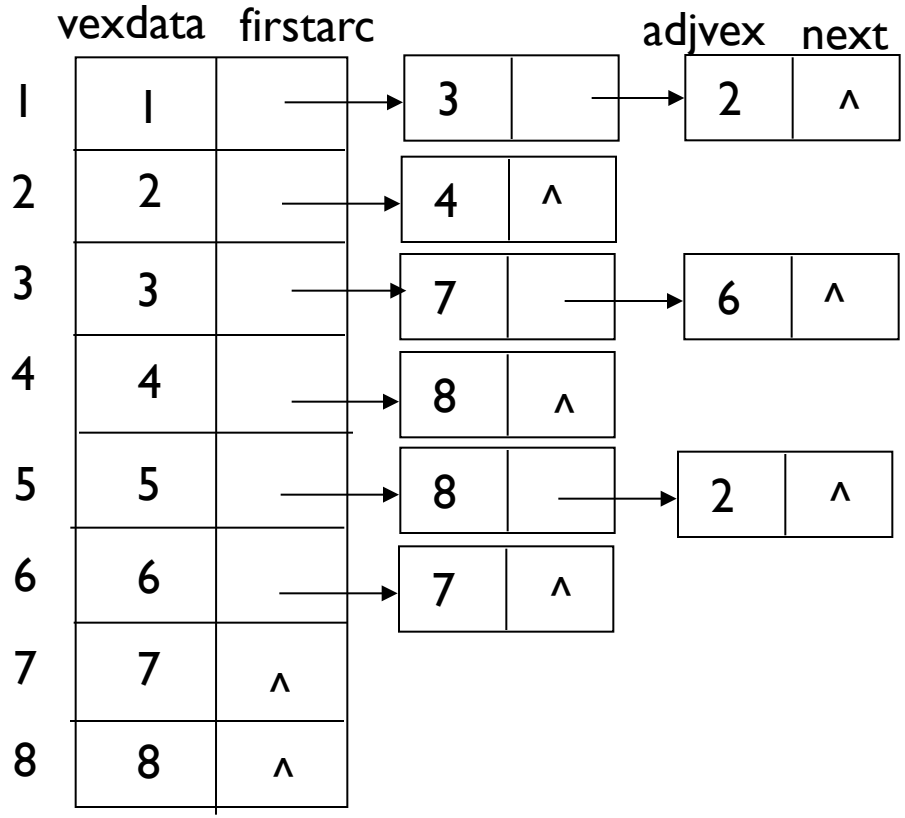
深度遍历：V1⇒ V3 ⇒ V7 ⇒ V6 ⇒ V2 ⇒ V5 ⇒ V8 ⇒ V4



例

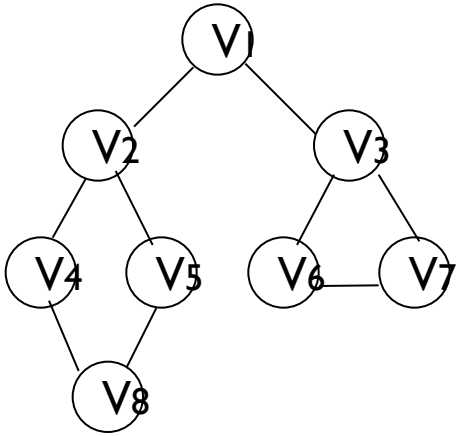


深度遍历：V1⇒ V3 ⇒ V7 ⇒ V6 ⇒ V2 ⇒ V4 ⇒ V8 ⇒ V5



图的遍历

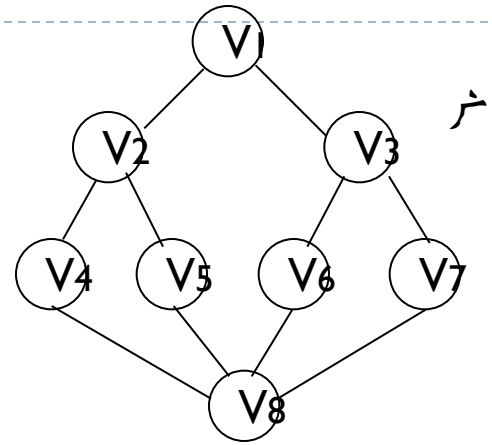
- 广度优先遍历(BFS)
  - 从图的某一顶点V0出发，访问此顶点后，依次访问V_0的各个未曾访问过的邻接点；然后分别从这些邻接点出发，广度优先遍历图，直至图中所有已被访问的顶点的邻接点都被访问到；若此时图中尚有顶点未被访问，则另选图中一个未被访问的顶点作起点，重复上述过程，直至图中所有顶点都被访问为止



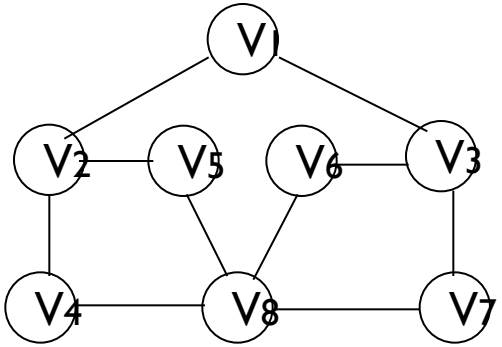
广度遍历：V1⇒V2 ⇒V3 ⇒V4 ⇒V5 ⇒V6 ⇒V7 ⇒V8

華中科技大學 人工智能与自动化学院

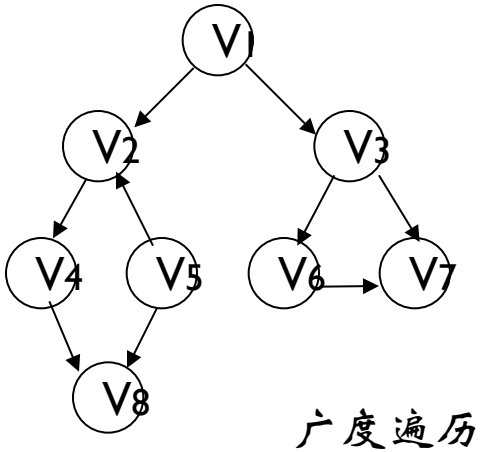
例



广度遍历：V1⇒V2 ⇒V3 ⇒V4 ⇒V5 ⇒V6 ⇒V7 ⇒V8



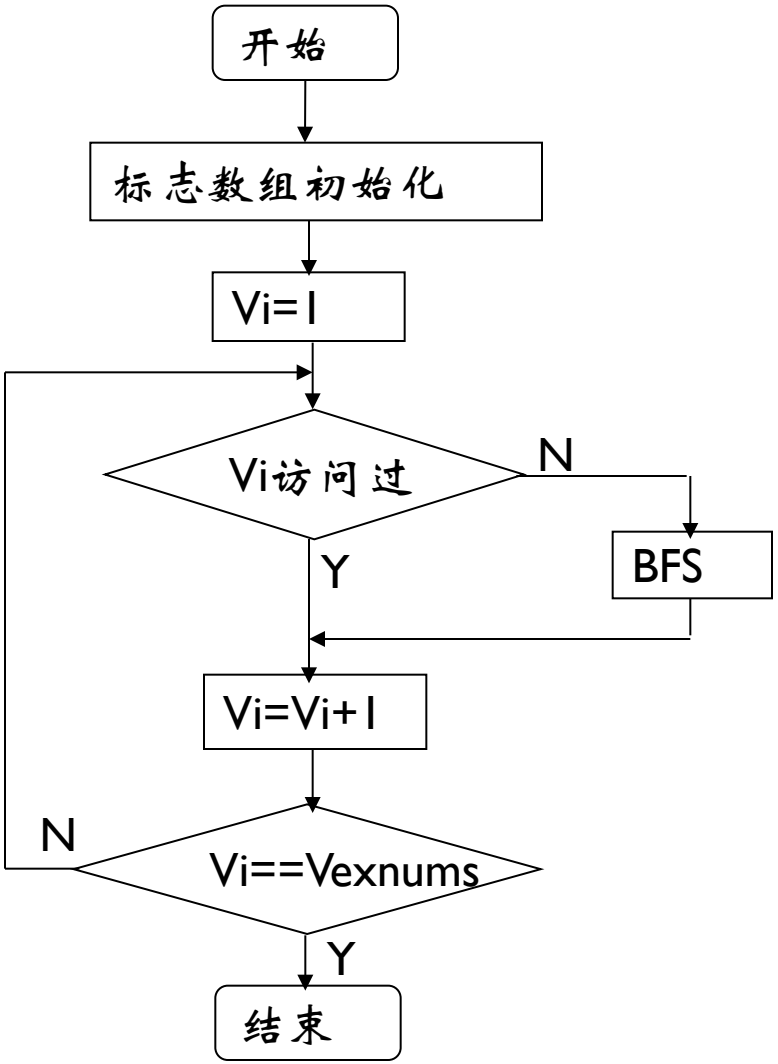
广度遍历：V1⇒V2 ⇒V3 ⇒V4 ⇒V5 ⇒V6 ⇒V7 ⇒V8



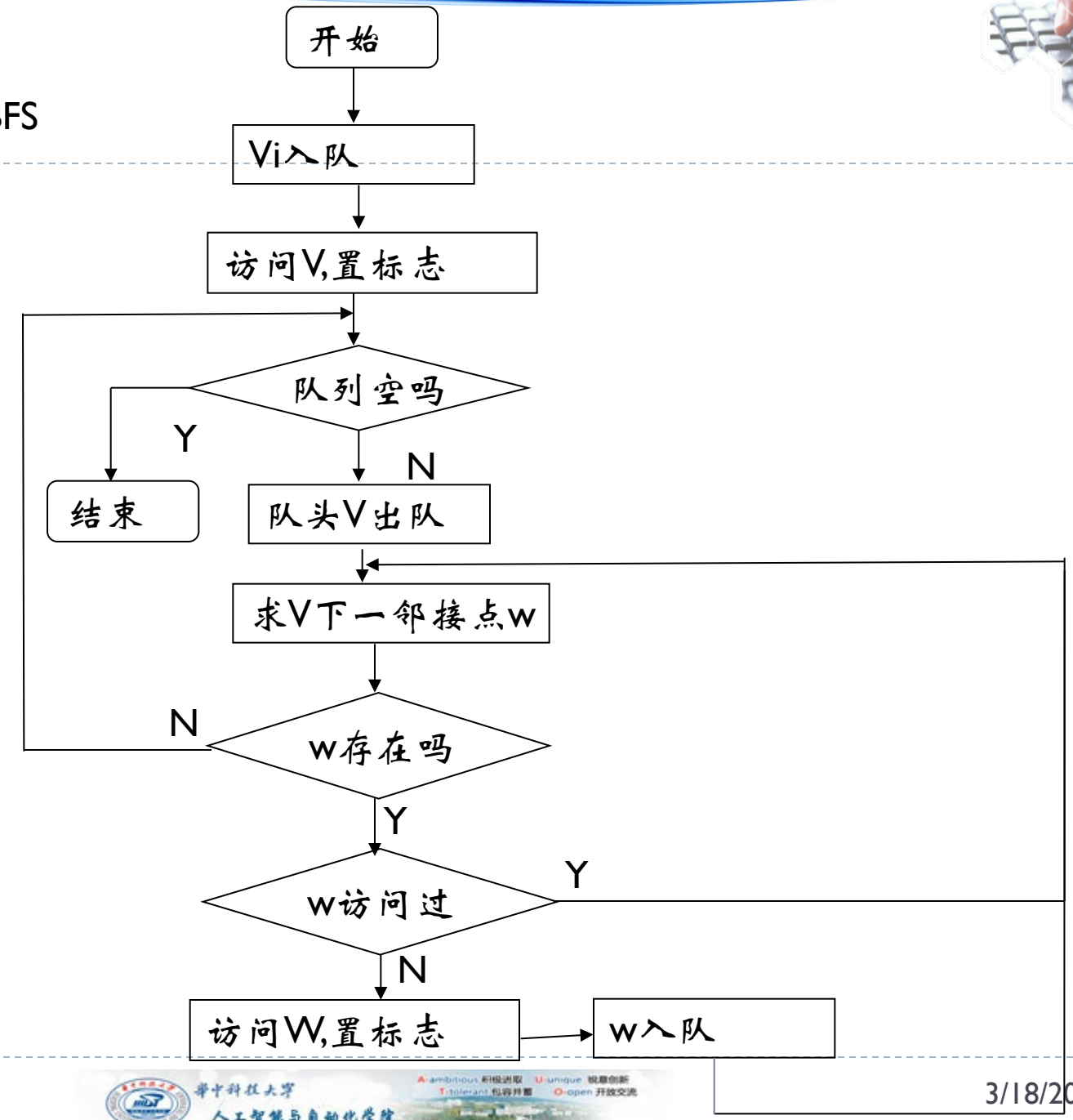
广度遍历：V1⇒V2 ⇒V3 ⇒V4 ⇒V6 ⇒V7 ⇒V8 ⇒V5

華中科技大學
人工智能与自动化学院
A-ambitious 积极进取 U-unique 锐意创新
T-tolerant 包容并蓄 O-open 开放交流

广度优先遍历算法-算法7.6



BFS



華中科技大學
人工智能与自动化学院
A-ambitious 积极进取 U-unique 锐意创新
T-tolerant 包容并蓄 O-open 开放交流

广度优先遍历算法代码

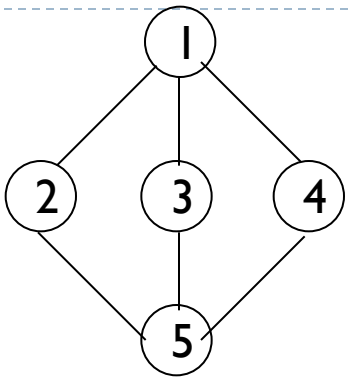
```
Boolean visited[MAX]; //访问标志数组
Queque Q;

void BFSTraverse(Graph G, Status (*Visit) (int v)) {//对图G作广度优先遍历
    for(v=0; v<G.vexnum;++v) Visited[v] = FALSE;    //置空辅助队列Q
    InitQueque(Q);
    for(v=0; v<G.vexnum;++v)
        if (!visited[v]) {                          //v尚未访问
            visited[v] = TRUE; Visit(v);
            EnQueque (Q, v);                        // v入队列
            while (!QueueEmpty (q)) {
              DeQueue(Q, u);                        //队头元素出队并置为u
              for (w = FirstAdjVex(G,u); w>=0; w = NextAdjVex(G,u,w))
                if (! Visited [w]) {                // w为u的尚未访问的邻接顶点
                  Visited [w] = TRUE; Visit (w);
                  EnQueue(Q,w);
                } //if
            }//while
        }//if
}
```

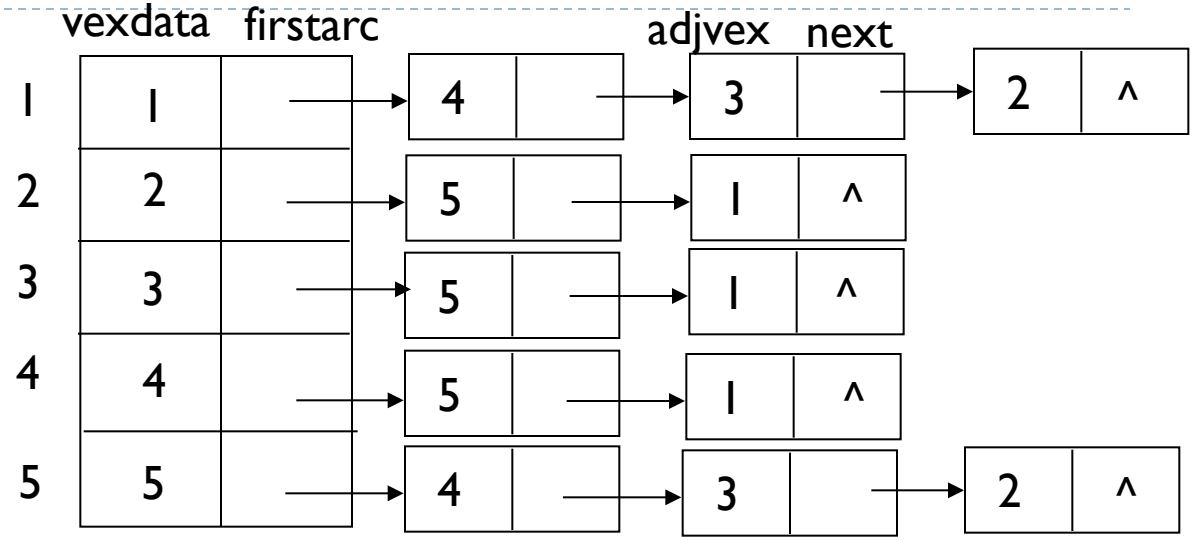
若顶点数为n,边数为e，则

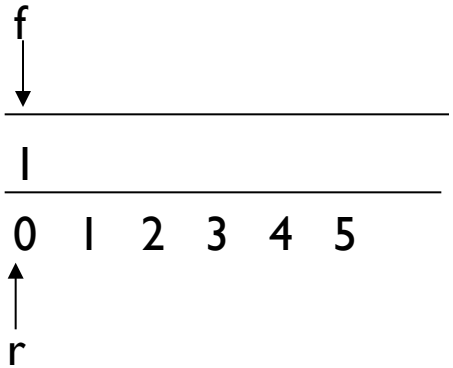
- 若使用邻接矩阵，则查找每个顶点的时间为$O(n^2)$
- 若使用邻接表， 时间复杂度为$O(n+e)$

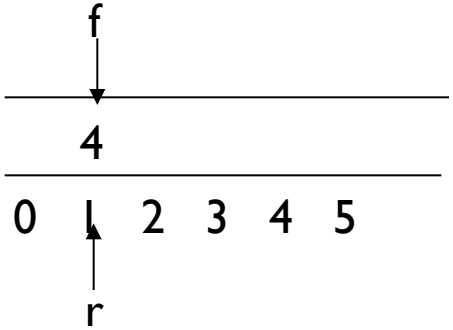
華中科技大學 人工智能与自动化学院

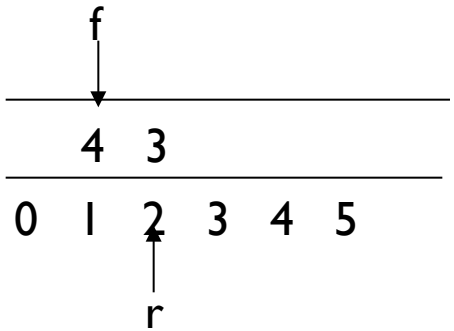
例











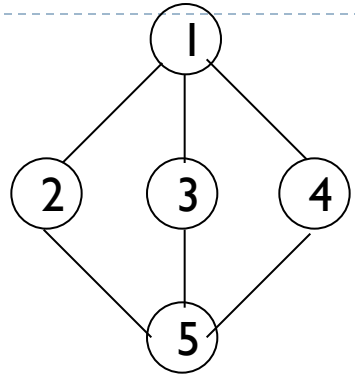
遍历序列：1

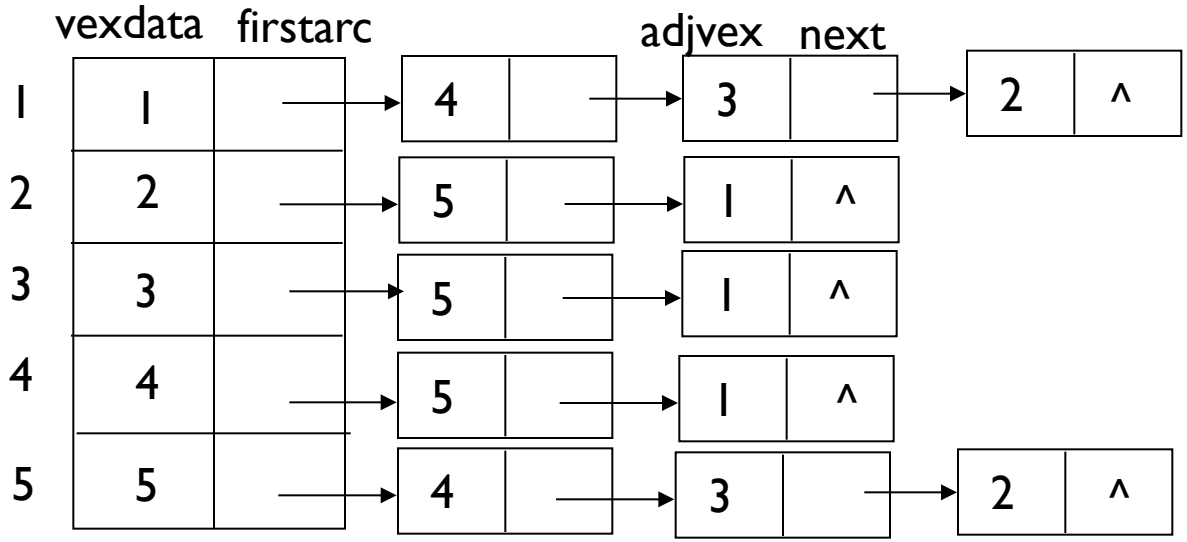
遍历序列：1 4

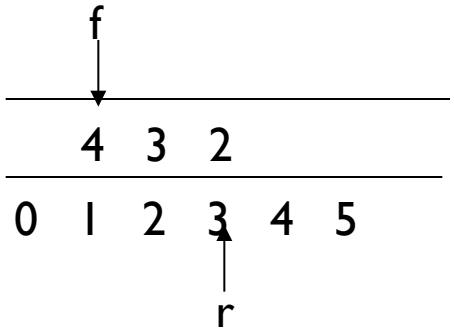
遍历序列：1 4 3

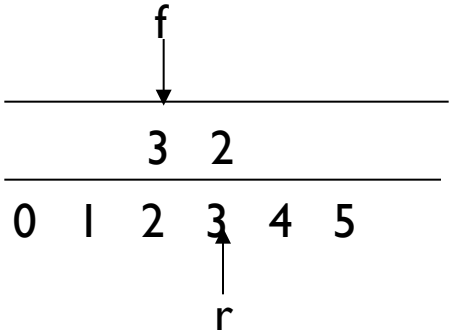
華中科技大學
人工智能与自动化学院

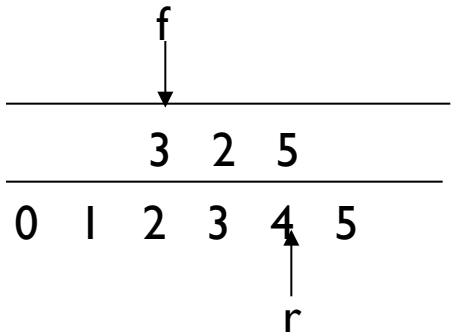
例











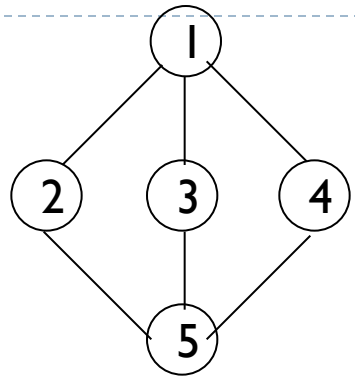
遍历序列：1 4 3 2

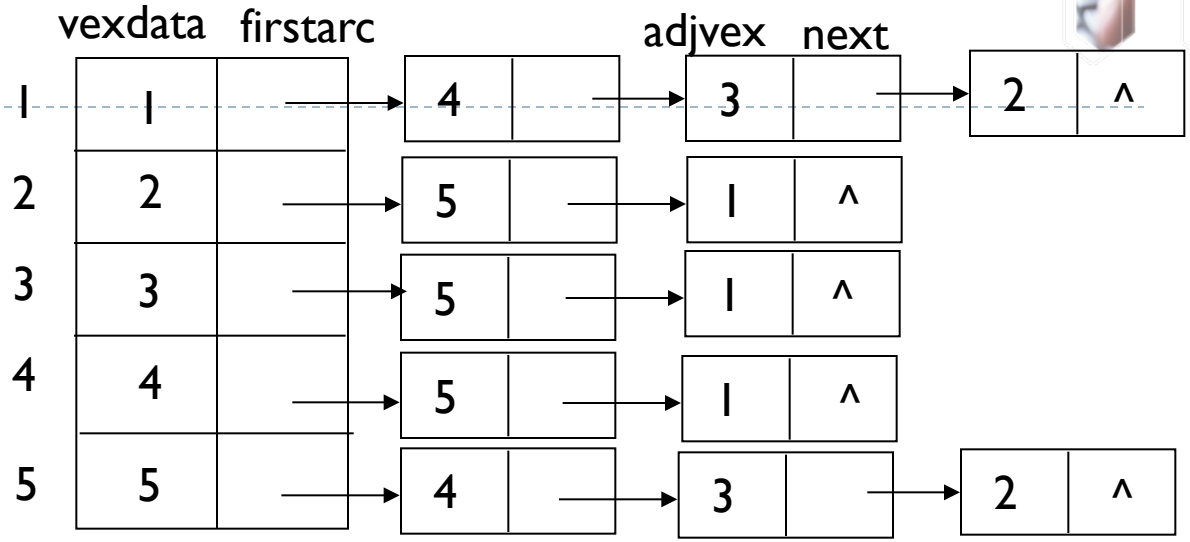
遍历序列：1 4 3 2

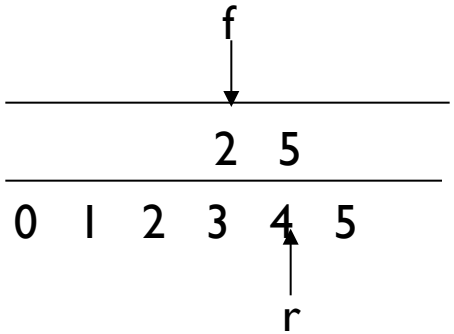
遍历序列：1 4 3 2 5

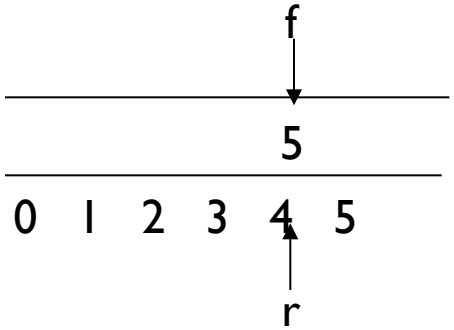
華中科技大學
人工智能与自动化学院

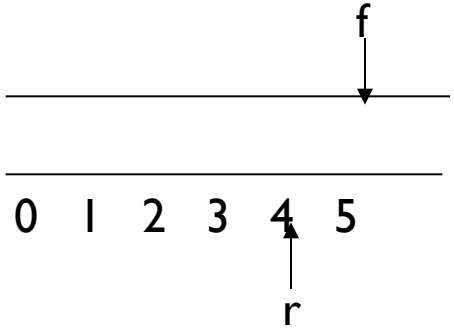
例











遍历序列：1 4 3 2 5

遍历序列：1 4 3 2 5

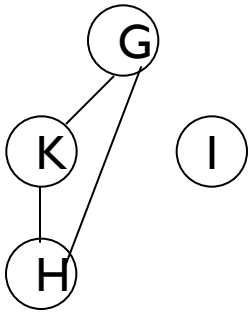
遍历序列：1 4 3 2 5

華中科技大學
人工智能与自动化学院
A-ambitious 积极进取 U-unique 锐意创新
T-tolerant 包容并蓄 O-open 开放交流

7.4 生成树

- 所有顶点均由边连接在一起，但不存在回路的图:深度优先生成树与广度优先生成树
- 生成森林：非连通图每个连通分量的生成树一起组成非连通图的~

✧ 说明　一个图可以有许多棵不同的生成树



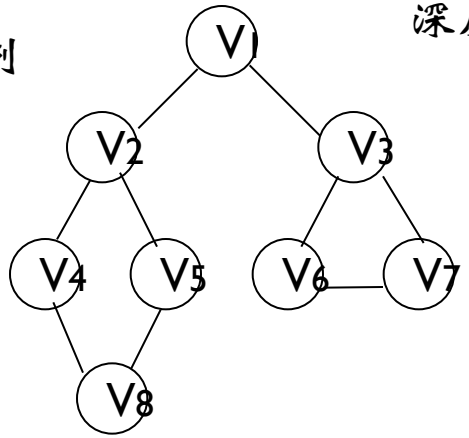
- 所有生成树具有以下共同特点：
  - 生成树的顶点个数与图的顶点个数相同
  - 生成树是图的极小连通子图
  - 一个有n个顶点的连通图的生成树有n-1条边
  - 生成树中任意两个顶点间的路径是唯一的
  - 在生成树中再加一条边必然形成回路
- 含n个顶点n-1条边的图不一定是生成树

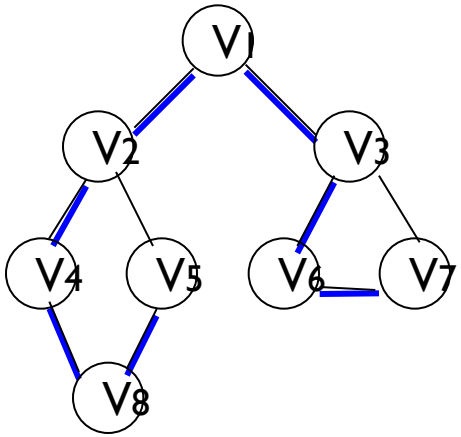
華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

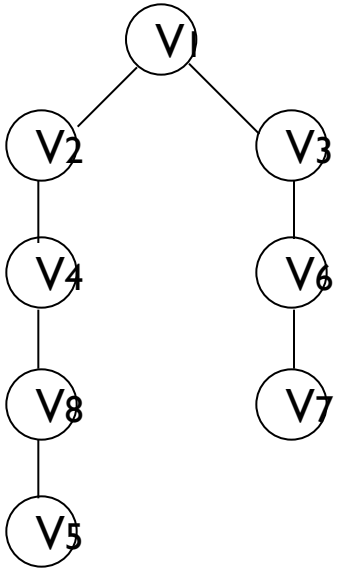
广度遍历：V1⇒V2 ⇒V3 ⇒V4 ⇒V5 ⇒V6 ⇒V7 ⇒V8

深度遍历：V1⇒V2 ⇒V4 ⇒V8 ⇒V5 ⇒V3 ⇒V6 ⇒V7

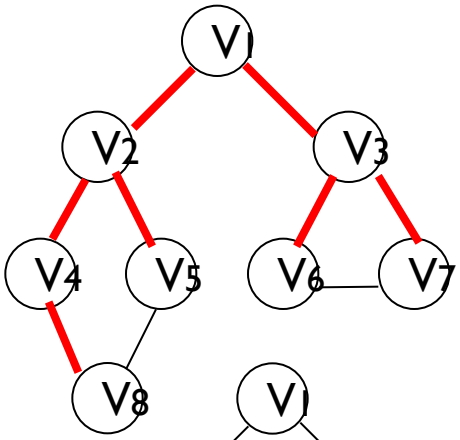
例

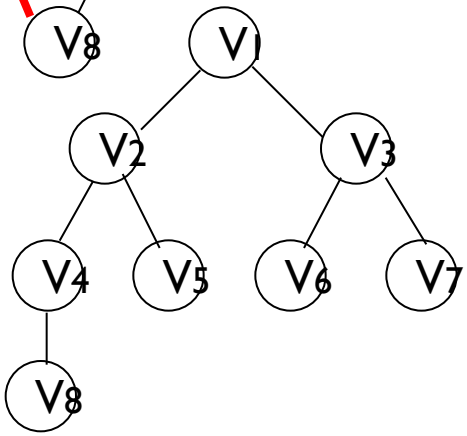






深度优先生成树

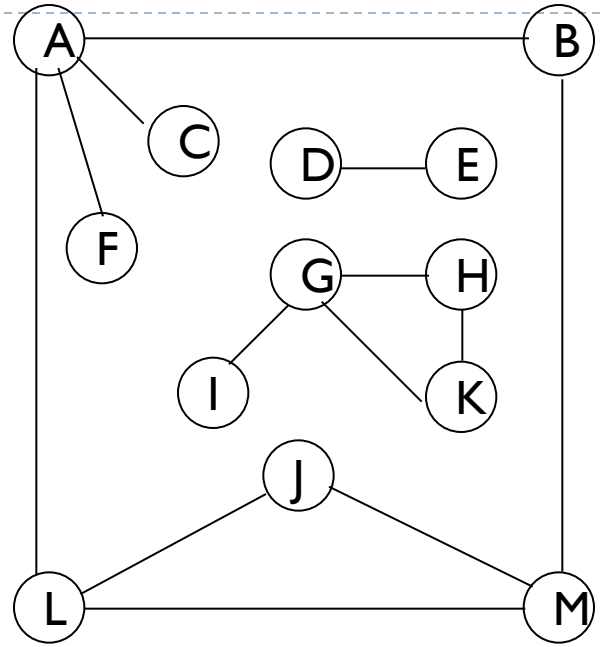


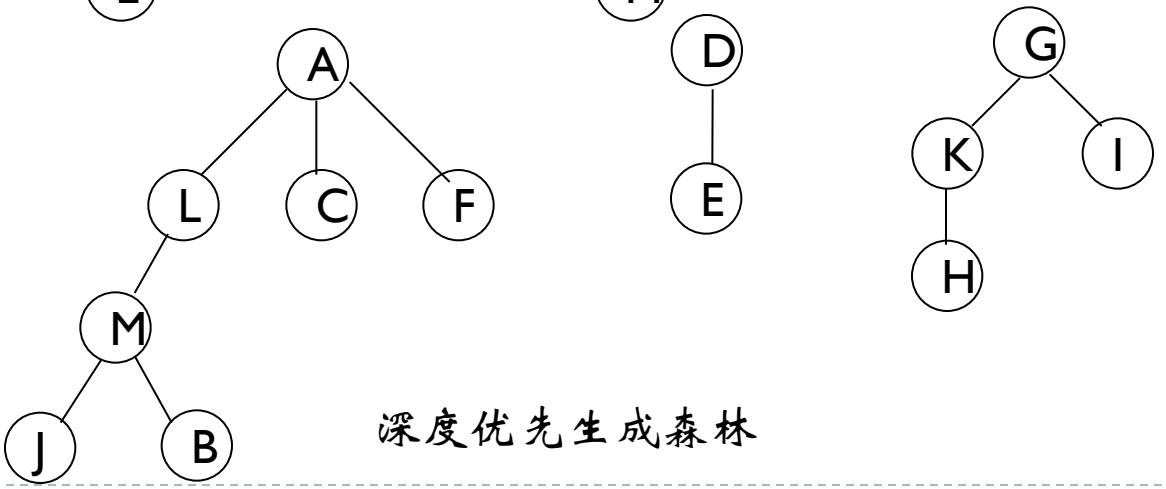


广度优先生成树

華中科技大學
人工智能与自动化学院
A-ambitious 积极进取 U-unique 锐意创新
T-tolerant 包容并蓄 O-open 开放交流

例

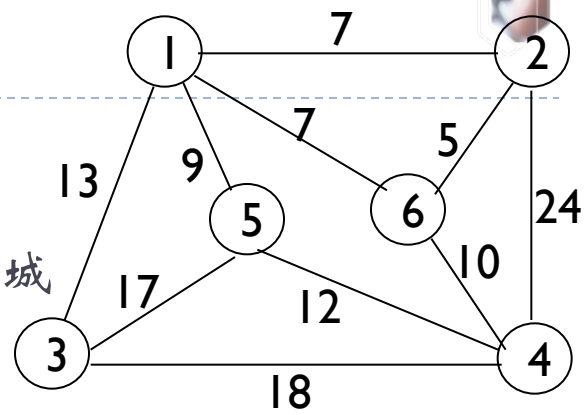




深度优先生成森林

華中科技大學
人工智能与自动化学院
A-ambitious 积极进取 U-unique 锐意创新
T-tolerant 包容并蓄 O-open 开放交流

最小生成树



- **问题提出**
  - 要在n个城市间建立通信联络网，顶点——表示城市，权——城市间建立通信线路所需花费代价。希望找到一棵生成树，它的每条边上的权值之和（即建立该通信网所需花费的总代价）最小——最小代价生成树。
- **问题分析**
  - n个城市间，最多可设置n(n-1)/2条线路,n个城市间建立通信网，只需n-1条线路,问题转化为：如何在可能的线路中选择n-1条，能把所有城市（顶点）均连起来，且总耗费（各边权值之和）最小。

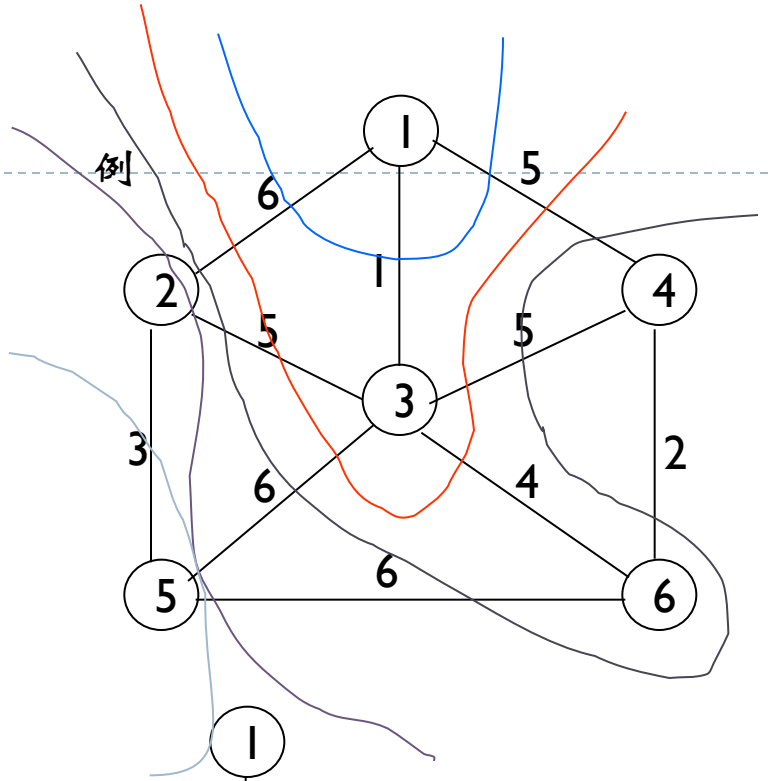
華中科技大學 人工智能与自动化学院

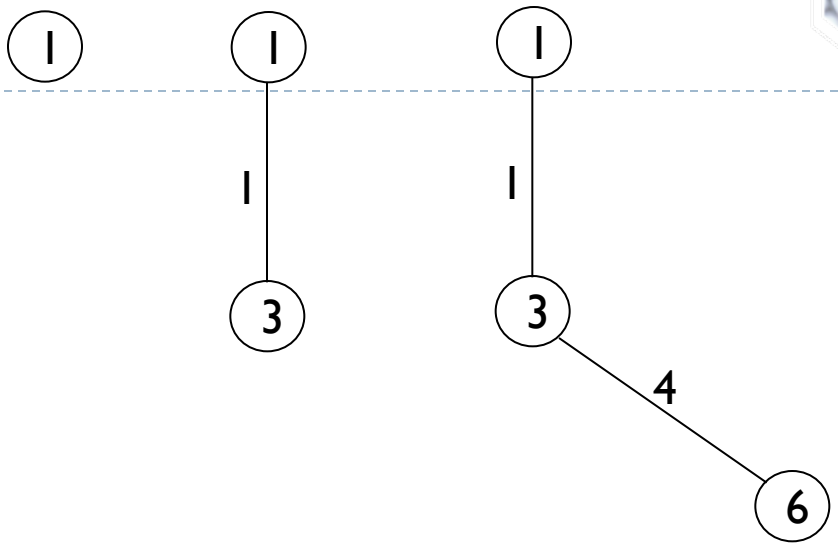
构造最小生成树方法

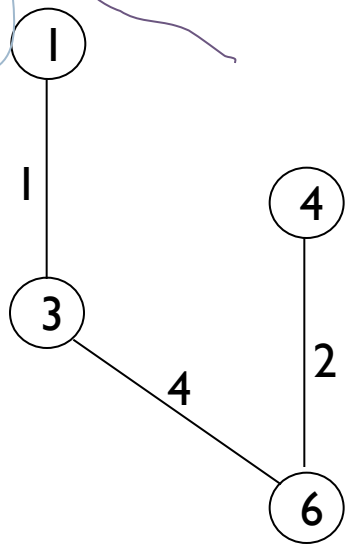
- 方法一：普里姆(Prim)算法
  - 算法思想：设N=(V,{E})是连通网，TE是N上最小生成树中边的集合
  - 初始令U={u0},(u0∈V),TE=Φ
  - 在所有u∈U,v∈V-U的边(u,v)∈E中，找一条代价最小的边(u0,v0)
  - 将(u0,v0)并入集合TE，同时v0并入U
  - 重复上述操作直至U=V为止，则T=(V,{TE})为N的最小生成树
  - 算法实现：图用邻接矩阵表示
  - 算法描述
  - 算法评价：$T(n)=O(n^2)$

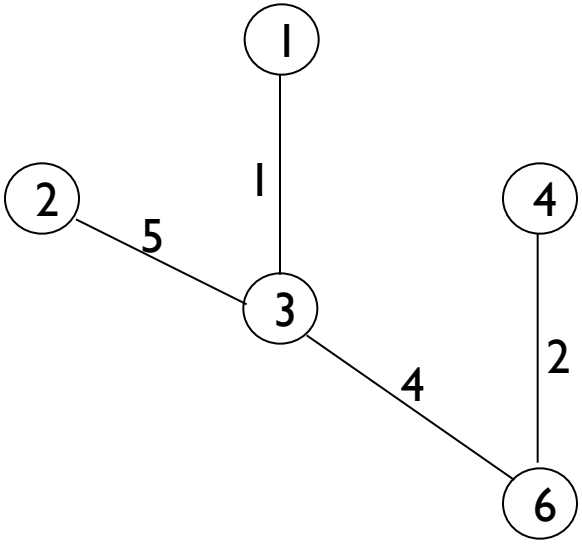
華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

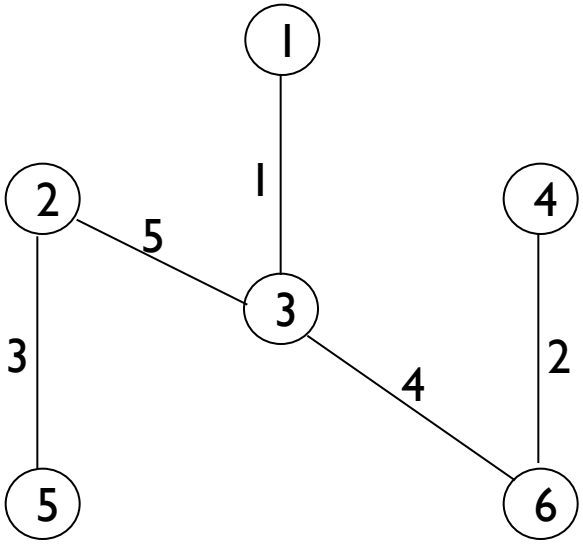
例





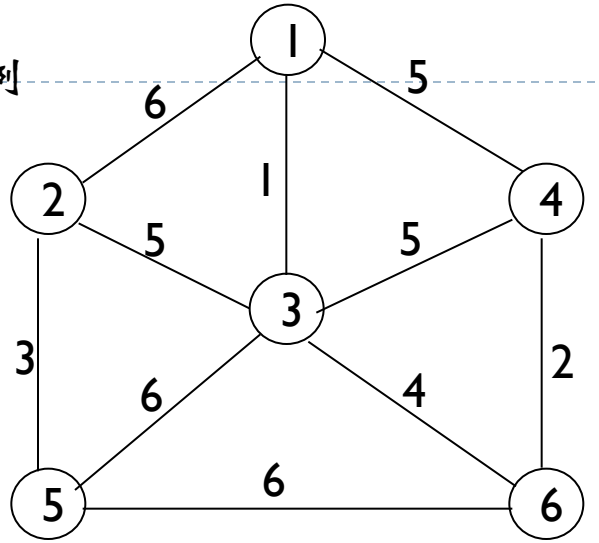






華中科技大学
人工智能与自动化学院
A-ambitious 积极进取 U-unique 独具创新
T-tolerant 包容并蓄 O-open 开放交流

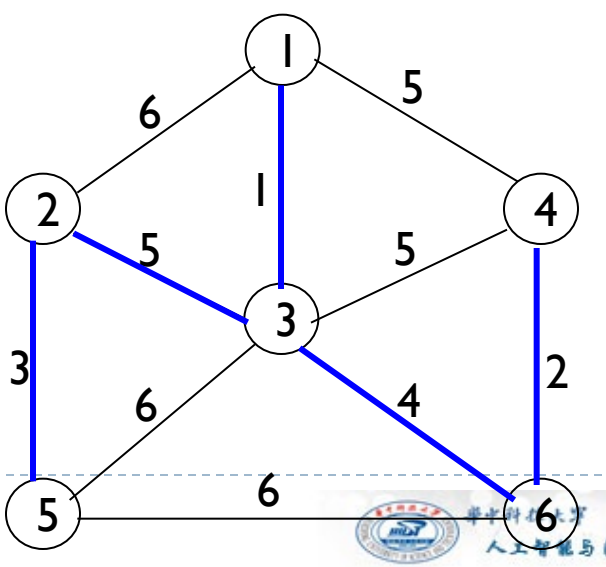
例

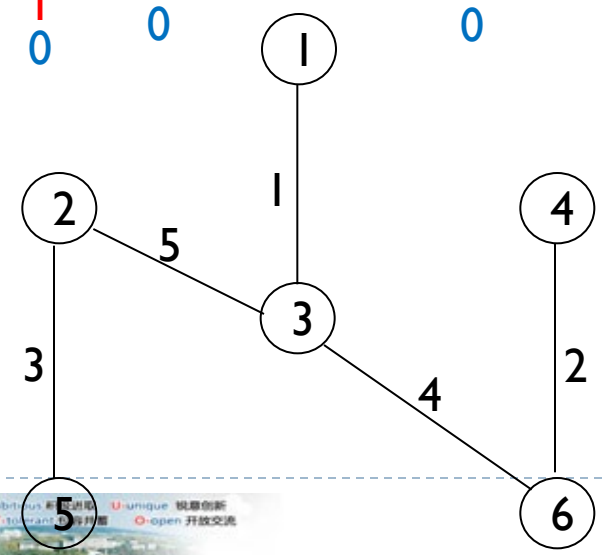


$$
\begin{array}{c|cccccc}
 & 1 & 2 & 3 & 4 & 5 & 6 \\
\hline
1 & 0 & 6 & 1 & 5 & \infty & \infty \\
2 & 6 & 0 & 5 & \infty & 3 & \infty \\
3 & 1 & 5 & 0 & 5 & 6 & 4 \\
4 & 5 & \infty & 5 & 0 & \infty & 2 \\
5 & \infty & 3 & 6 & \infty & 0 & 6 \\
6 & \infty & \infty & 4 & 2 & 6 & 0
\end{array}
$$

closedge[i].lowcost

1	2	3	4	5	6
~~∞~~ 5	~~∞~~ 5	~~4~~ 0	~~2~~ 0	6	0
0	0			0	





普里姆算法实现

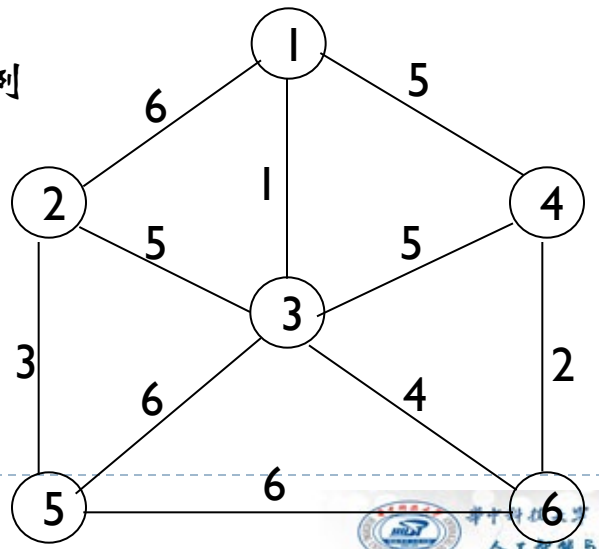
```
void MiniSpanTree_PRIM( MGraph G, VertexType u ) {
  // 用普里姆算法从第 u 个顶点出发构造网 G 的最小生成树 T,输出 T 的各条边。
  // 记录从顶点集 U 到 V-U 的代价最小的边的辅助数组定义:
  //   struct {
  //       VertexType  adjvex;
  //       VRType      lowcost;
  //   }closedge[MAX_VERTEX_NUM];
  k = LocateVex ( G, u );
  for ( j=0; j<G.vexnum; ++j )      // 辅助数组初始化
     if (j!=k)  closedge[j] = { u, G.arcs[k][j].adj };   // {adjvex, lowcost }
  closedge[k].lowcost = 0;                // 初始,U={u}
  for (i=1; i<G.vexnum; ++i) {            // 选择其余 G.vexnum-1 个顶点
     k = minimum(closedge);               // 求出 T 的下一个结点:第 k 顶点
     // 此时 closedge[k].lowcost =
     //              MIN{ closedge[vi].lowcost | closedge[vi].lowcost>0, vi∈V-U }
     printf(closedge[k].adjvex, G.vexs[k]);        // 输出生成树的边
     closedge[k].lowcost = 0;              // 第 k 顶点并入 U 集
     for (j=0; j<G.vexnum; ++j)
        if (G.arcs[k][j].adj < closedge[j].lowcost)        // 新顶点并入 U 后重新选择最
                                                              小边
           closedge[j] = { G.vexs[k], G.arcs[k][j].adj };
  }
}// MiniSpanTree
```

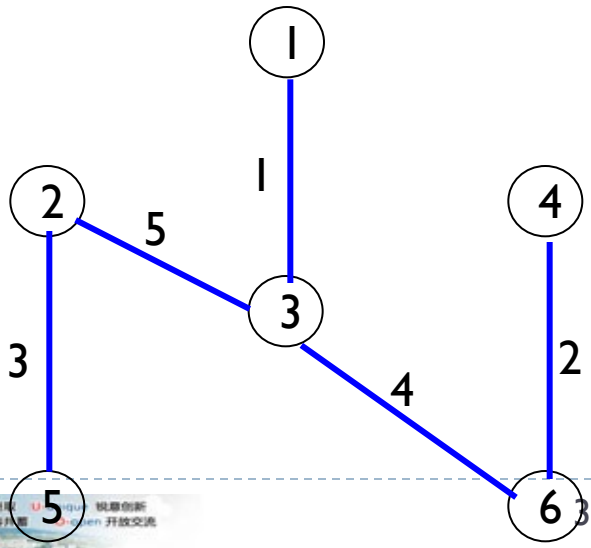
算法 7.9

方法二：克鲁斯卡尔(Kruskal)算法

- 算法思想：设连通网N=(V,{E})，令最小生成树
  - 初始状态为只有n个顶点而无边的非连通图T=(V,{Φ})，每个顶点自成一个连通分量
  - 在E中选取代价最小的边，若该边依附的顶点落在T中不同的连通分量上，则将此边加入到T中；否则，舍去此边，选取下一条代价最小的边
  - 依此类推，直至T中所有顶点都在同一连通分量上为止

例





算法实现

顶点结点：

```
typedef   struct
{    int data;    //顶点信息
     int  jihe;
}VEX;
```

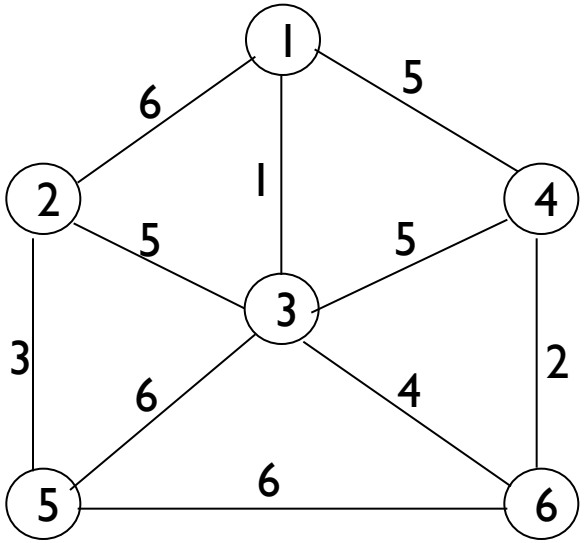
边结点：

```
typedef   struct
{    int   vexh, vext;  //边依附的两顶点
     int  weight;        //边的权值
     int  flag;          //标志域
}EDGE;
```

- 0）用顶点数组和边数组存放顶点和边信息
- 1）初始时，令每个顶点的jihe互不相同；每个边的flag为0
- 2）选出权值最小且flag为0的边
- 3）若该边依附的两个顶点的jihe值不同，即非连通，则令该边的flag=1，选中该边；再令该边依附的两顶点的jihe以及两集合中所有顶点的jihe 相同，若该边依附的两个顶点的jihe值相同，即连通，则令该边的flag=2，即舍去该边
- 4）重复上述步骤，直到选出n-1条边为止
- 计算复杂度：elog(e)

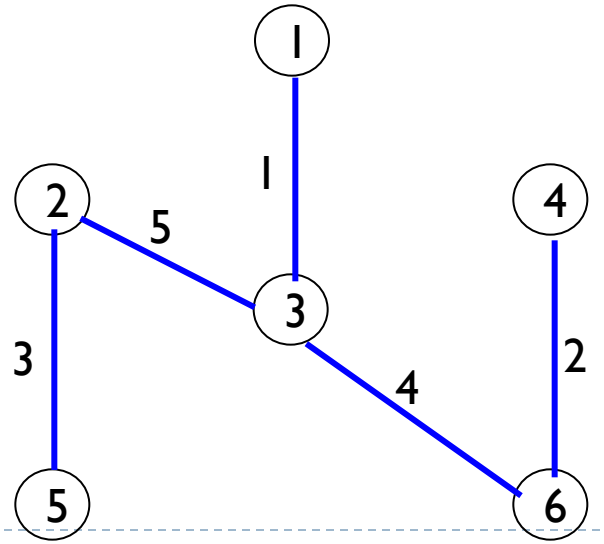
華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

例



	data	jihe
1	1	~~1~~ 2
2	2	2
3	3	~~3~~ 2
4	4	~~4~~ 2
5	5	~~5~~ 2
6	6	~~6~~ 1 2

	vexh	vext	weight	flag
0	1	2	6	0
1	1	3	1	~~0~~
2	1	4	5	~~0~~ 2
3	2	3	5	~~0~~
4	2	5	3	~~0~~
5	3	4	5	0
6	3	5	6	0
7	3	6	4	~~0~~
8	4	6	2	~~0~~
9	5	6	6	0



華中科技大学
人工智能与自动化学院
A-ambitious 积极进取 U-unique 锐意创新
T-tolerant 包容并蓄 O-open 开放交流

7.5 拓扑排序

- 问题提出：学生选修课程问题

  顶点——表示课程

  有向弧——表示先决条件，若课程i是课程j的先决条件，则图中有弧<i,j>

  学生应按怎样的顺序学习这些课程，才能无矛盾、顺利地完成学业——拓扑排序

- 定义
  - AOV网——用顶点表示活动，用弧表示活动间优先关系的有向图称为顶点表示活动的网(Activity On Vertex network)，简称AOV网
  - 若<vi,vj>是图中有向边，则vi是vj的直接前驱；vj是vi的直接后继
  - AOV网中不允许有回路,即某项活动以自己为先决条件

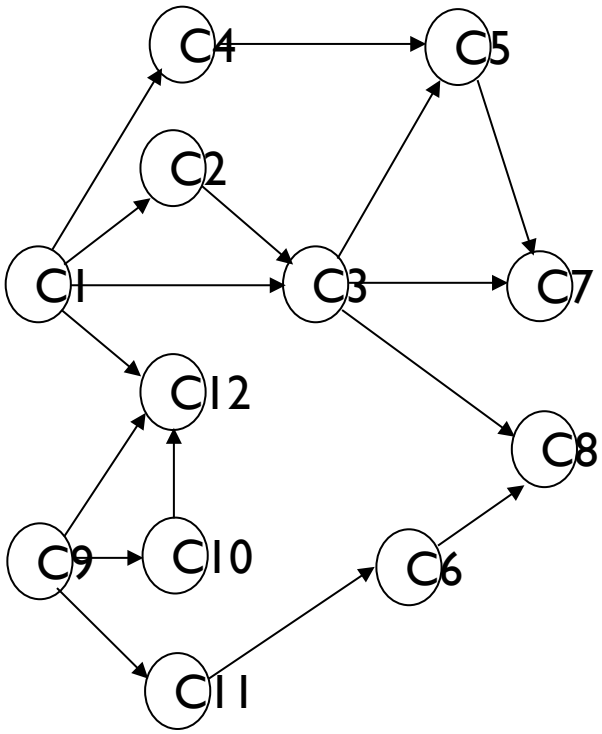
拓扑排序

- 拓扑排序——把AOV网络中各顶点按照它们相互之间的优先关系排列成一个线性序列的过程叫~
  - 检测AOV网中是否存在环方法：对有向图构造其顶点的拓扑有序序列，若网中所有顶点都在它的拓扑有序序列中，则该AOV网必定不存在环
- 拓扑排序的方法
  - 在有向图中选一个没有前驱的顶点且输出之
  - 从图中删除该顶点和所有以它为尾的弧
  - 重复上述两步，直至全部顶点均已输出；或者当图中不存在无前驱的顶点为止

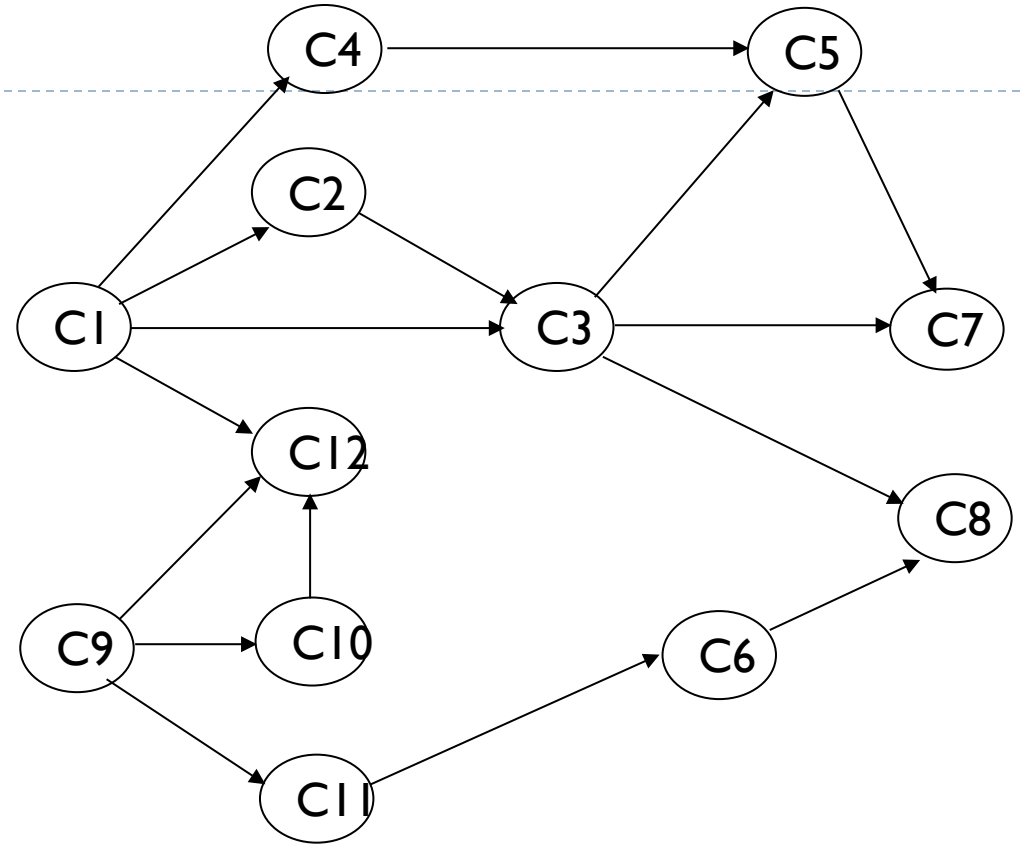
華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

例

课程代号	课程名称	先修棵
C1	程序设计基础	无
C2	离散数学	C1
C3	数据结构	C1,C2
C4	汇编语言	C1
C5	语言的设计和分析	C3,C4
C6	计算机原理	C11
C7	编译原理	C3.C5
C8	操作系统	C3,C6
C9	高等数学	无
C10	线性代数	C9
C11	普通物理	C9
C12	数值分析	C1,C9,C10



例

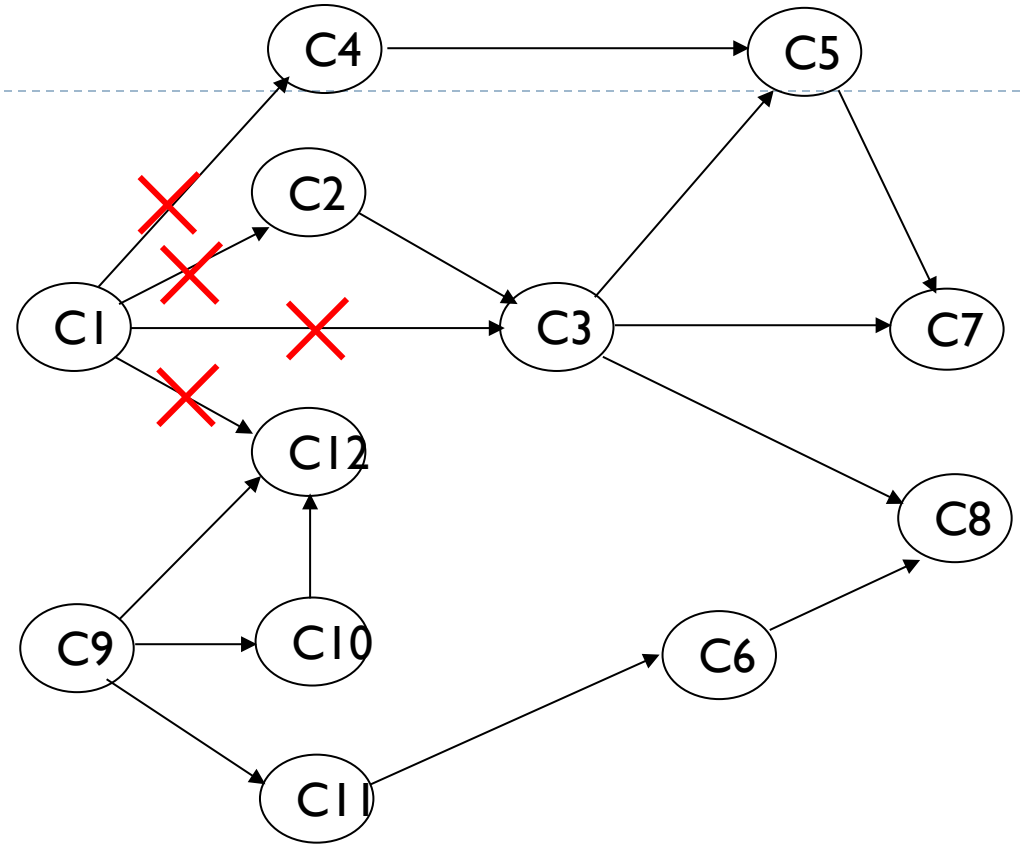


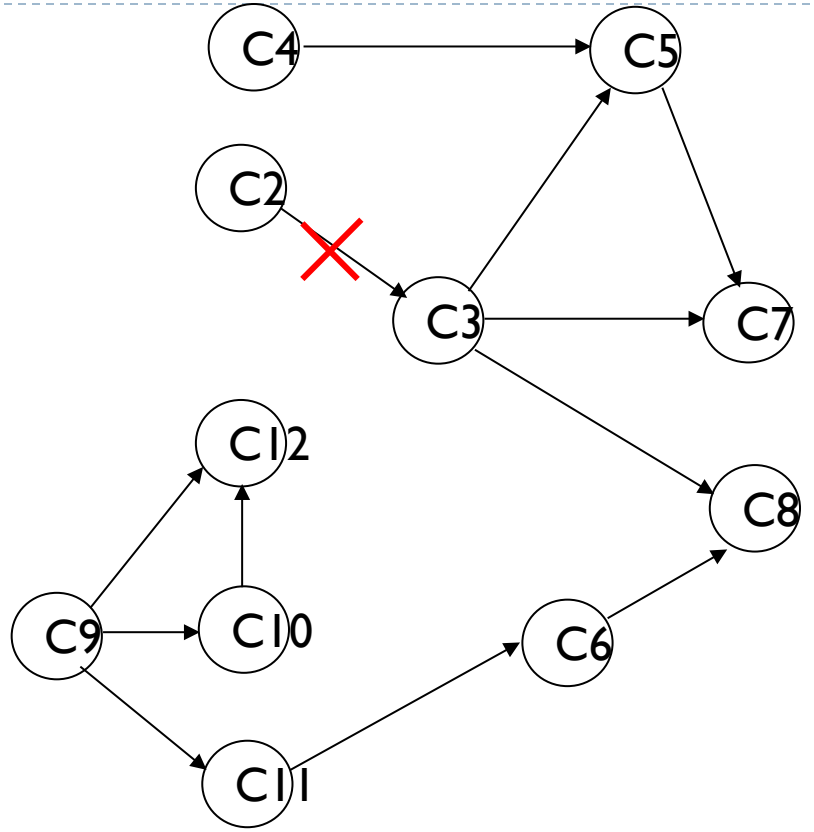
拓扑序列：C1--C2--C3--C4--C5--C7--C9--C10--C11--C6--C12--C8

或　　：C9--C10--C11--C6--C1--C12--C4--C2--C3--C5--C7--C8

一个AOV网的拓扑序列不是唯一的

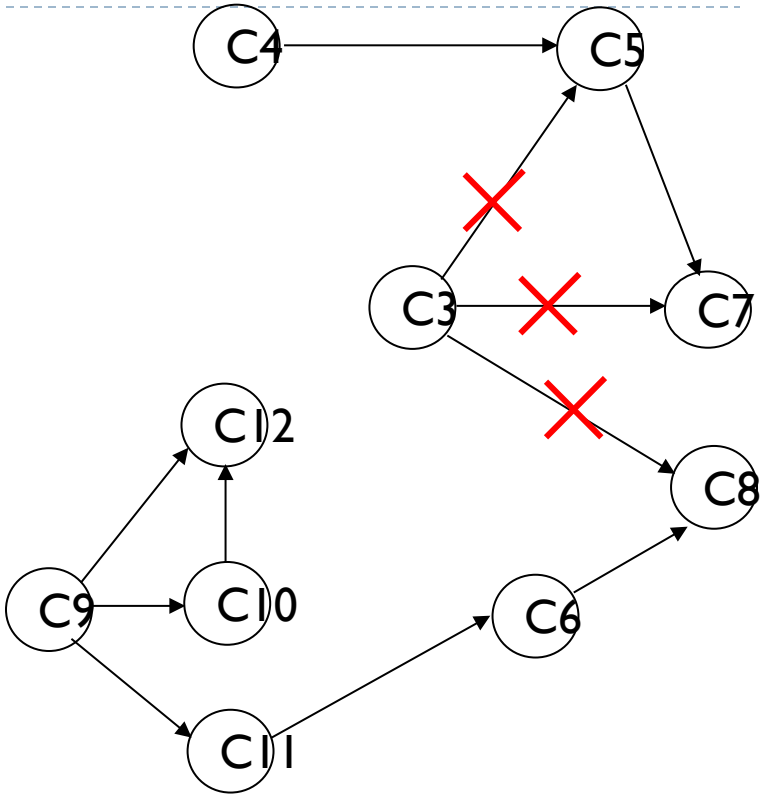
華中科技大學 人工智能与自动化学院


C4
C5
C2
C1
C3
C7
C12
C8
C9
C10
C6
C11



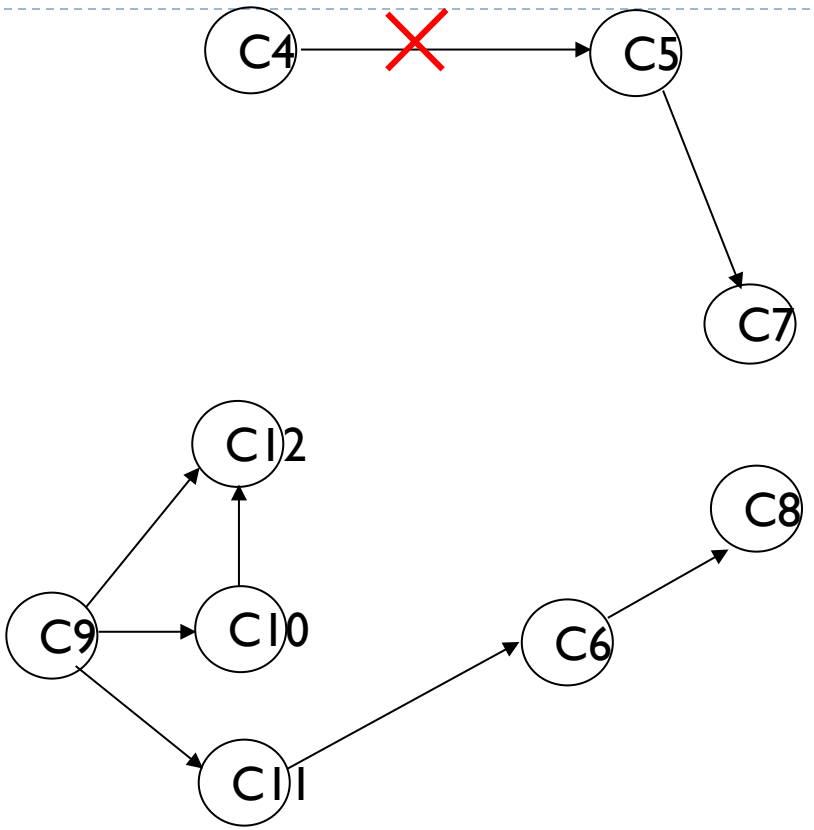
拓扑序列：C1

（1）



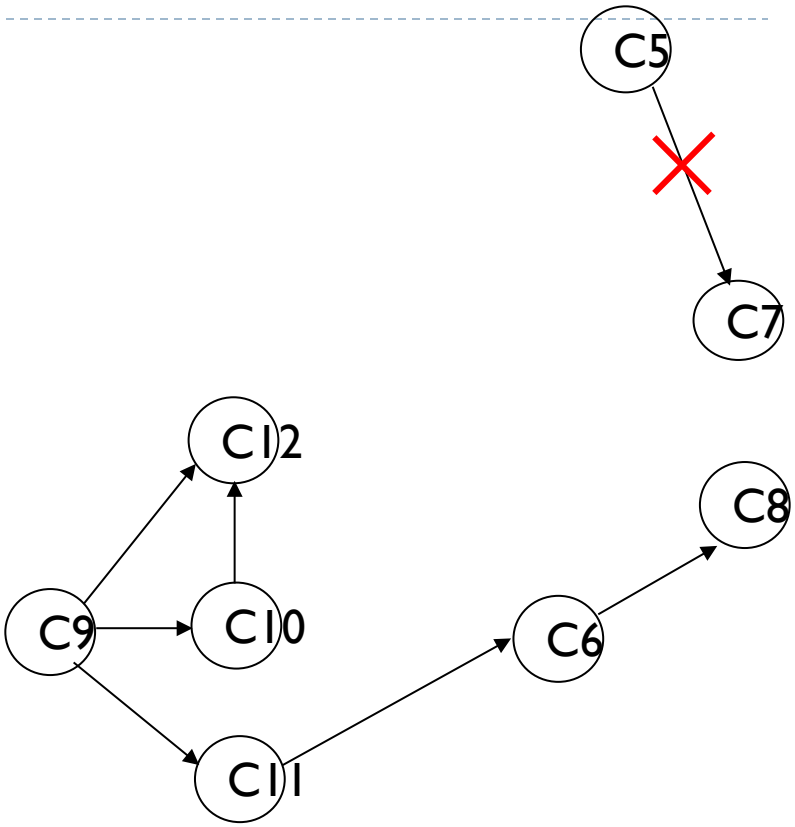
拓扑序列：C1--C2

（2）



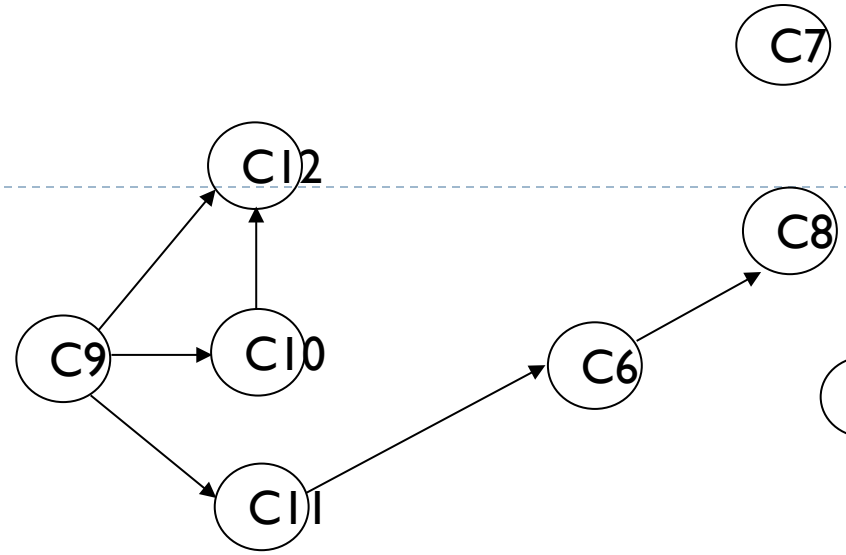
拓扑序列：C1--C2--C3

（3）



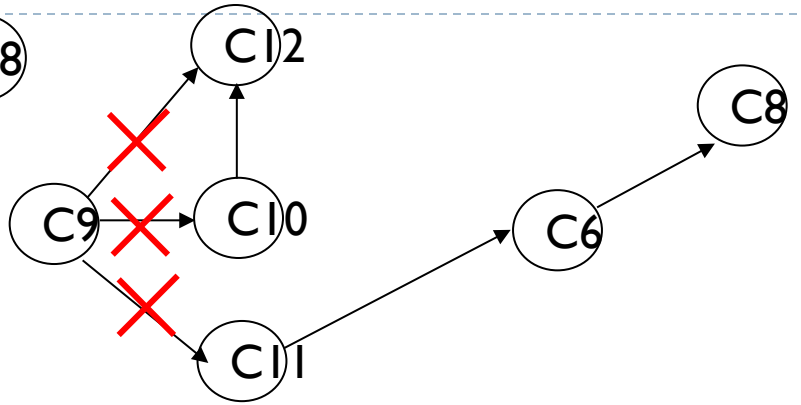
拓扑序列：C1--C2--C3--C4

（4）



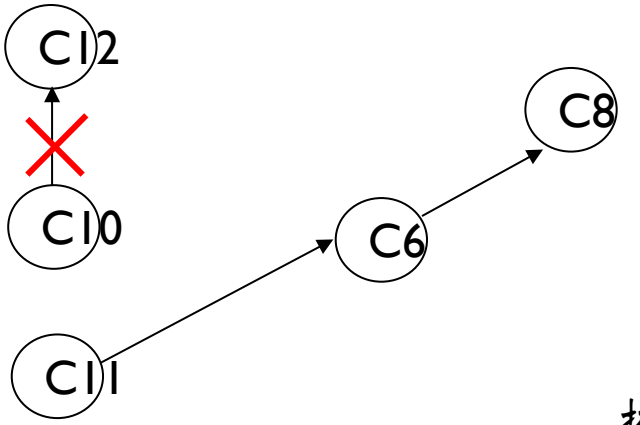
拓扑序列：C1--C2--C3--C4--C5

（5）

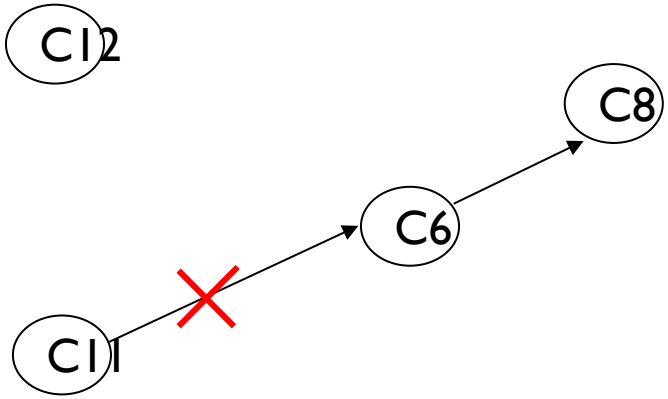


拓扑序列：C1--C2--C3--C4--C5--C7

（6）



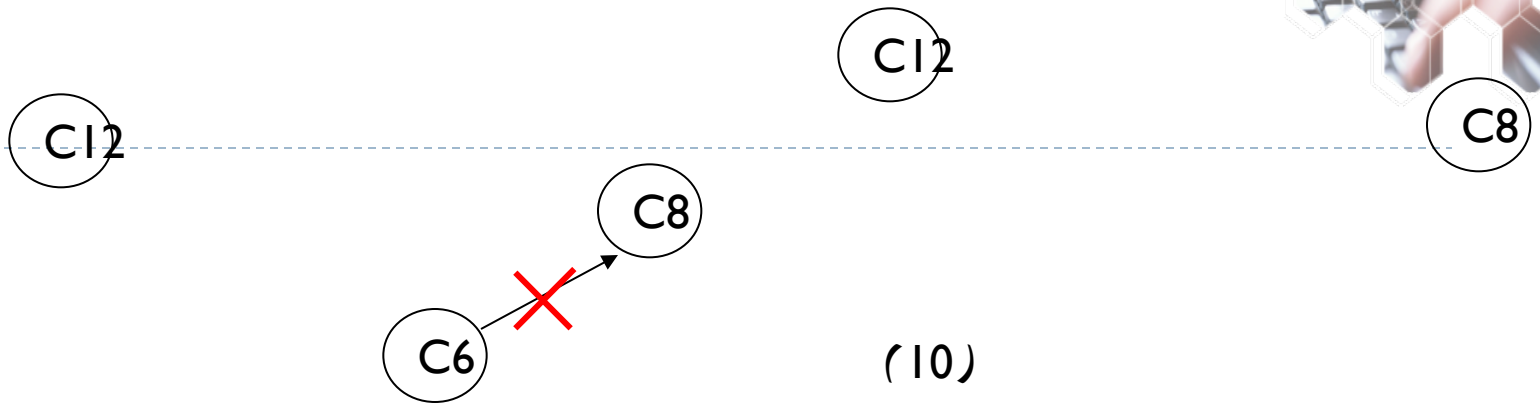
拓扑序列：C1--C2--C3--C4--C5--C7--C9



拓扑序列：C1--C2--C3--C4--C5--C7--C9--C10

（8）

華中科技大學 人工智能与自动化学院



(9)

拓扑序列：C1--C2--C3--C4--C5--C7--C9
--C10--C11

(10)

拓扑序列：C1--C2--C3--C4--C5--C7--C9
--C10--C11--C6

(11)

拓扑序列：C1--C2--C3--C4--C5--C7--C9
--C10--C11--C6--C12--C8

(12)

拓扑序列：C1--C2--C3--C4--C5--C7--C9
--C10--C11--C6--C12

算法实现

- 以邻接表作存储结构
- 把邻接表中所有入度为0的顶点进栈
- 栈非空时，输出栈顶元素V_j并退栈；在邻接表中查找V_j的直接后继V_k，把V_k的入度减1；若V_k的入度为0则进栈
- 重复上述操作直至栈空为止。若栈空时输出的顶点个数不是n，则有向图有环；否则，拓扑排序完毕

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

算法代码

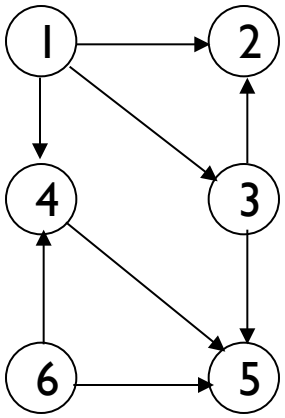
```
Status TopologicalSort(ALGraph G) {
   // 有向图 G 采用邻接表存储结构。
   // 若 G 无回路,则输出 G 的顶点的一个拓扑序列并返回 OK,否则 ERROR。
   FindInDegree(G, indegree);            // 对各顶点求入度 indegree[0..vernum-1]
   InitStack(S);
   for (i=0; i<G.vexnum; ++i)            // 建零入度顶点栈 S
      if (!indegree[i]) Push(S, i);      // 入度为 0 者进栈
   count = 0;                            // 对输出顶点计数
   while (!StackEmpty(S)) {
      Pop(S, i);  printf(i, G.vertices[i].data);  ++count;  // 输出 i 号顶点并计数
      for (p=G.vertices[i].firstarc;  p;  p=p->nextarc) {
         k = p->adjvex;                          // 对 i 号顶点的每个邻接点的入度减 1
         if (!(--indegree[k])) Push(S, k); // 若入度减为 0,则入栈
      }// for
   }// while
   if (count<G.vexnum)  return ERROR;          // 该有向图有回路
   else return OK;
}// TopologicalSort
```

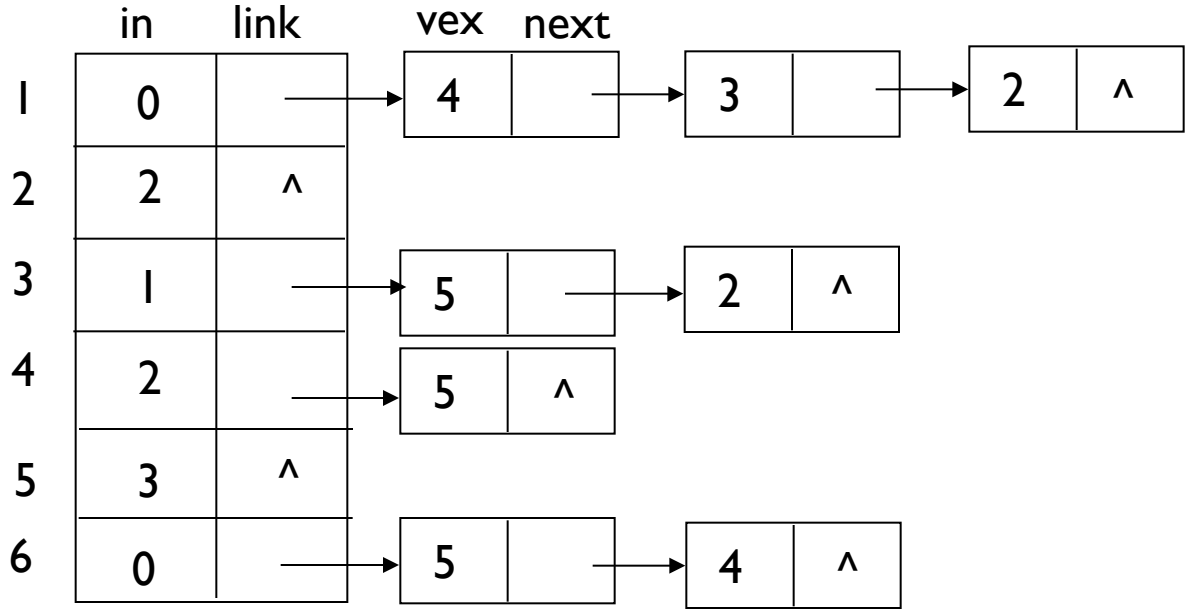
算法 7.12

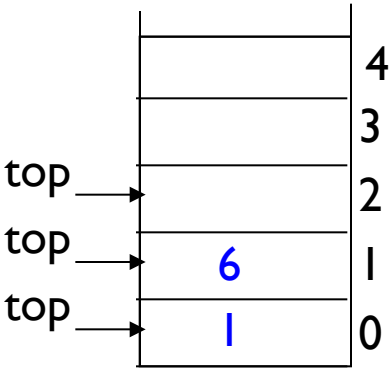
華中科技大學 人工智能与自动化学院

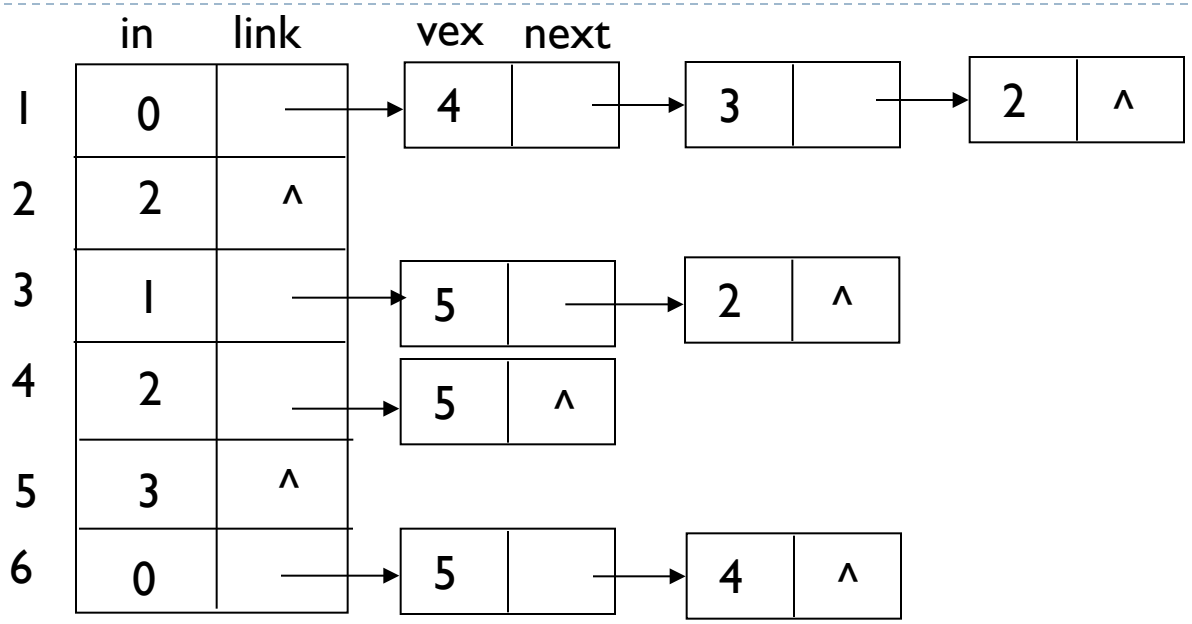
算法描述

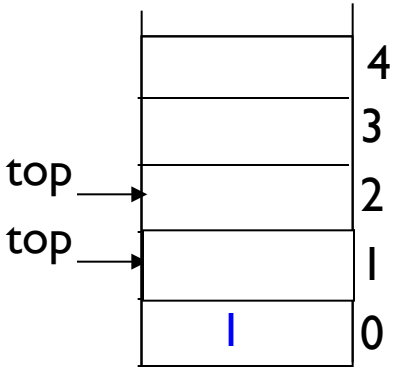
例


1
2
3
4
5
6


in
link
vex
next
1
0
4
3
2
^
2
2
^
3
1
5
2
^
4
2
5
^
5
3
^
6
0
5
4
^

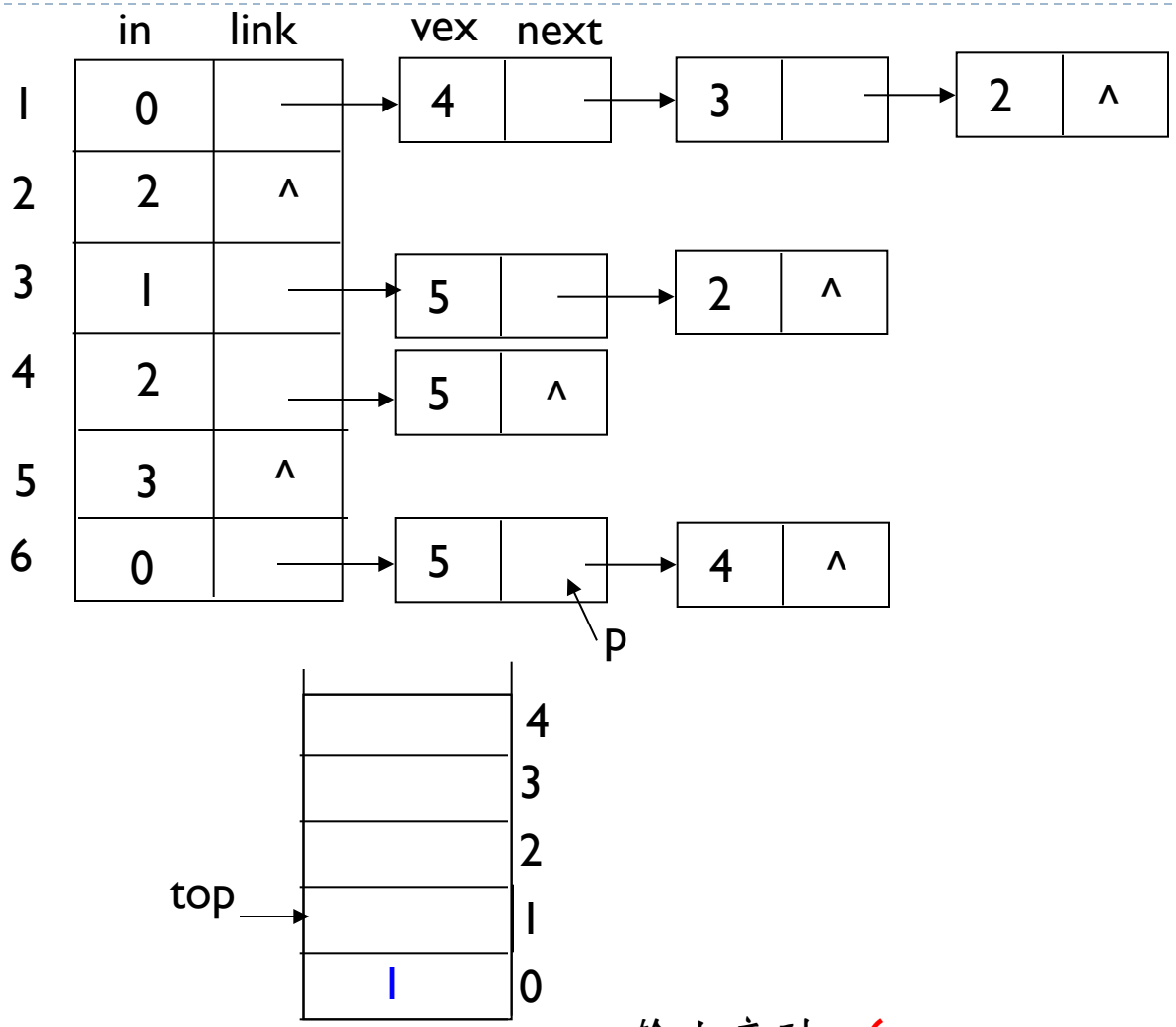

top
top
top
6
1
4
3
2
1
0



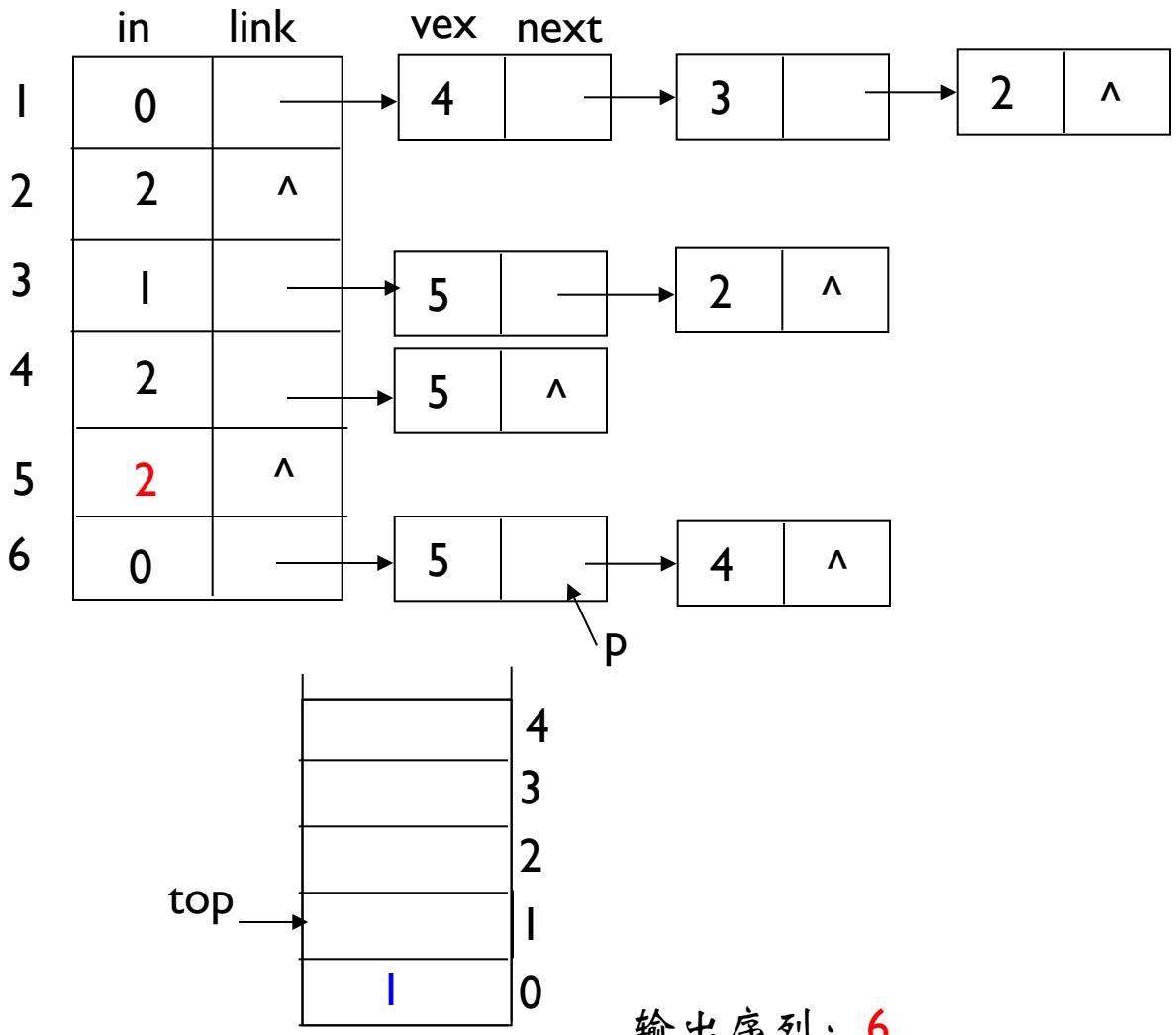


输出序列：6

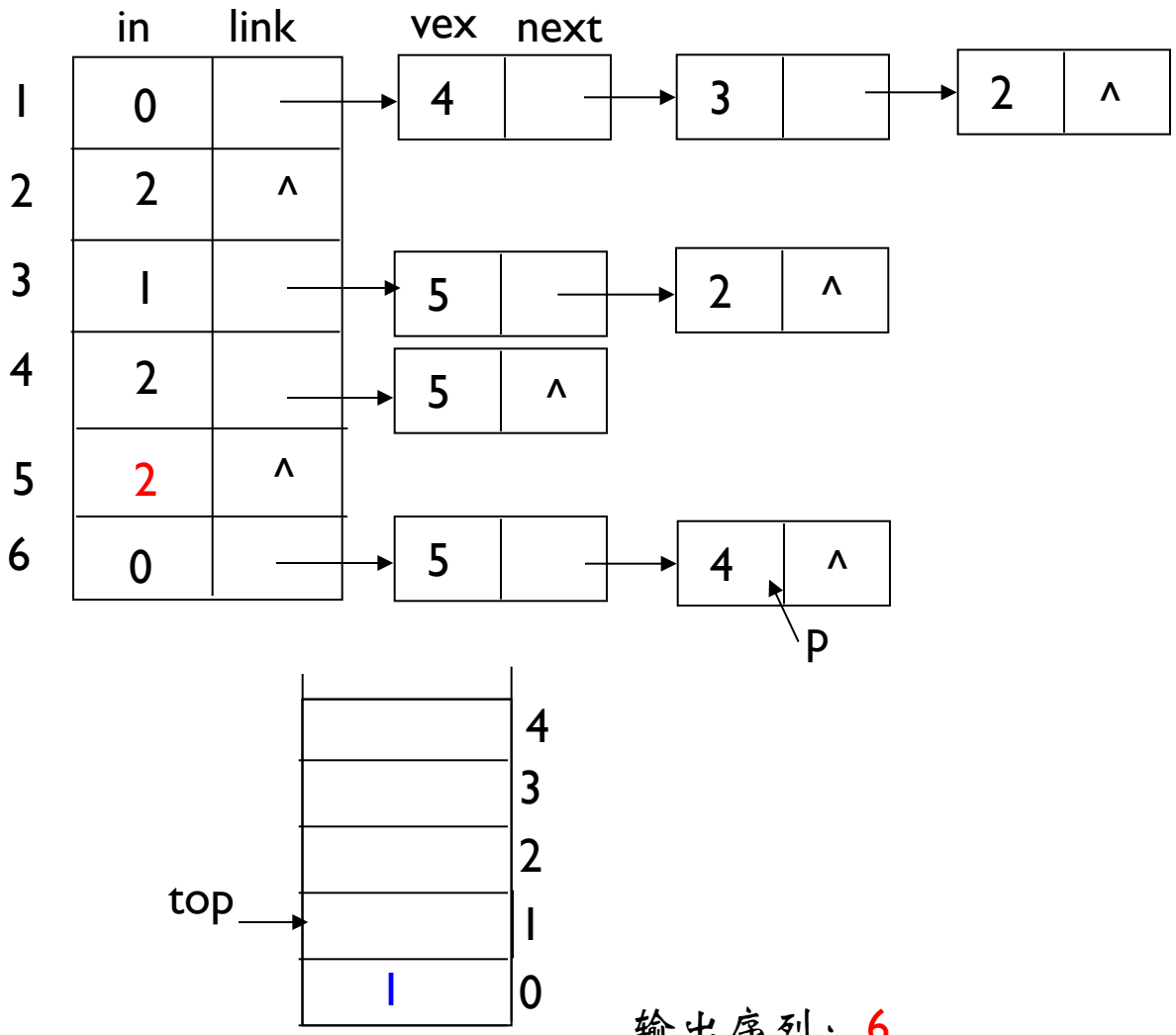
華中科技大學
人工智能与自动化学院



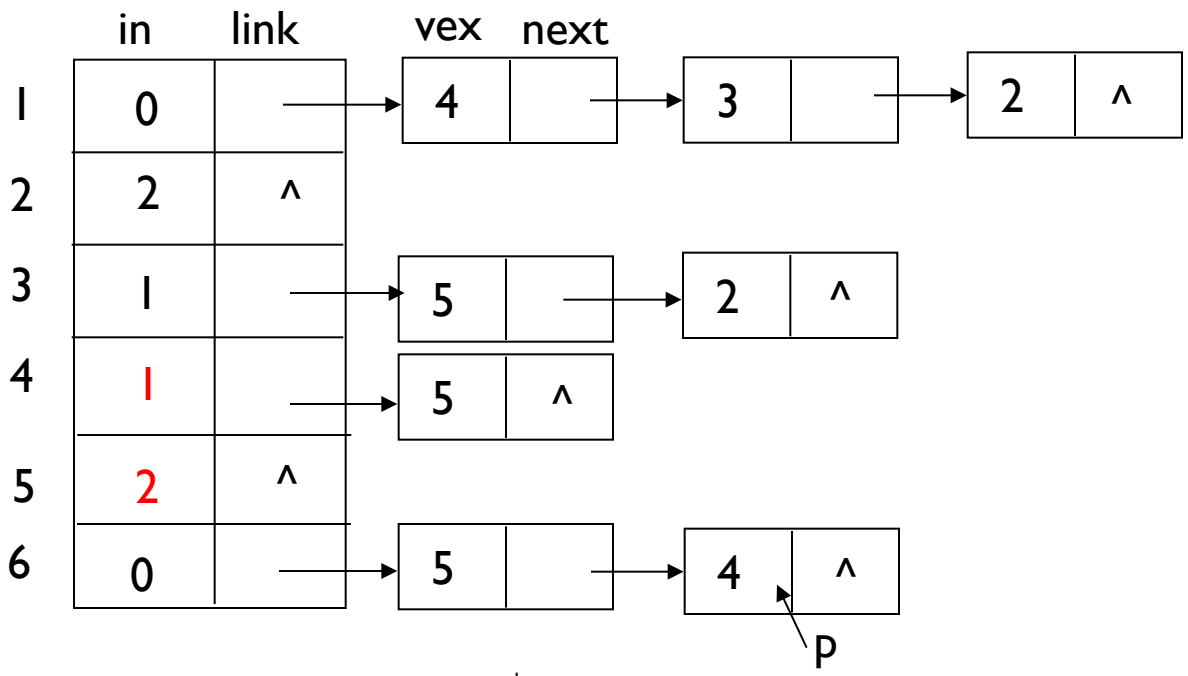
输出序列：6

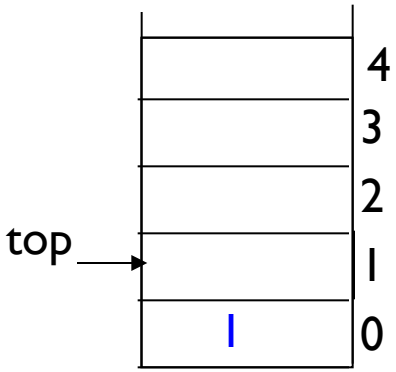


输出序列：6



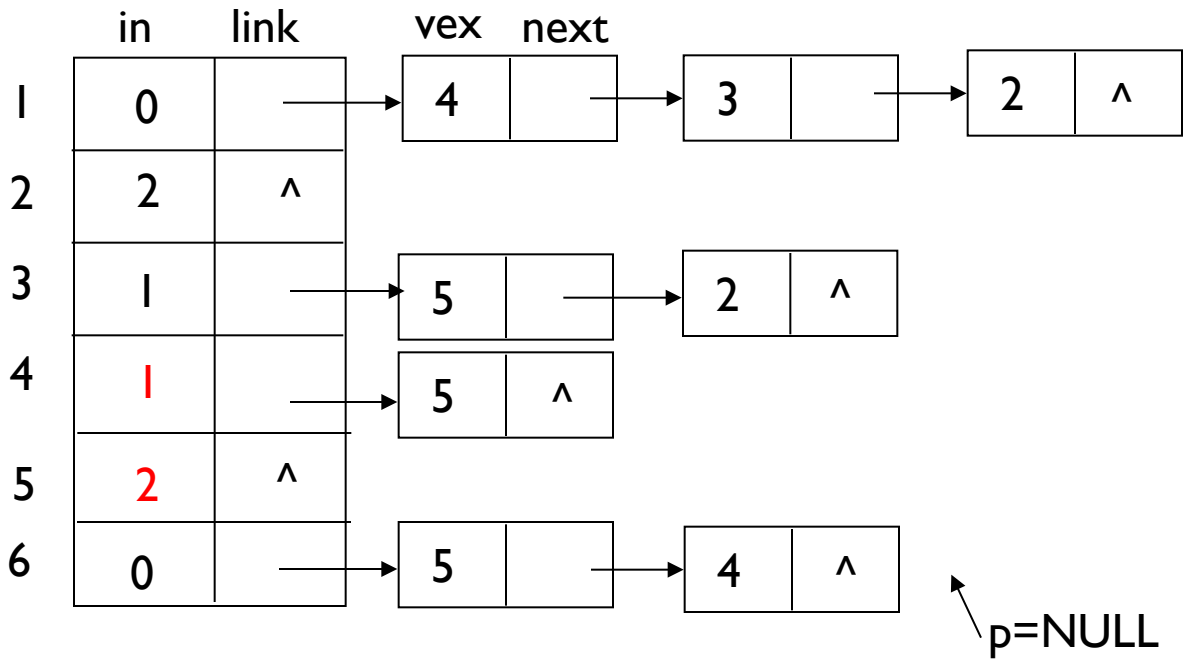
输出序列：6

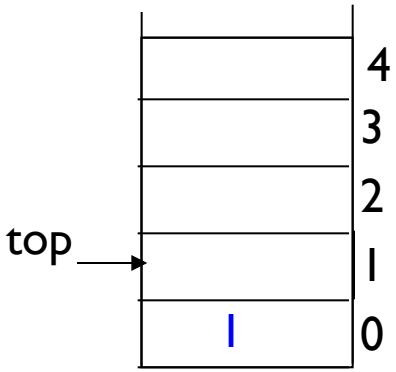




输出序列：6

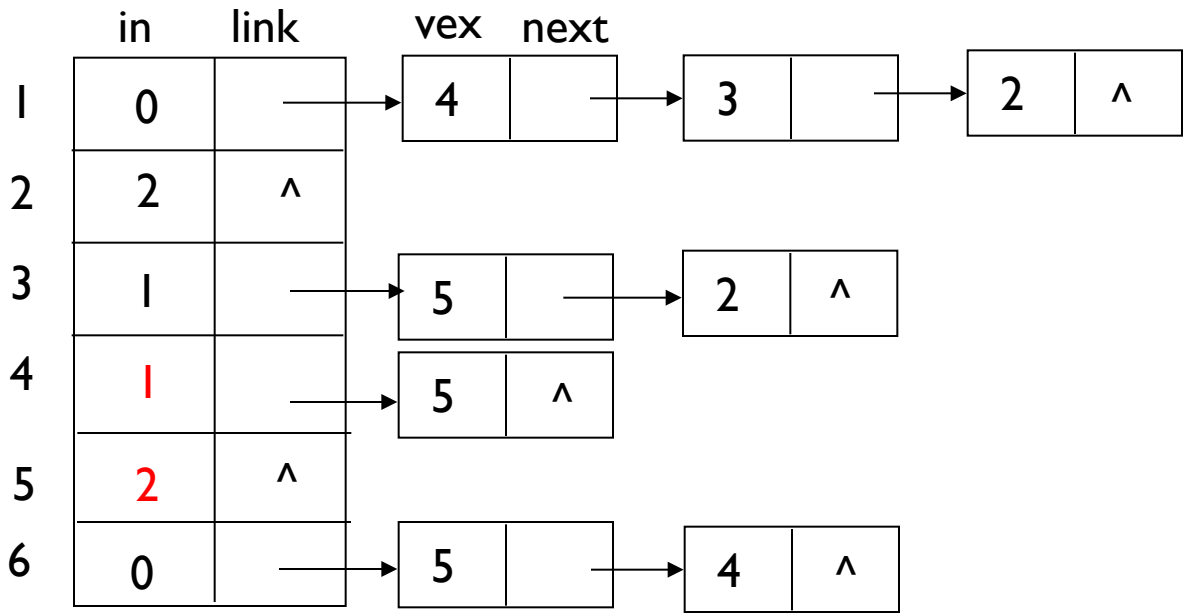
華中科技大學
人工智能与自动化学院

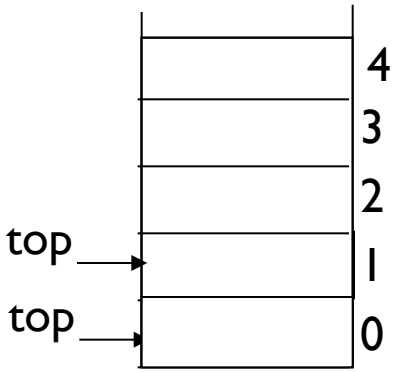




输出序列：6

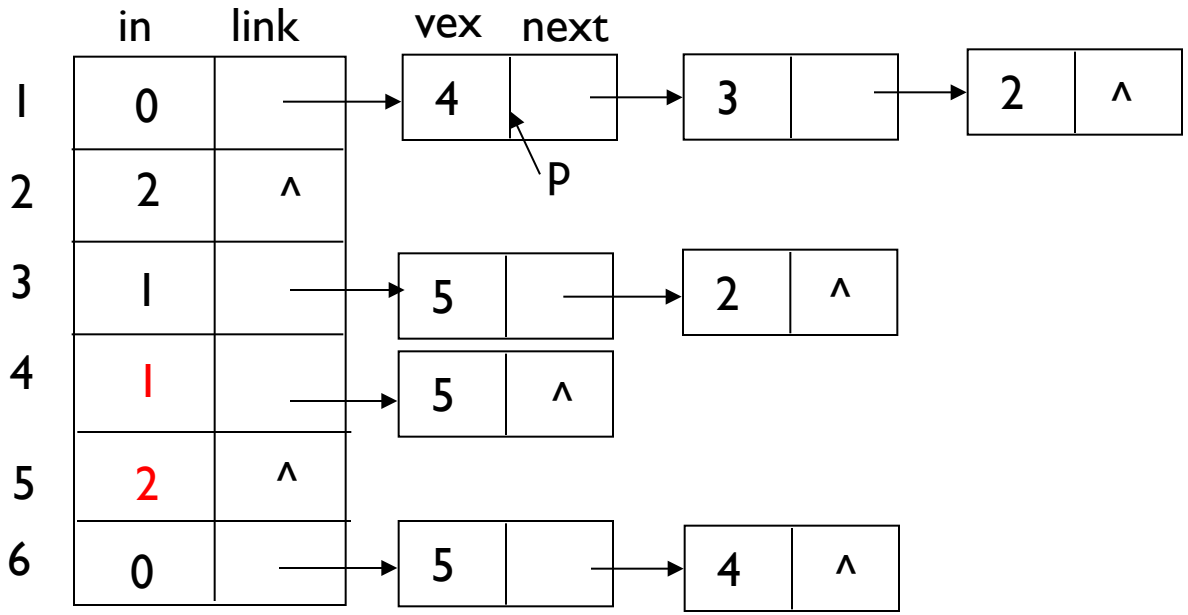
華中科技大學
人工智能与自动化学院

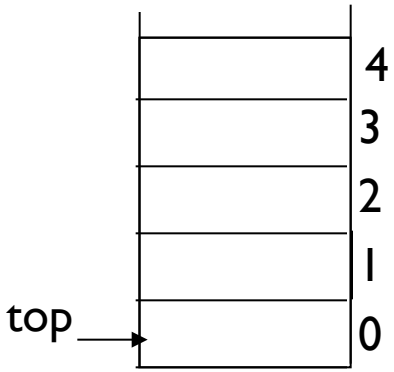




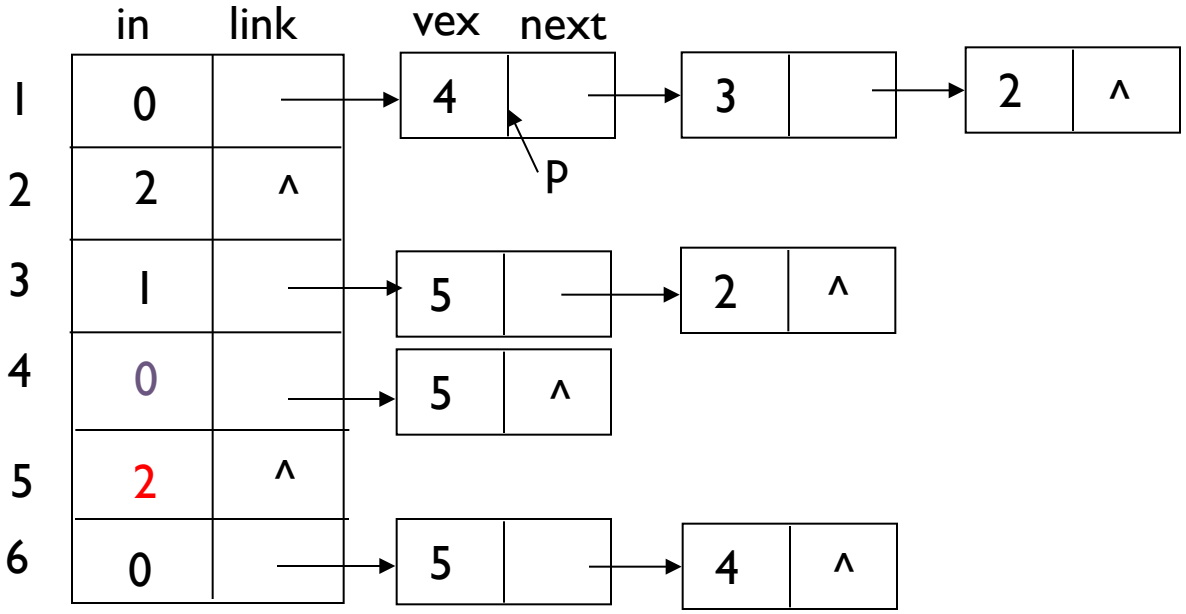
输出序列：6 1

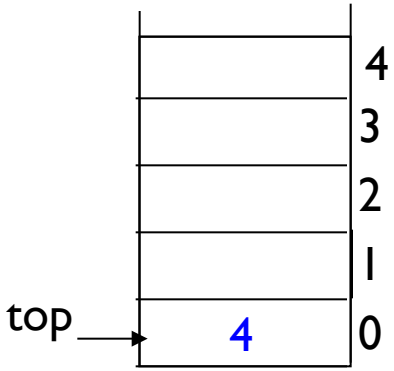
華中科技大學
人工智能与自动化学院





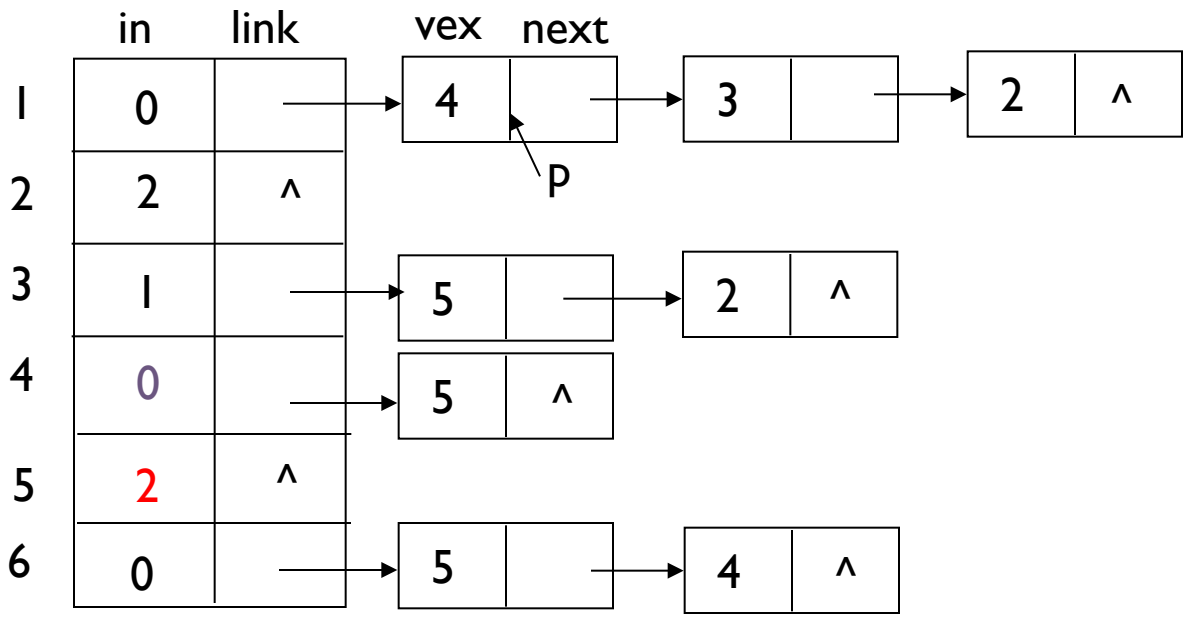
输出序列：6 1

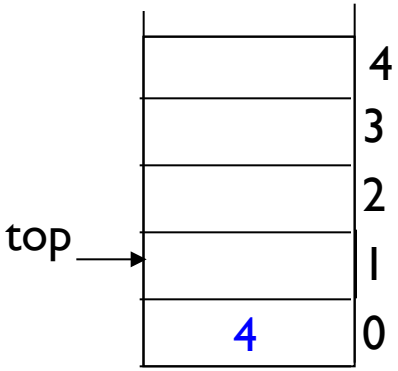




输出序列：6 1

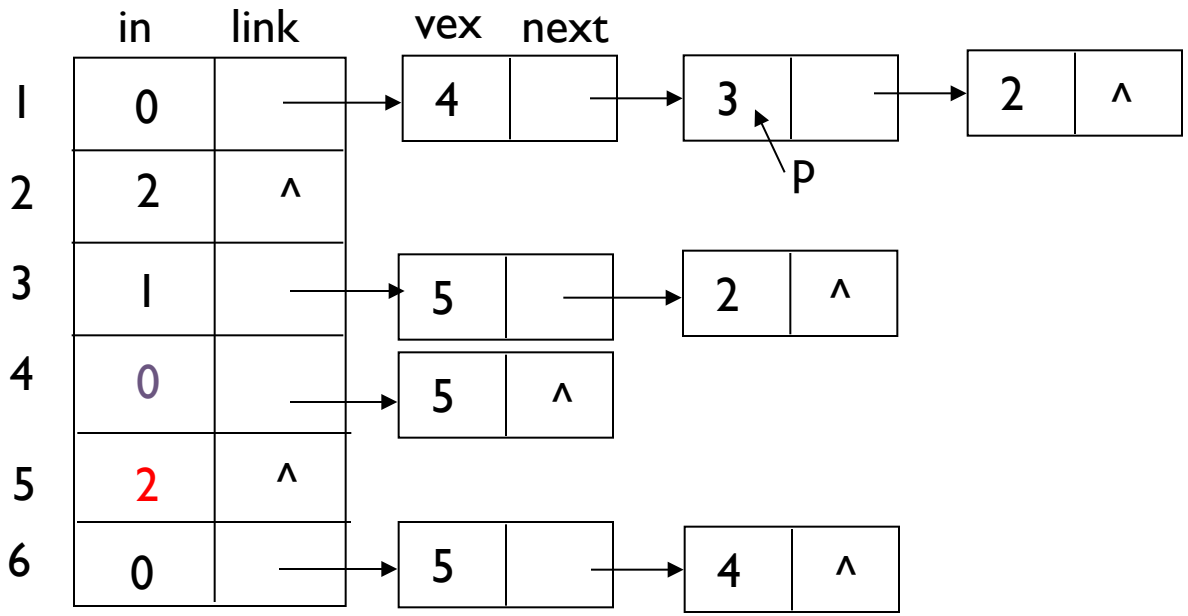
華中科技大學
人工智能与自动化学院

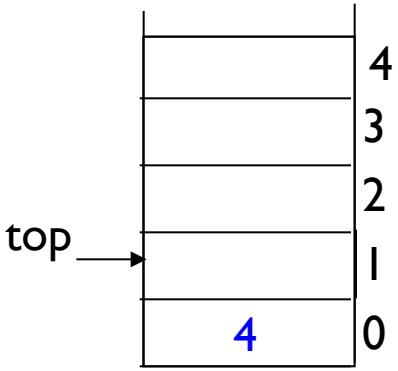




输出序列：6 1

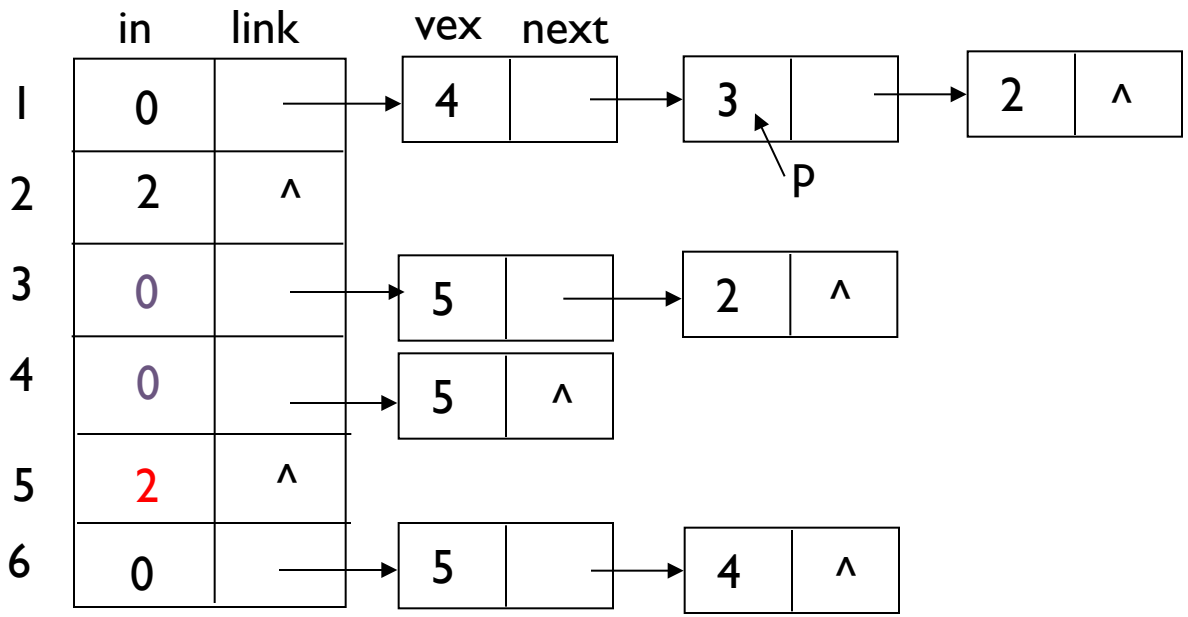
華中科技大學
人工智能与自动化学院

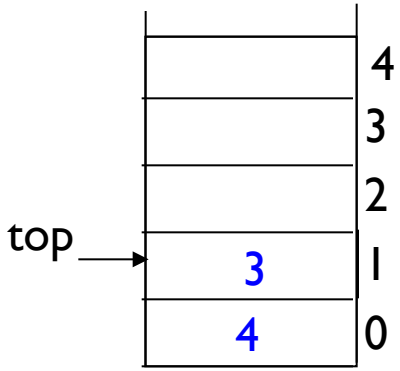




输出序列：6 1

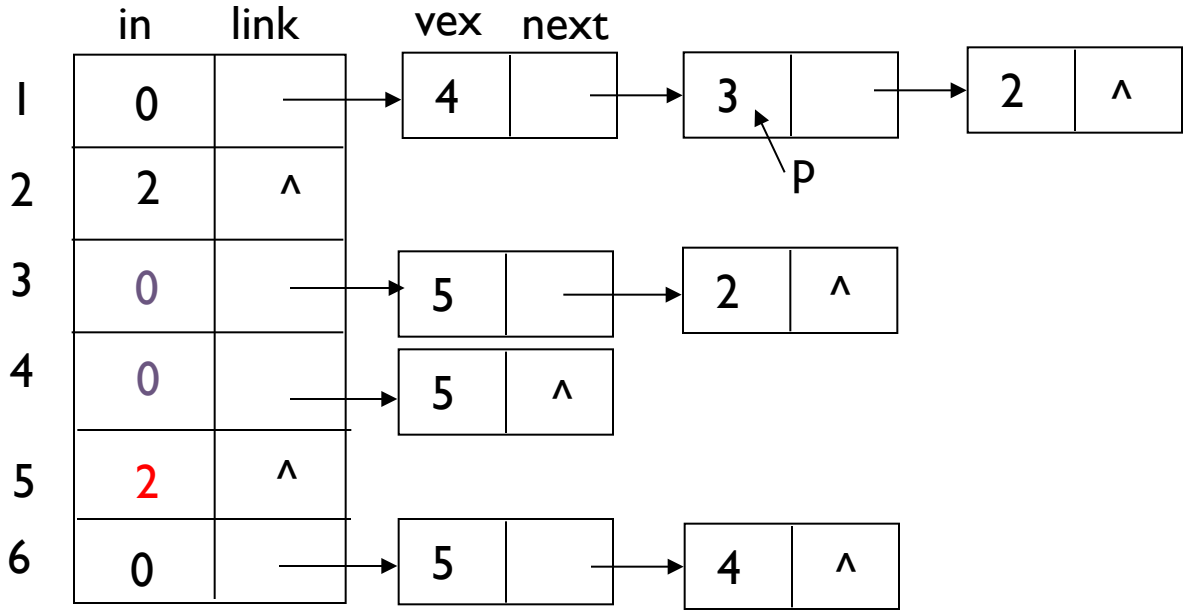
華中科技大學
人工智能与自动化学院

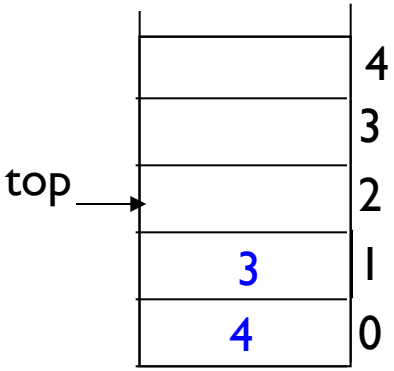




输出序列：6 1

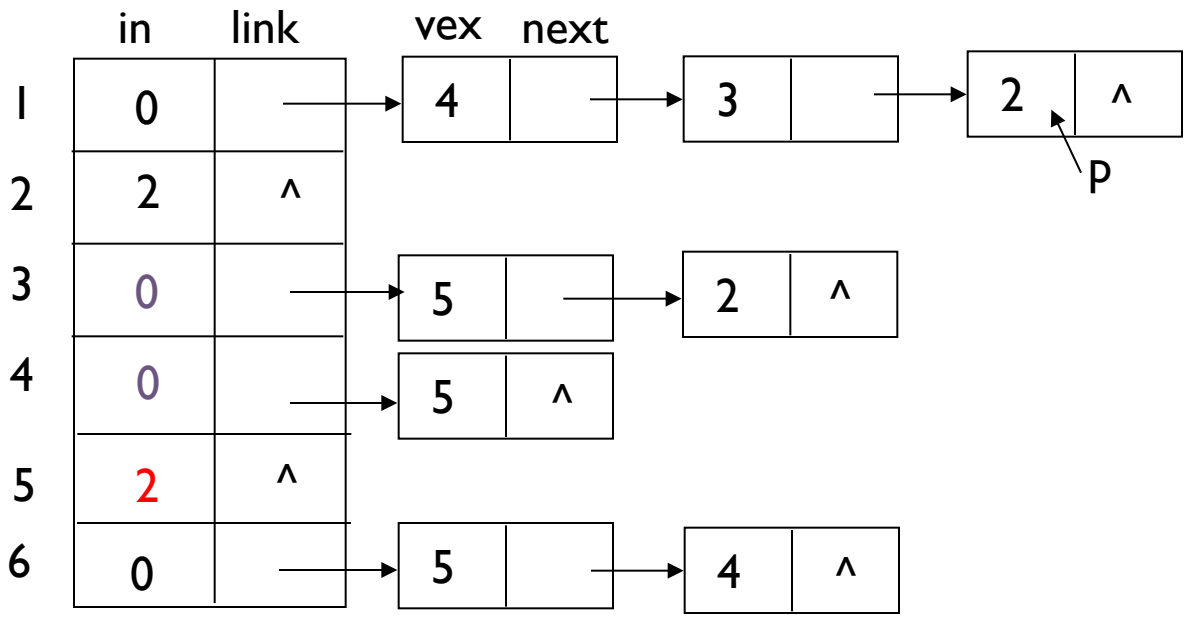
華中科技大學
人工智能与自动化学院

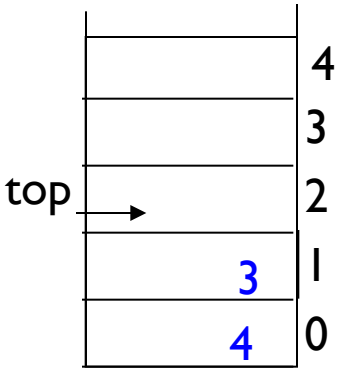




输出序列：6 1

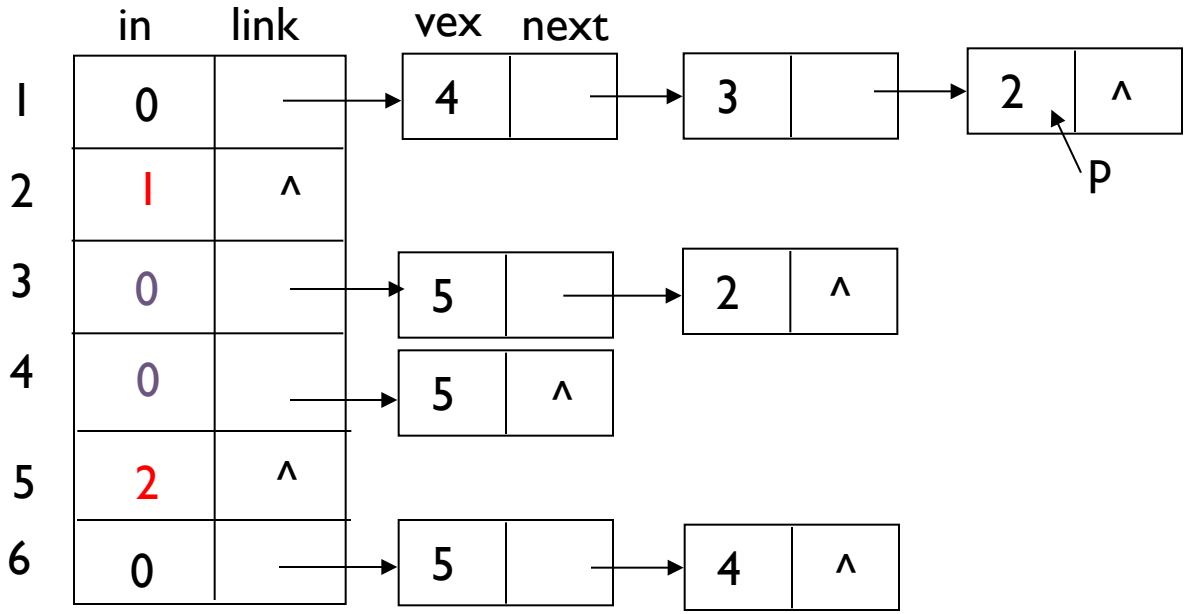
華中科技大學
人工智能与自动化学院

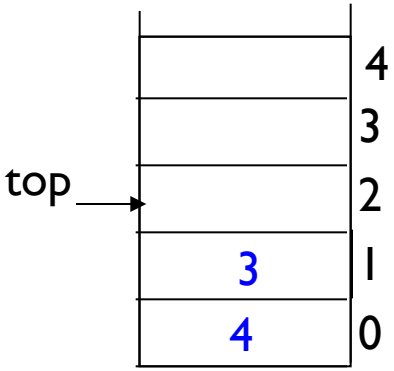




输出序列：6 1

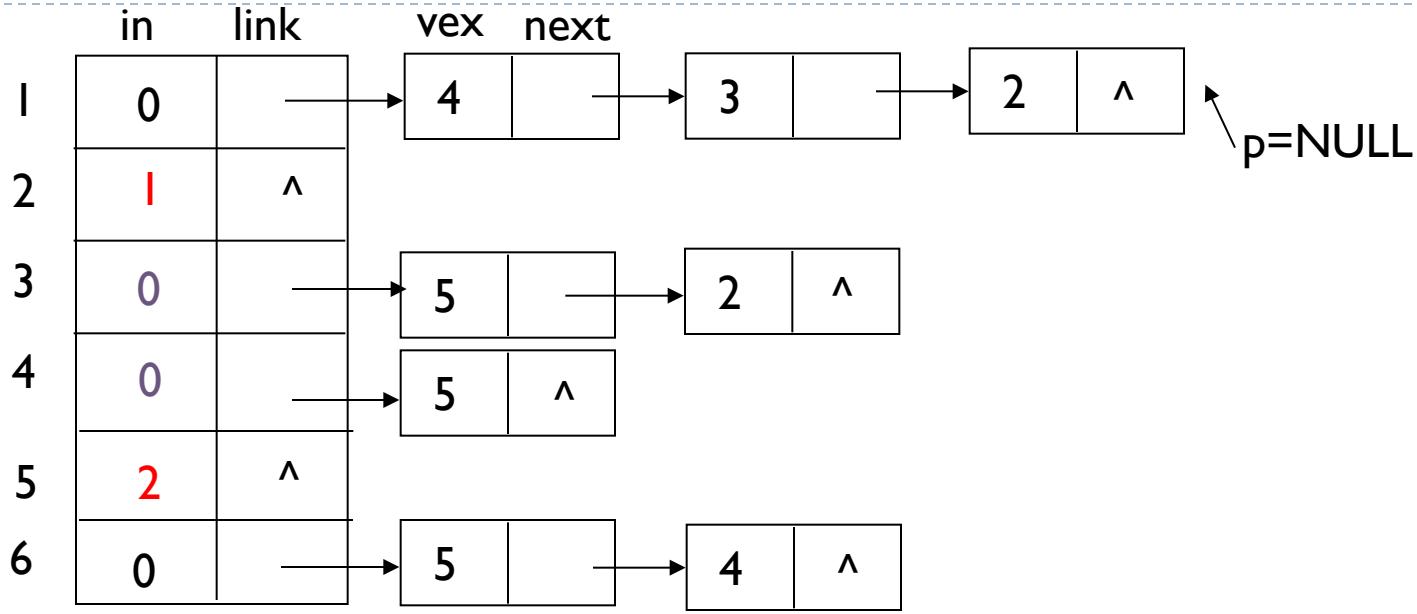
華中科技大學
人工智能与自动化学院

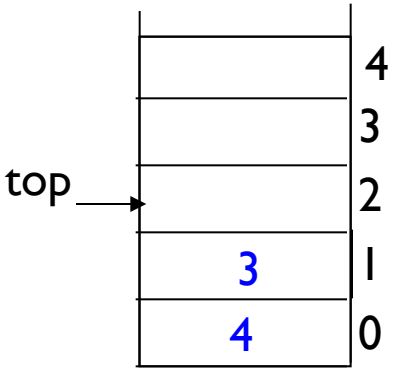




输出序列：6 1

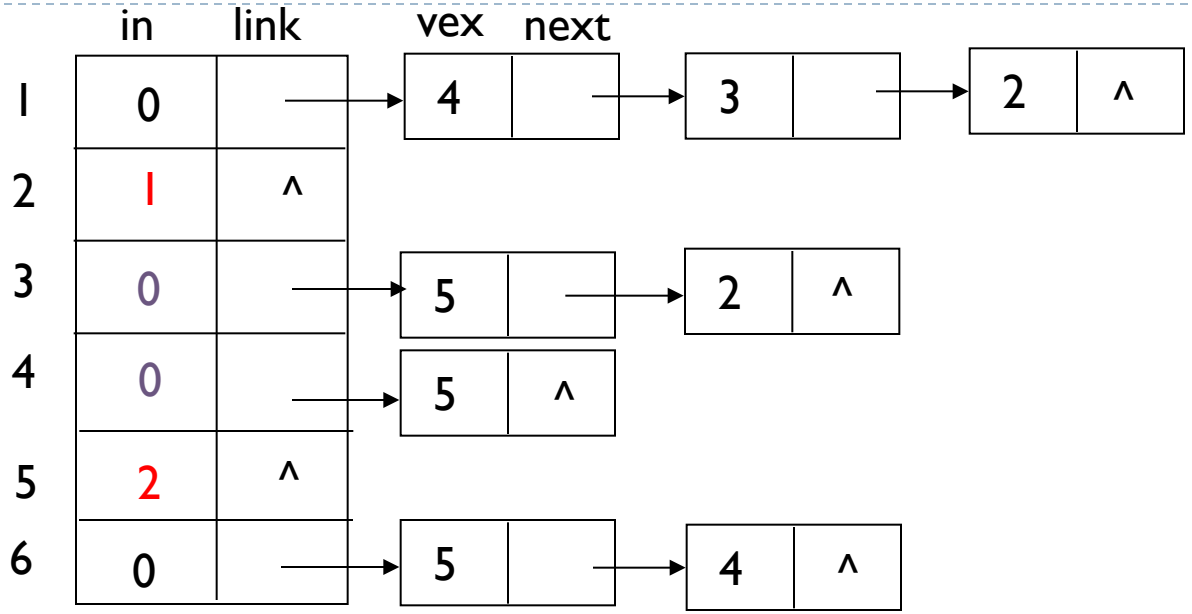
華中科技大學
人工智能与自动化学院

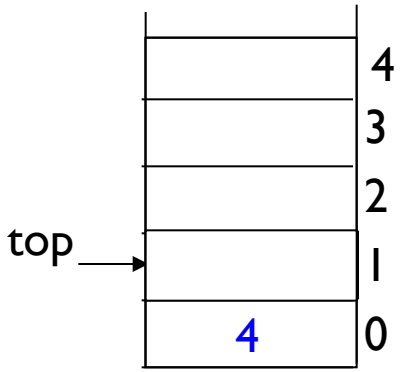




输出序列：6 1

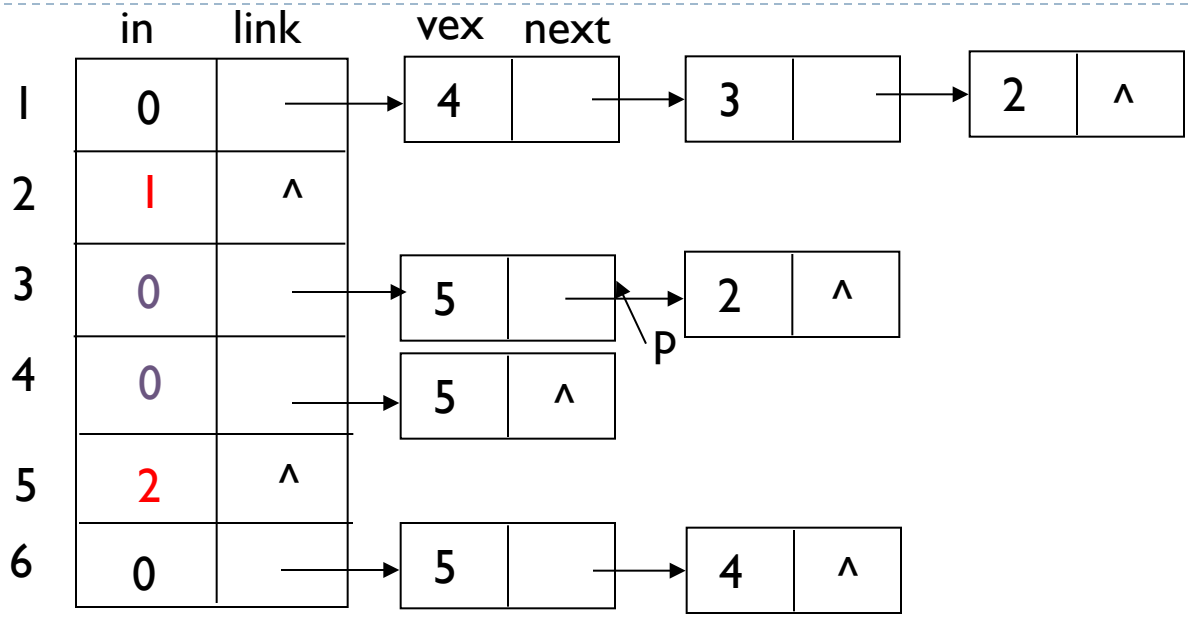
華中科技大學
人工智能与自动化学院

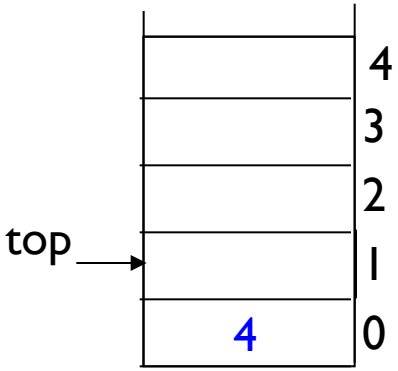




输出序列：6 1 3

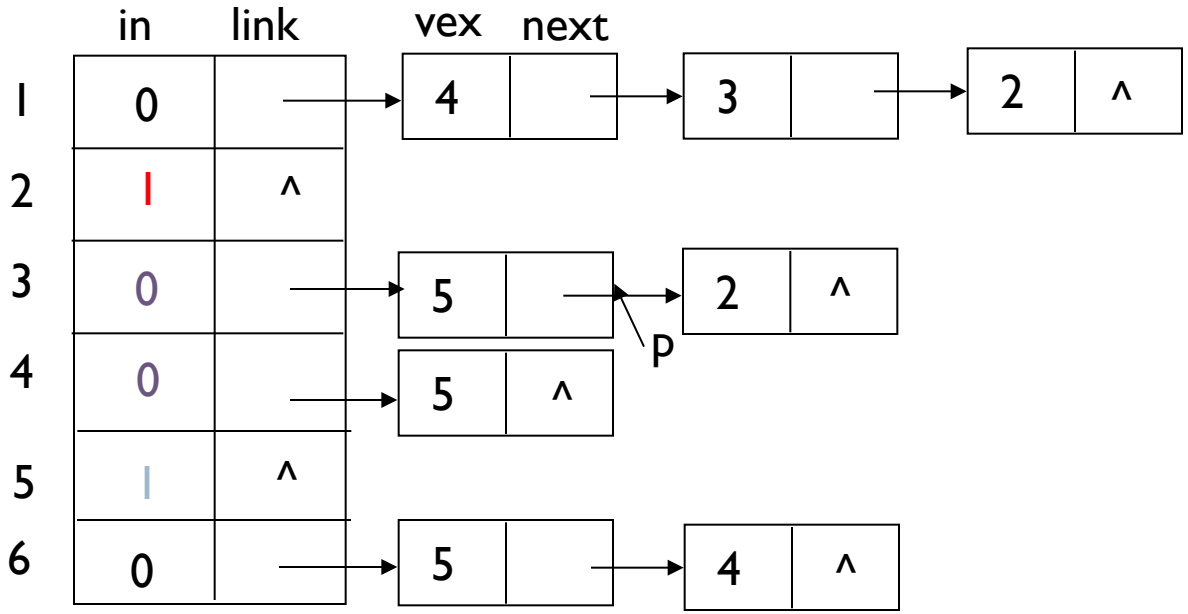
華中科技大學
人工智能与自动化学院

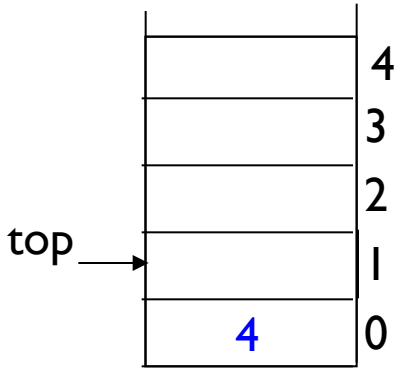




输出序列：6 1 3

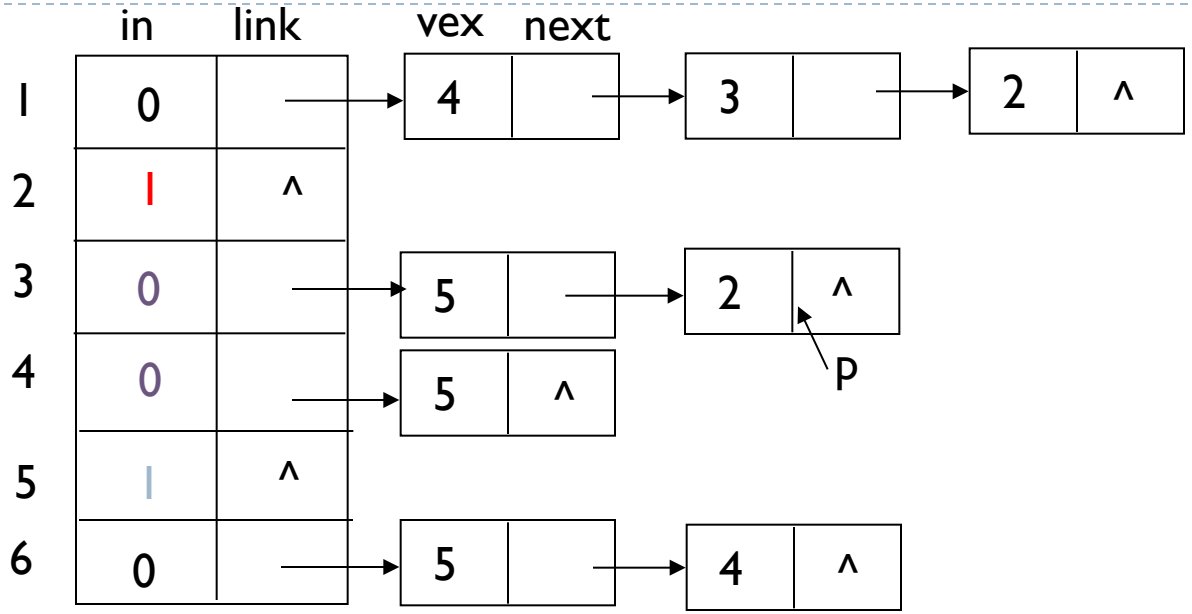
華中科技大學
人工智能与自动化学院

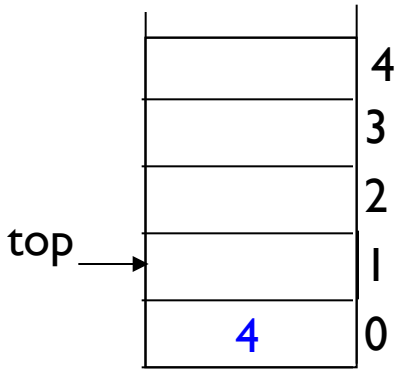




输出序列：6 1 3

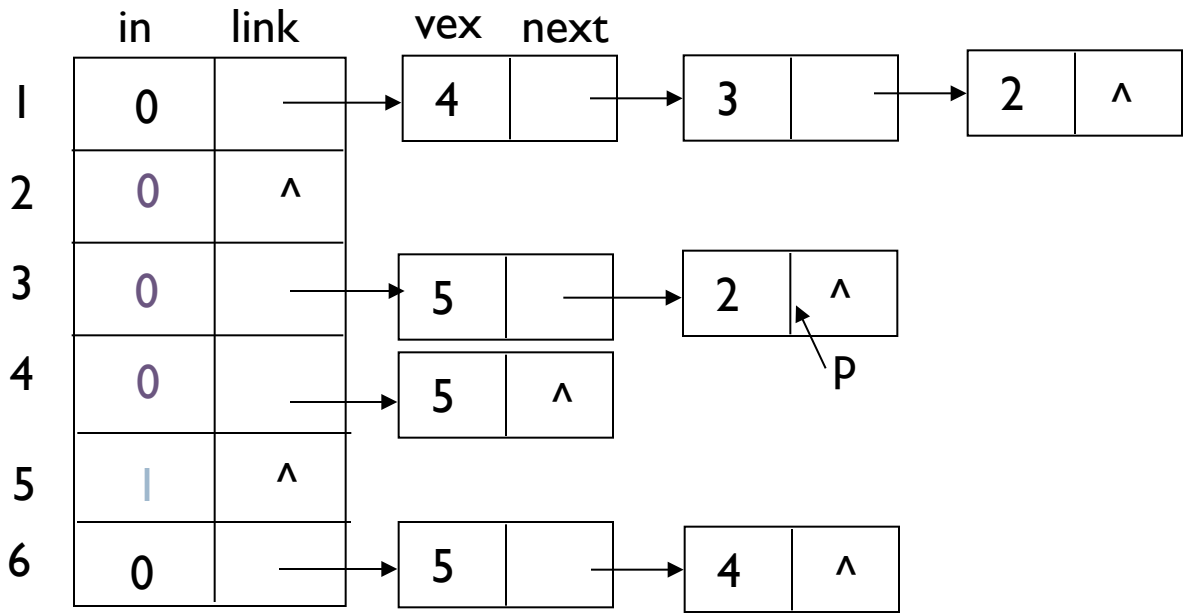
華中科技大學
人工智能与自动化学院

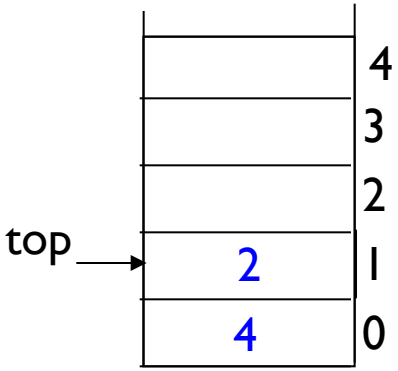




输出序列：6 1 3

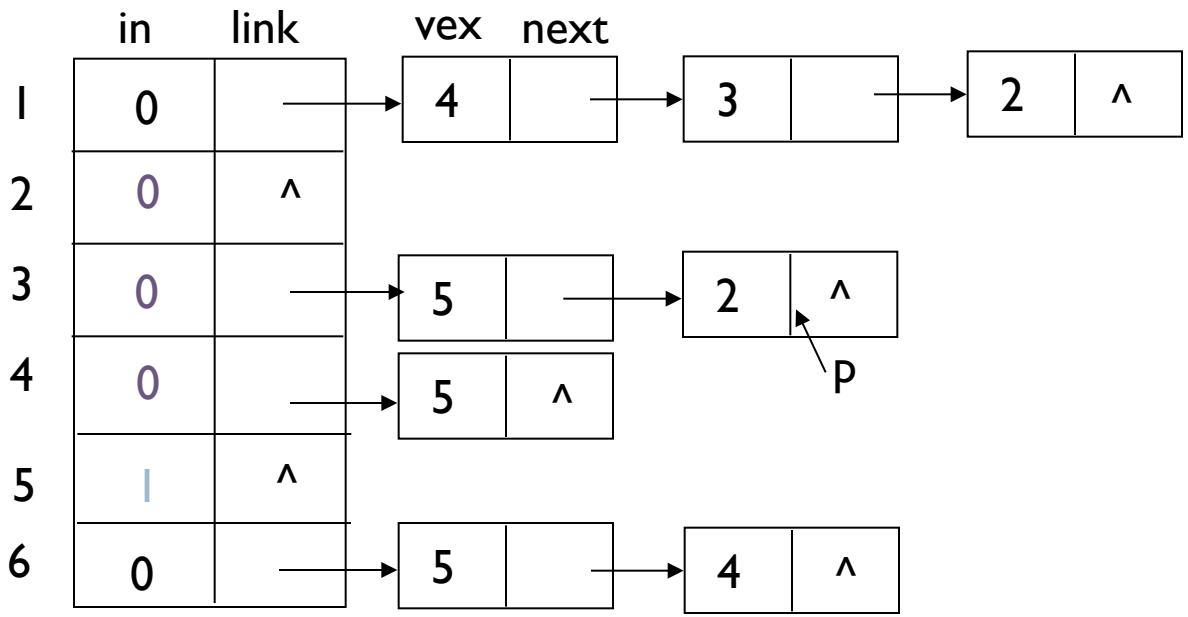
華中科技大學
人工智能与自动化学院
A-ambitious 积极进取 U-unique 锐意创新
T-tolerant 包容并蓄 O-open 开放交流

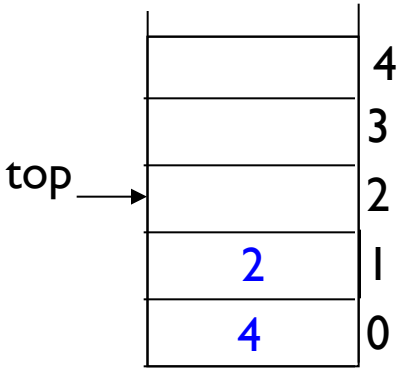




输出序列：6 1 3

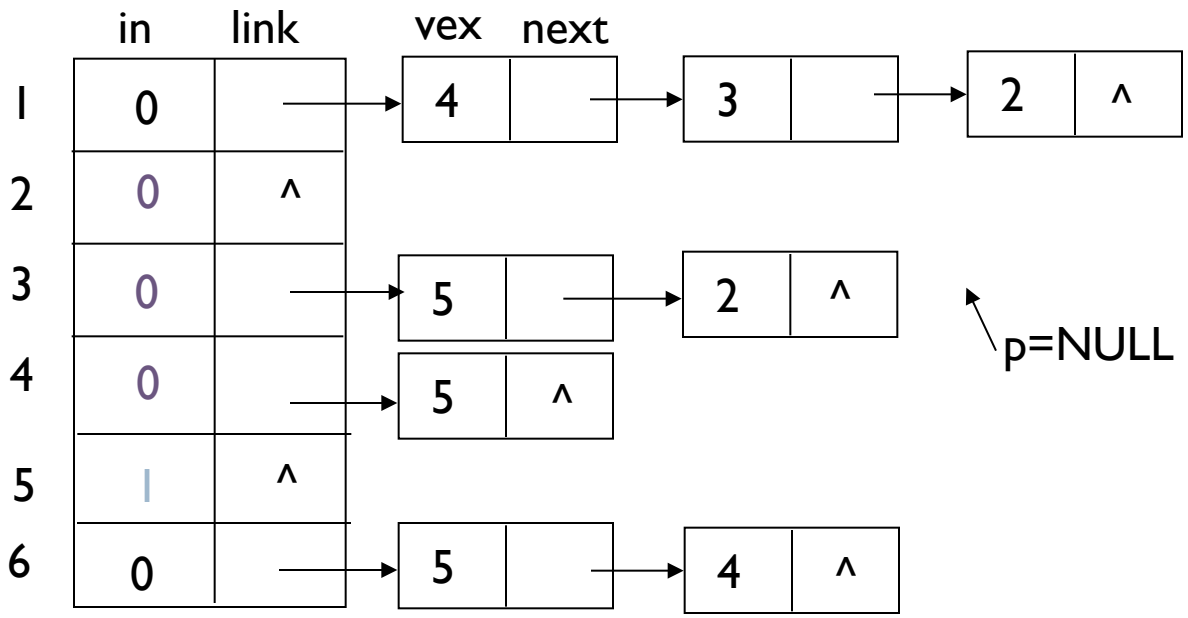
華中科技大學
人工智能与自动化学院

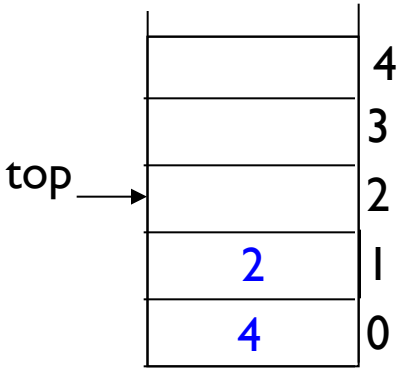




输出序列：6 1 3

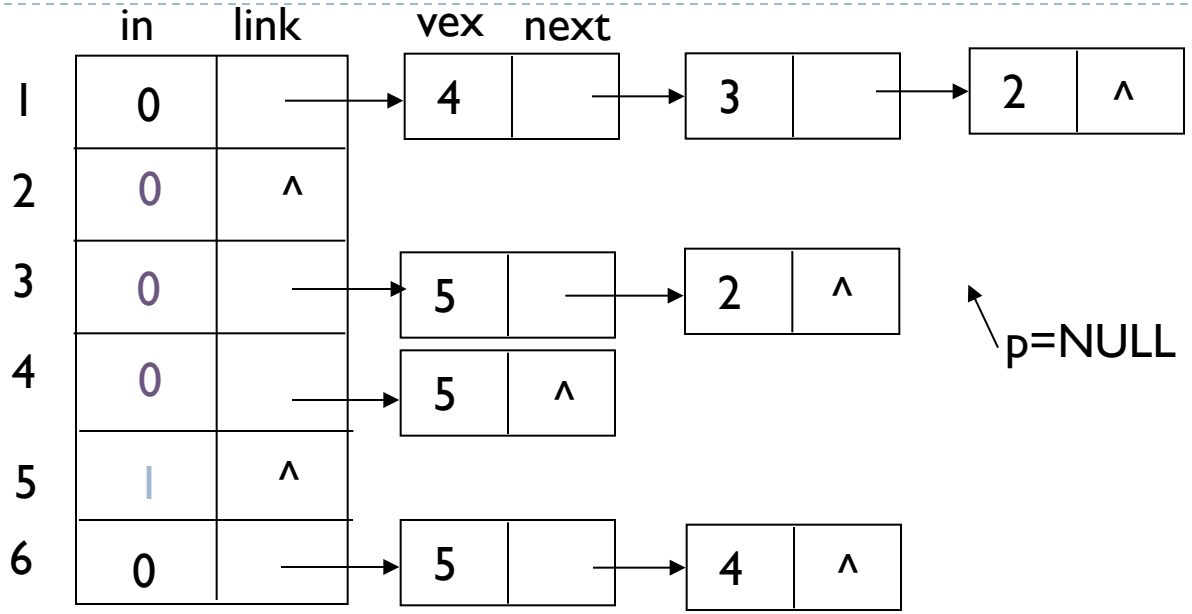
華中科技大學
人工智能与自动化学院

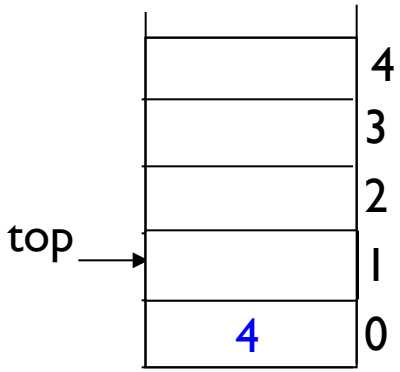




输出序列：6 1 3

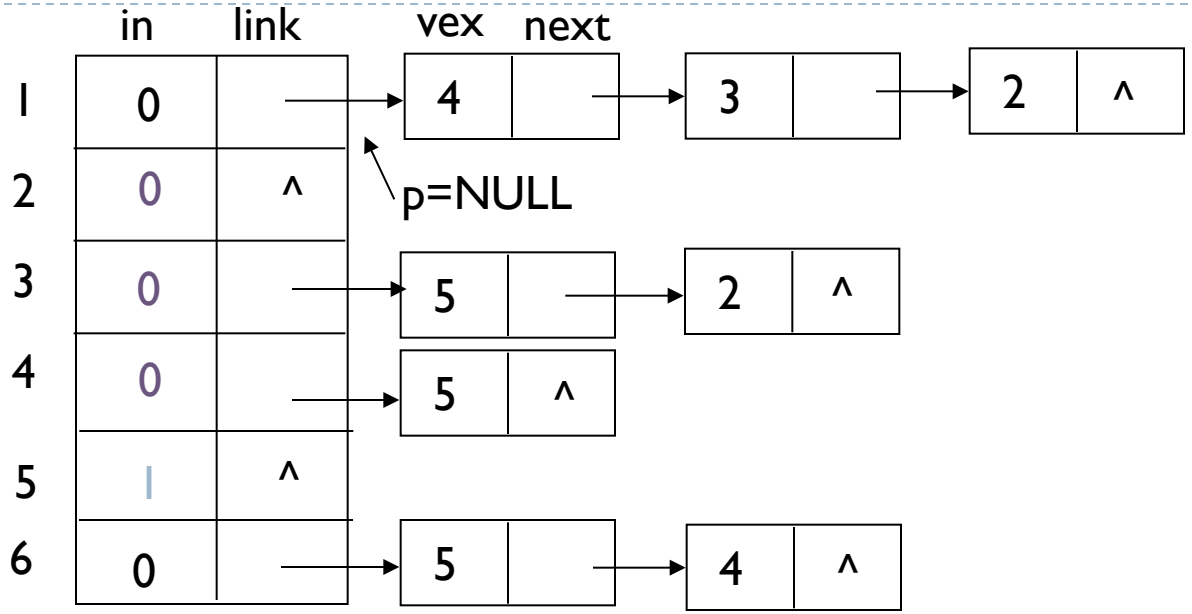
華中科技大學
人工智能与自动化学院

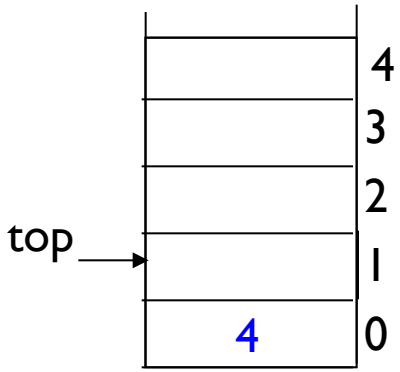




输出序列：6 1 3 2

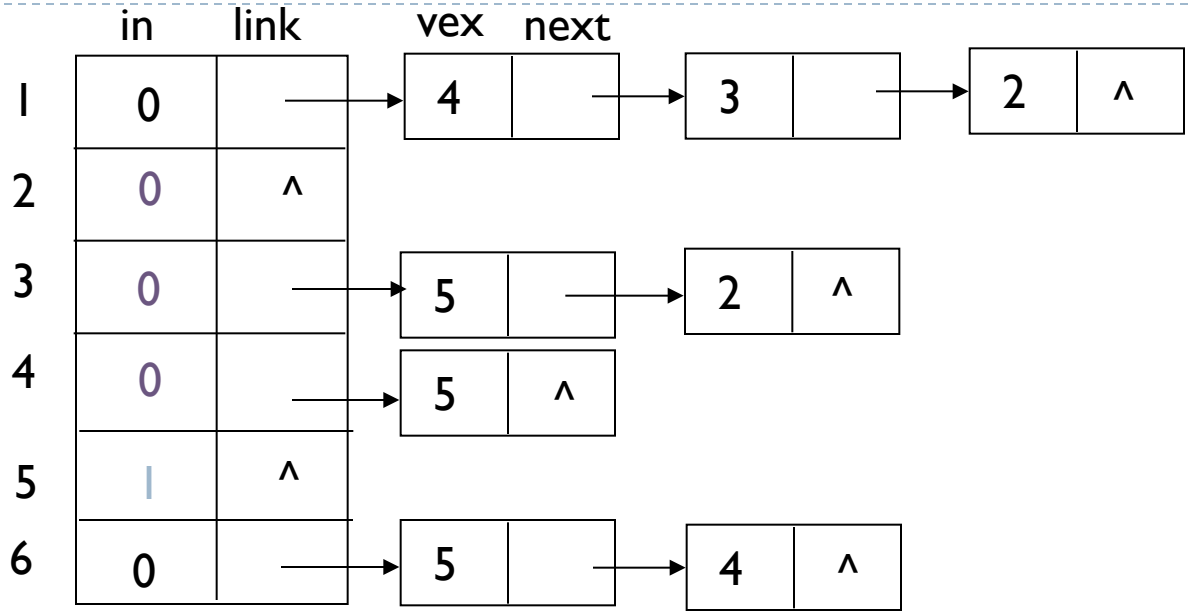
華中科技大學
人工智能与自动化学院

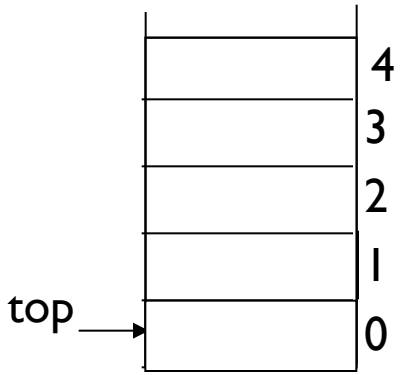




输出序列：6 1 3 2

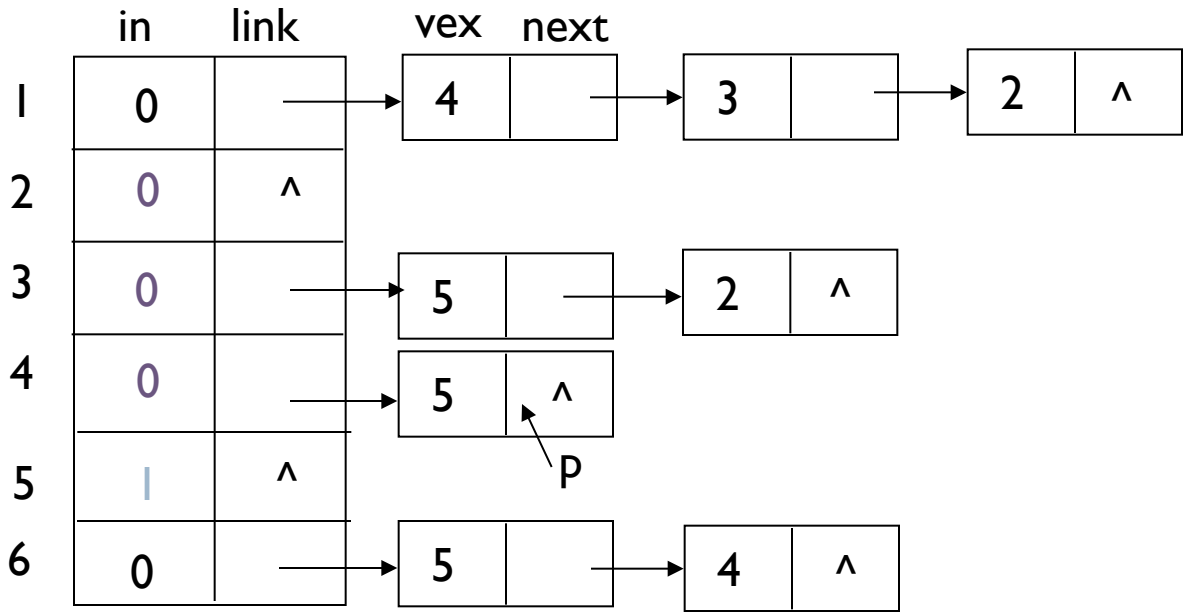
華中科技大學
人工智能与自动化学院

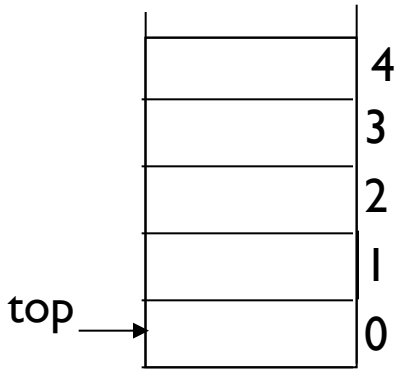




输出序列：6 1 3 2 4

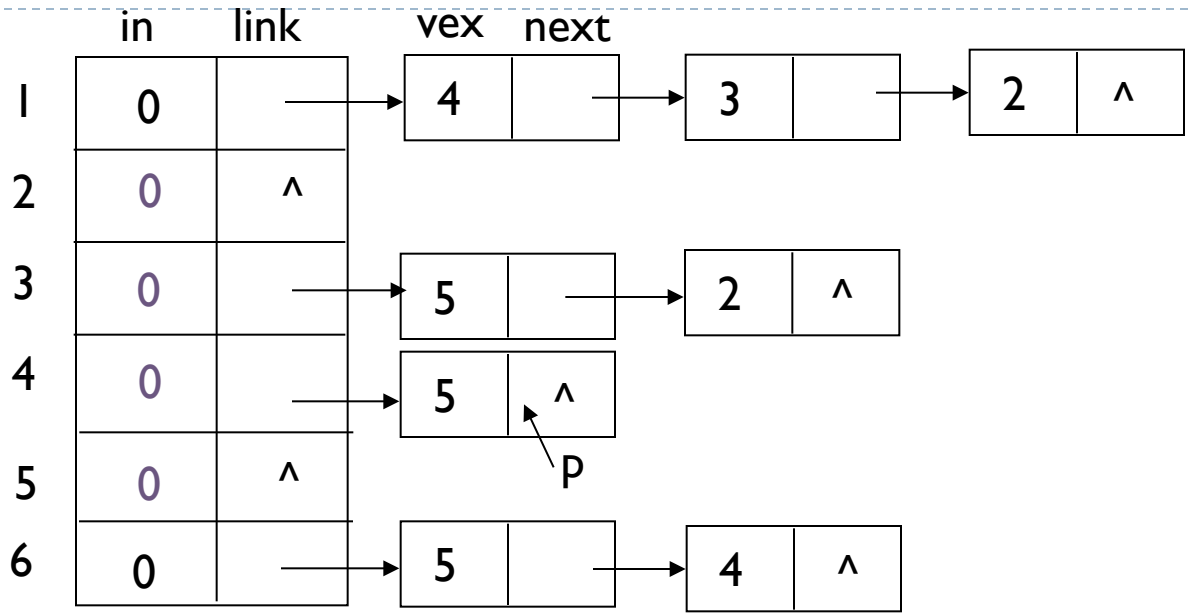
華中科技大學
人工智能与自动化学院

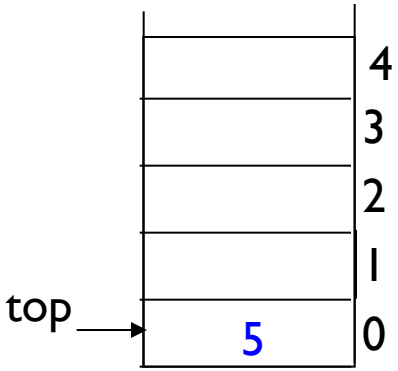




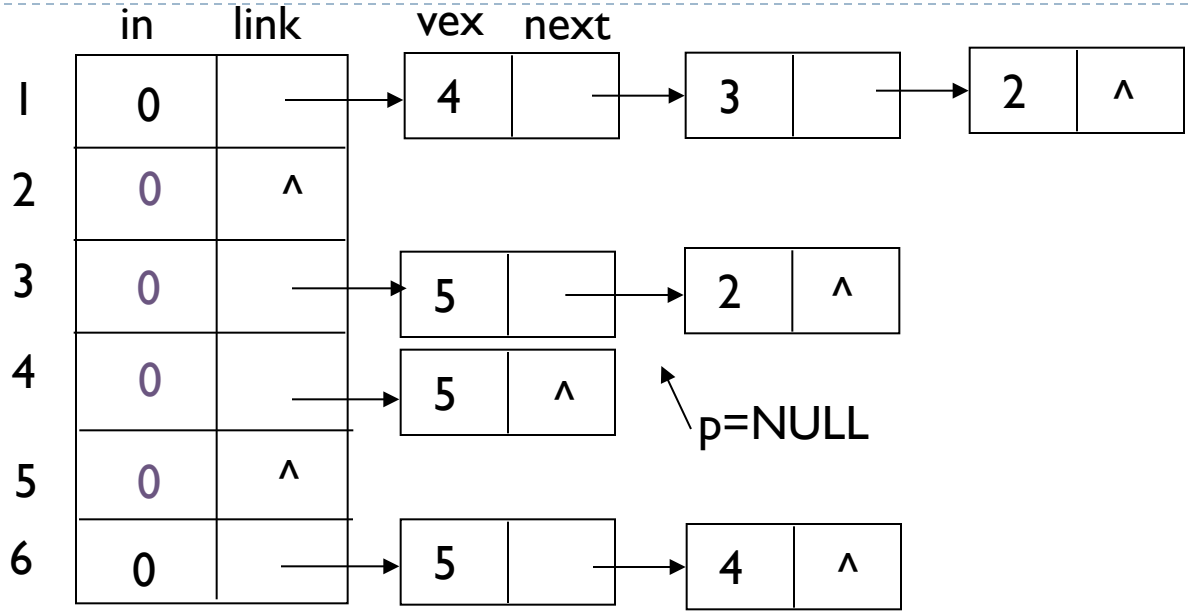
输出序列：6 1 3 2 4

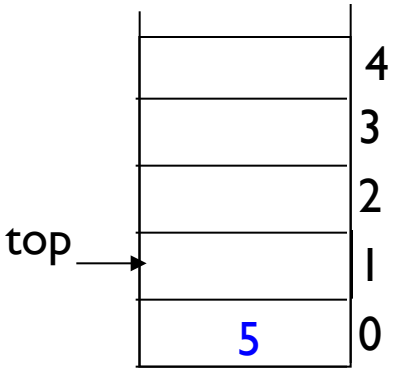
華中科技大學
人工智能与自动化学院



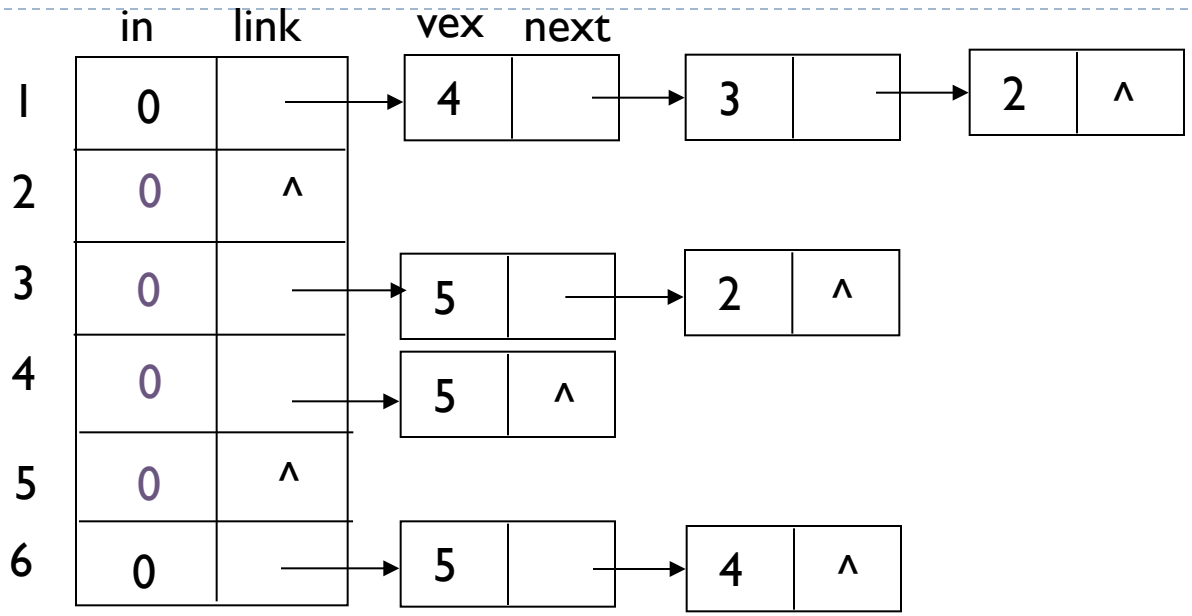


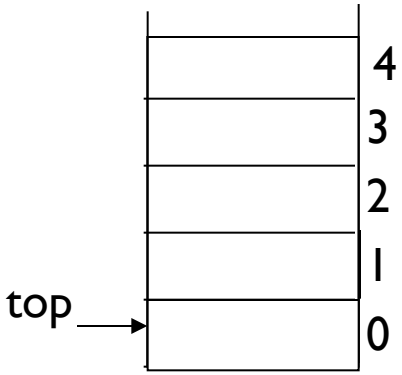
输出序列：6 1 3 2 4





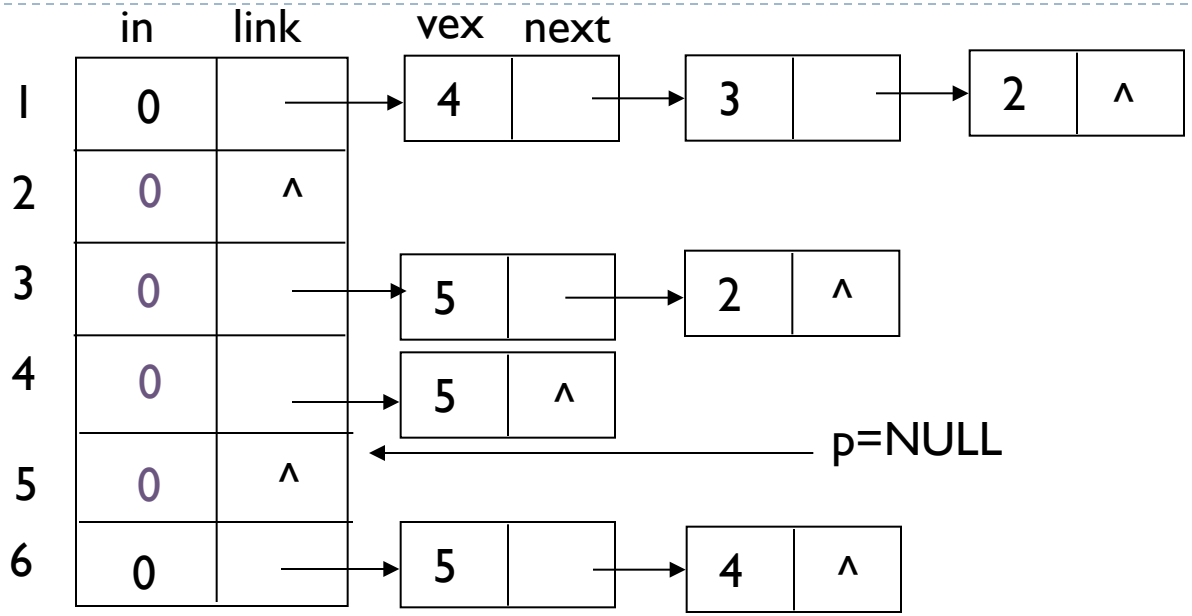
输出序列：6 1 3 2 4

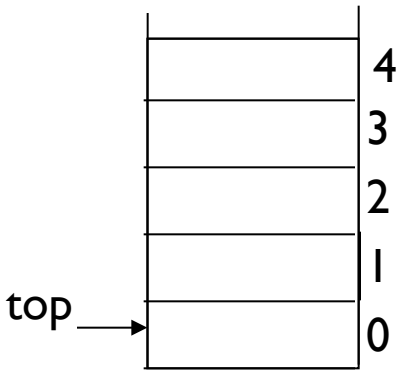




输出序列：6 1 3 2 4 5

華中科技大學
人工智能与自动化学院





输出序列：6 1 3 2 4 5

華中科技大學
人工智能与自动化学院

算法分析

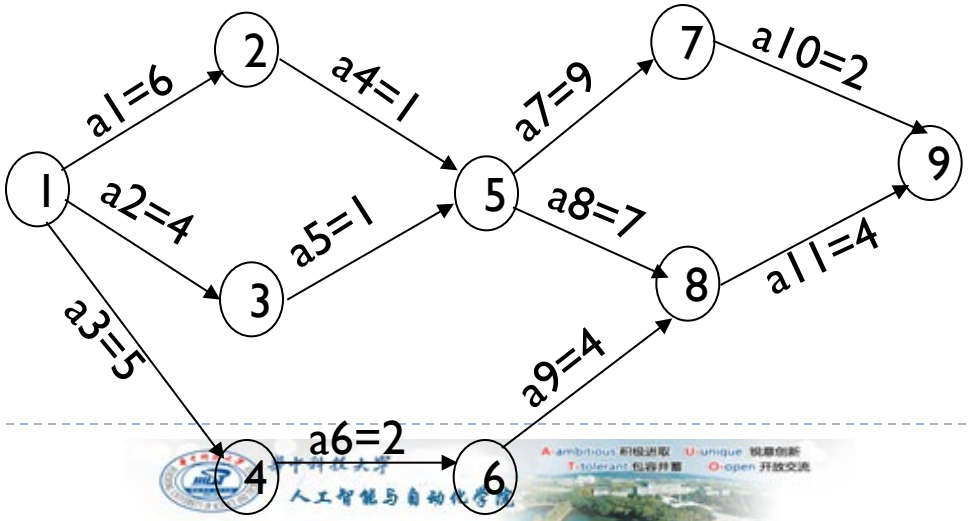
- 建邻接表：T(n)=O(e)
- 搜索入度为0的顶点的时间：T(n)=O(n)
- 拓扑排序：T(n)=O(n+e)

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

7.6 关键路径

- **问题提出**
  - 工程计划表示为有向图，用顶点表示事件，弧表示活动；
  - 每个事件表示在它之前的活动已完成，在它之后的活动可以开始

例 设一个工程有11项活动，9个事件
事件 V1——表示整个工程开始
事件V9——表示整个工程结束
问题：（1）完成整项工程至少需要多少时间？
（2）哪些活动是影响工程进度的关键？



相关定义与术语

- AOE网(Activity On Edge)——也叫边表示活动的网。AOE网是一个带权的有向无环图，其中顶点表示事件，弧表示活动，权表示活动持续时间
- 路径长度——路径上各活动持续时间之和
- 关键路径——路径长度最长的路径叫~
- Ve(j)——表示事件V_j的最早发生时间
- Vl(j)——表示事件V_j的最迟发生时间
- e(i)——表示活动a_i的最早开始时间
- l(i)——表示活动a_i的最迟开始时间
- l(i)-e(i)——表示完成活动a_i的时间余量
- 关键活动——关键路径上的活动叫~，即l(i)=e(i)的活动

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

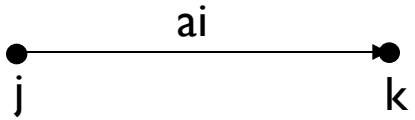
问题分析

- 如何找e(i)=l(i)的关键活动？

设活动a_i用弧<j, k>表示，其持续时间记为：dut(<j, k>)

则有：（1）e(i)=Ve(j)

（2）l(i)=Vl(k) - dut(<j,k>)



✧如何求Ve(j)和Vl(j)？

(1)从Ve(1)=0开始向前递推

$$Ve(j) = \underset{i}{Max}\{Ve(i) + dut(<i, j>)\}, <i, j> \in T, 2 \le j \le n$$

其中T是所有以j为头的弧的集合

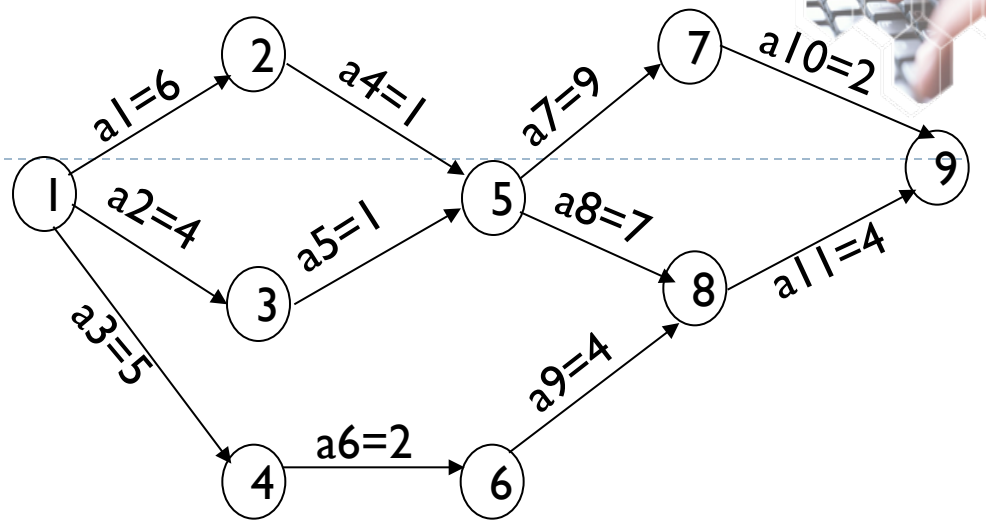
(2)从Vl(n)=Ve(n)开始向后递推

$$Vl(i) = \underset{j}{Min}\{Vl(j) - dut(<i, j>)\}, <i, j> \in S, 1 \le i \le n-1$$

其中S是所有以i为尾的弧的集合

華中科技大學 人工智能与自动化学院

- 求关键路径步骤
  - 求Ve(i)
  - 求Vl(i)
  - 求e(i)
  - 求l(i)
  - 计算l(i)-e(i)



顶点	Ve	Vl
V1	0	0
V2	6	6
V3	4	6
V4	5	8
V5	7	7
V6	7	10
V7	16	16
V8	14	14
V9	18	18

活动	e	l	l-e
a1	0	0	0 ✓
a2	0	2	2
a3	0	3	3
a4	6	6	0 ✓
a5	4	6	2
a6	5	8	3
a7	7	7	0 ✓
a8	7	7	0 ✓
a9	7	10	3
a10	16	16	0 ✓
a11	14	14	0 ✓

华中科技大学 人工智能与自动化学院

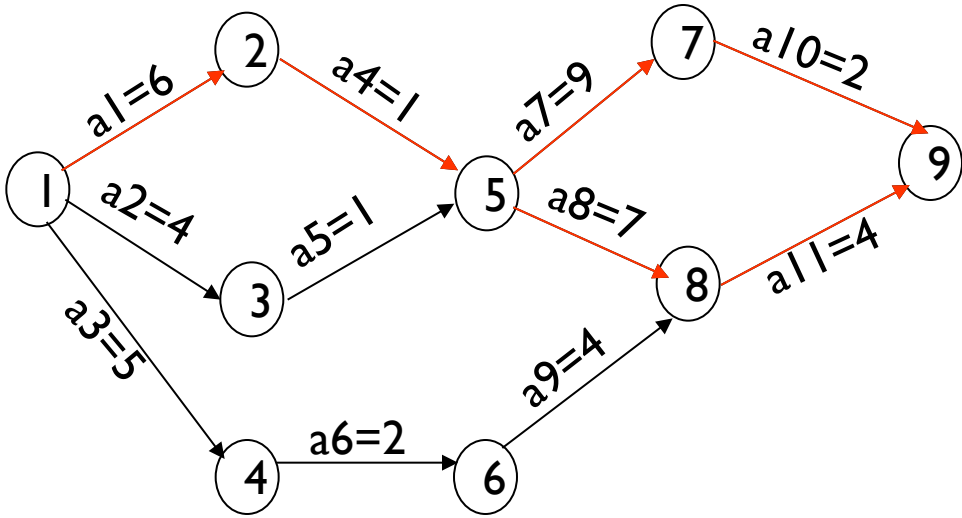
算法实现

- 以邻接表作存储结构
- 从源点V1出发，令Ve[1]=0,按拓扑序列求各顶点的Ve[i]
- 从汇点Vn出发，令Vl[n]=Ve[n]，按逆拓扑序列求其余各顶点的Vl[i]
- 根据各顶点的Ve和Vl值，计算每条弧的e[i]和l[i]，找出e[i]=l[i]的关键活动

華中科技大学 人工智能与自动化学院

算法描述

- 输入顶点和弧信息，建立其邻接表
- 计算每个顶点的入度
- 对其进行拓扑排序
  - 排序过程中求顶点的Ve[i]
  - 将得到的拓扑序列进栈
- 按逆拓扑序列求顶点的Vl[i]
- 计算每条弧的e[i]和l[i],找出e[i]=l[i]的关键活动



華中科技大學 人工智能与自动化学院

```
Status TopologicalOrder(ALGraph G, Stack &T) {
    // 有向网 G 采用邻接表存储结构，求各顶点事件的最早发生时间 ve(全局变量)。
    // T 为拓扑序列顶点栈，S 为零入度顶点栈。
    // 若 G 无回路，则用栈 T 返回 G 的一个拓扑序列，且函数值为 OK，否则为 ERROR。
    FindInDegree(G, indegree); // 对各顶点求入度 indegree[0..vernum-1]
    建零入度顶点栈 S;
    InitStack(T);  count = 0;  ve[0..G.vexnum-1] = 0;  // 初始化
    while (!StackEmpty(S)) {
反序原栈  Pop(S, j);  Push(T, j);  ++count;          // j 号顶点入 T 栈并计数
        for (p=G.vertices[j].firstarc;  p;  p=p->nextarc) {
            k = p->adjvex;                  // 对 j 号顶点的每个邻接点的入度减 1
            if (--indegree[k] == 0) Push(S, k);  // 若入度减为 0，则入栈
更新ve[k]   if (ve[j]+*(p->info) > ve[k])  ve[k] = ve[j]+*(p->info);
        }// for   *(p->info)=dut(<j,k>)

    }// while
    if (count<G.vexnum)  return ERROR;  // 该有向网有回路
    else return OK;
}// TopologicalOrder
```

算法 7.13

人工智能与自动化学院

图的邻接表存储结构

	vexdata	in	link
1	1	0	
2	2	0	
3	3	0	
4	4	0	
5	5	1	
6	6	0	
7	7	0	
8	8	1	
9	9	1	^

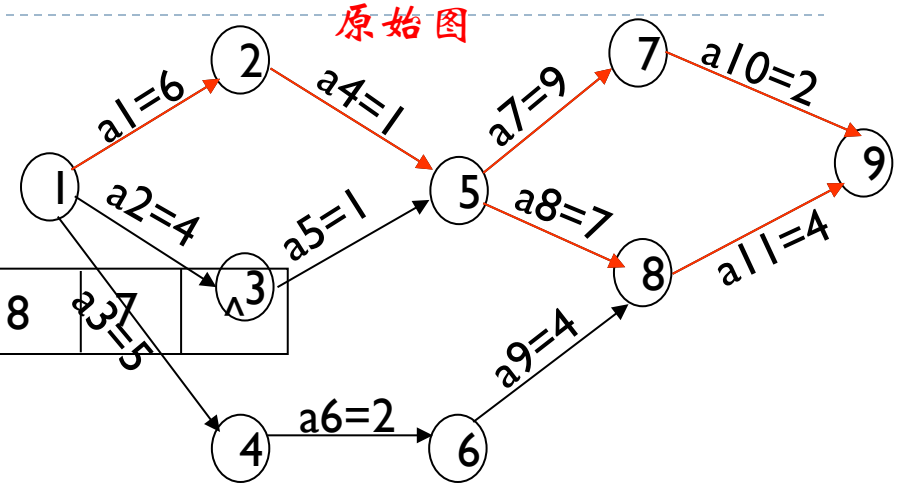
vex	length	next
2	6	
5	1	^
5	1	^
6	2	^
7	9	
8	4	^
9	2	^
9	4	^

3	4	

4	5	^

8	7	

原始图



栈S

	4
	3
	2
	1
	0

栈T

9	
7	
8	
5	
2	4
3	3
6	2
4	1
1	0

Ve(i)

1	0
2	6
3	4
4	5
5	7
6	7
7	16
8	14
9	18

华中科技大学 人工智能与自动化学院

```
Status CriticalPath((ALGraph G) {
    // G为有向网,输出G的各项关键活动。
    if (! TopologicalOrder(G, T))  return ERROR;
    vl[0..G.vexnum - 1] = ve[G.vexnum - 1];      // 初始化顶点事件的最迟发生时间
    while (! StackEmpty(T))                      // 按拓扑逆序求各顶点的 vl 值
        for (Pop(T, j), p = G.vertices[j].firstarc;  p;  p = p->nextarc) {
          k = p->adjvex;  dut = *(p->info);      // dut<j,k>
          if (vl[k]—dut<vl[j])  vl[j] = vl[k]—dut;
        }//for
   for (j = 0; j<G.vexnum; ++j)                  // 求 ee,el 和关键活动
        for (p = G.vertices[j]; .firstarc;p; p = p->nextarc) {
          k = p->adjvex;  dut = *(p->info);
          ee = ve[j]; el = vl[k]—dut;
          tag = (ee == el) ? '*' : '';
           printf (j, k, dut, ee, el, tag);   // 输出关键活动
        }
}// CriticalPath
```

算法 7.14

華中科技大學 人工智能与自动化学院

7.7 最短路径

问题提出

用带权的有向图表示一个交通运输网，图中：

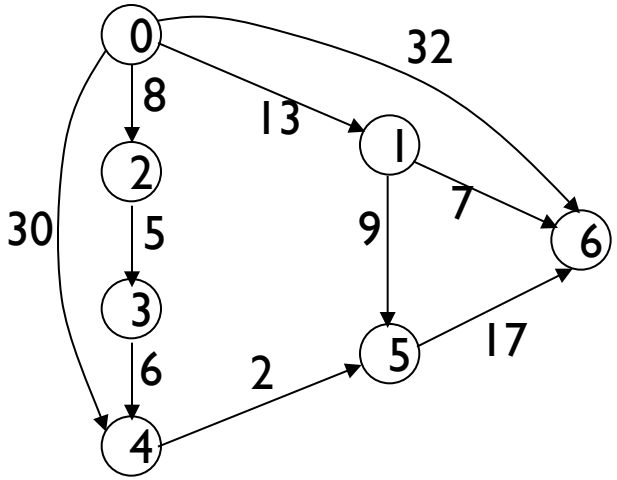
顶点——表示城市

边——表示城市间的交通联系

权——表示此线路的长度或沿此线路运输所花的时间或费用等

问题：从某顶点出发，沿图的边到达另一顶点所经过的路径中，各边上权值之和最小的一条路径——最短路径

从某个源点到其余各顶点的最短路径



最短路径	长度
<V0,V1>	13
<V0,V2>	8
<V0,V2,V3>	13
<V0,V2,V3,V4>	19
<V0,V2,V3,V4,V5>	21
<V0,V1,V6>	20

華中科技大學 人工智能与自动化学院

迪杰斯特拉(Dijkstra)算法思想

- 按路径长度递增次序产生最短路径算法：

把V分成两组：

（1）S：已求出最短路径的顶点的集合

（2）V-S=T：尚未确定最短路径的顶点集合

将T中顶点按最短路径递增的次序加入到S中，

保证：（1）从源点V0到S中各顶点的最短路径长度都不大于从V0到T中任何顶点的最短路径长度

（2）每个顶点对应一个距离值

S中顶点：从V0到此顶点的最短路径长度

下一个T中顶点最短路径：从V0到此顶点的只包括S中顶点作中间顶点的最短路径长度

依据：V0到T中顶点Vk的路径为下一条最短路径，则其或是从V0到Vk的直接路径的权值，或是从V0经S中顶点到Vk的路径权值之和

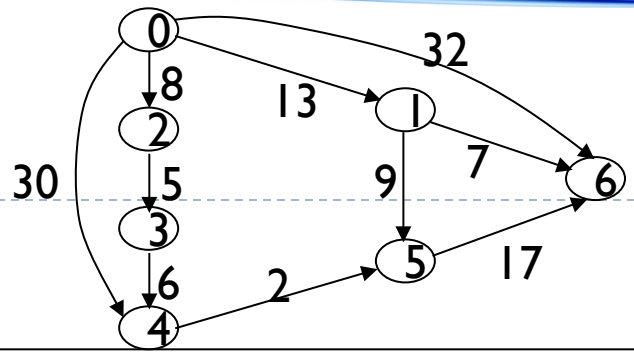
（反证法可证）

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

求最短路径步骤

- 初始时令 S={V0},T={其余顶点}，T中顶点对应的距离值
  - 若存在<V0,Vi>，为<V0,Vi>弧上的权值
  - 若不存在<V0,Vi>，为∞
- 从T中选取一个其距离值为最小的顶点W，加入S
- 对T中顶点的距离值进行修改：若加进W作中间顶点，从V0到Vi的距离值比不加W的路径要短，则修改此距离值
- 重复上述步骤，直到S中包含所有顶点，即S=V为止

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流



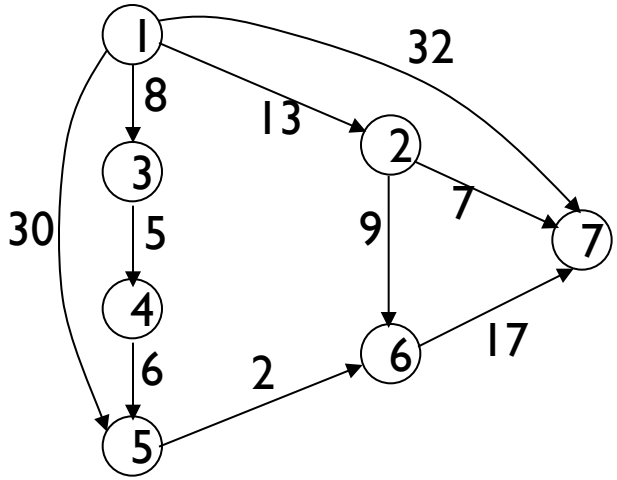
终点	从V0到各终点的最短路径及其长度				
V1	13 <V0,V1>	13 <V0,V1>	-------	-------	--------
V2	8 <V0,V2>	-------	-------	-------	--------
V3	∝	13 <V0,V2,V3>	13 <V0,V2,V3>	-------	--------
V4	30 <V0,V4>	30 <V0,V4>	30 <V0,V4>	19 <V0,V2,V3,V4>	--------
V5	∝	∝	22 <V0,V1,V5>	22 <V0,V1,V5>	21 <V0,V2,V3,V4,V5>
V6	32 <V0,V6>	32 <V0,V6>	20 <V0,V1,V6>	20 <V0,V1,V6>	20 <V0,V1,V6>
Vj	V2:8 <V0,V2>	V1:13 <V0,V1>	V3:13 <V0,V2,V3>	V4:19 <V0,V2,V3,V4>	V6:20 <V0, V1,V6>

算法实现

- 图用带权邻接矩阵存储arcs[][]
- 数组D[]存放当前找到的从源点V0到每个终点的最短路径长度，其初态为图中直接路径权值
- 数组pre[]表示从V0到各终点的最短路径上，此顶点的前一顶点的序号；若从V0到某终点无路径，则用0作为其前一顶点的序号

華中科技大學 人工智能与自动化学院

算法描述



$$arcs[][] = \begin{bmatrix} 0 & 13 & 8 & \infty & 30 & \infty & 32 \\ \infty & 0 & \infty & \infty & \infty & 9 & 7 \\ \infty & \infty & 0 & 5 & \infty & \infty & \infty \\ \infty & \infty & \infty & 0 & 6 & \infty & \infty \\ \infty & \infty & \infty & \infty & 0 & 2 & \infty \\ \infty & \infty & \infty & \infty & \infty & 0 & 17 \\ \infty & \infty & \infty & \infty & \infty & \infty & 0 \end{bmatrix}$$

最短路径	长度
<V1,V2>	13
<V1,V3>	8
<V1,V3,V4>	13
<V1,V3,V4,V5>	19
<V1,V3,V4,V5,V6>	21
<V1,V2,V7>	20

	1	2	3	4	5	6	7
D	0	13	8	∞ 13	30 19	∞ 21	32 20

	1	2	3	4	5	6	7
pre	0	1	1	0 3	1 4	0 5	1 2

k=1 (1)

✧算法分析：$T(n)=O(n^2)$

華中科技大學 人工智能与自动化学院

算法代码

```
void ShortestPath_DIJ( MGraph G, int v0, PathMatrix &P, ShortPathTable &D ) {
  // 用 Dijkstra 算法求有向网 G 的 v0 顶点到其余顶点 v 的最短路径 P[v]及其带权长度 D[v]。
  // 若 P[v][w]为 TRUE,则 w 是从 v0 到 v 当前求得最短路径上的顶点。
  // final[v]为 TRUE 当且仅当 v∈S,即已经求得从 v0 到 v 的最短路径。
  for (v = 0; v<G.vexnum; ++v) {
     final[v] = FALSE;  D[v] = G.arcs[v0][v];
     for (w = 0; w<G.vexnum; ++w)  P[v][w] = FALSE;     // 设空路径
     if (D[v] < INFINITY) {P[v][v0] = TRUE;  P[v][v] = TRUE; }
  }// for
  D[v0] = 0;  final[v0] = TRUE;                  // 初始化,v0 顶点属于 S 集
  //开始主循环,每次求得 v0 到某个 v 顶点的最短路径,并加 v 到 S 集
  for (i = 1; i<G.vexnum; ++i) {                 // 其余 G.vexnum - 1 个顶点
     min = INFINITY;                             // 当前所知离 v0 顶点的最近距离
     for (w = 0; w<G.vexnum; ++w)
        if (!final[w])                           // w 顶点在 V - S 中
           if (D[w]<min) {v = w;  min = D[w]; }     // w 顶点离 v0 顶点更近
     final[v] = TRUE;                            // 离 v0 顶点最近的 v 加入 S 集
     for (w = 0; w<G.vexnum; ++w)  // 更新当前最短路径及距离
        if (!final[w] && (min + G.arcs[v][w]<D[w])) {  // 修改 D[w]和 P[w], w∈V - S
           D[w] = min + G.arcs[v][w];
           P[w] = P[v];  P[w][w] = TRUE;  // P[w] = P[v] + [w]
        }// if
  }// for
}// ShortestPath_DIJ
```

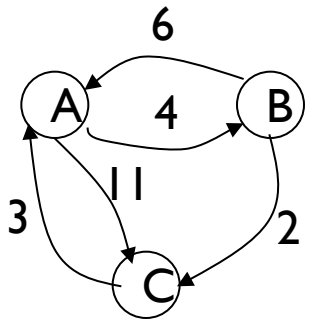

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

每一对顶点之间的最短路径

- 方法一：每次以一个顶点为源点，重复执行Dijkstra算法n次——$T(n)=O(n^3)$
- 方法二：弗洛伊德(Floyd)算法
  - 算法思想：逐个顶点试探法
  - 求最短路径步骤
    - 初始时设置一个n阶方阵，令其对角线元素为0，若存在弧<Vi,Vj>，则对应元素为权值；否则为∞
    - 逐步试着在原直接路径中增加中间顶点，若加入中间点后路径变短，则修改之；否则，维持原值
    - 所有顶点试探完毕，算法结束

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

例



初始：
$$\begin{pmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & \infty & 0 \end{pmatrix}$$

路径：

	AB	AC
BA		BC
CA		

加入V1：
$$\begin{pmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{pmatrix}$$

路径：

	AB	AC
BA		BC
CA	CAB	

加入V2：
$$\begin{pmatrix} 0 & 4 & 6 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{pmatrix}$$

路径：

	AB	ABC
BA		BC
CA	CAB	

加入V3：
$$\begin{pmatrix} 0 & 4 & 6 \\ 5 & 0 & 2 \\ 3 & 7 & 0 \end{pmatrix}$$

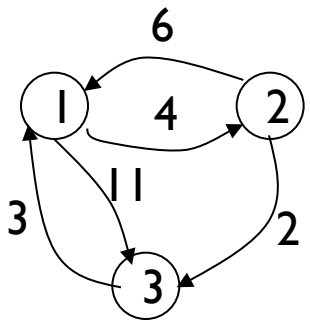
路径：

	AB	ABC
BCA		BC
CA	CAB	

華中科技大學 人工智能与自动化学院

- 算法实现
  - 图用邻接矩阵存储
  - length[][]存放最短路径长度
  - path[i][j]是从Vi到Vj的最短路径上Vj前一顶点序号
- 算法描述

例



初始：

$$length=\begin{pmatrix}0&4&11\\6&0&2\\3&\infty&0\end{pmatrix}\quad path=\begin{pmatrix}0&1&1\\2&0&2\\3&0&0\end{pmatrix}$$

加入V1：

$$length=\begin{pmatrix}0&4&11\\6&0&2\\3&7&0\end{pmatrix}\quad path=\begin{pmatrix}0&1&1\\2&0&2\\3&1&0\end{pmatrix}$$

加入V2：

$$length=\begin{pmatrix}0&4&6\\6&0&2\\3&7&0\end{pmatrix}\quad path=\begin{pmatrix}0&1&2\\2&0&2\\3&1&0\end{pmatrix}$$

加入V3：

$$length=\begin{pmatrix}0&4&6\\5&0&2\\3&7&0\end{pmatrix}\quad path=\begin{pmatrix}0&1&2\\3&0&2\\3&1&0\end{pmatrix}$$

- 算法分析：$T(n)=O(n^3)$

華中科技大學 人工智能与自动化学院

Take-home Message

- 图是特殊的非线性数据结构：多对多
- 存储表示：邻接矩阵、邻接表、十字链表、多重邻接表
- 遍历：深度优先与广度优先
- 最小生成树算法：普里姆和克鲁斯卡尔算法
- 拓扑排序算法：构建栈实现
- 关键路径算法：以拓扑排序为基础
- 最短路径算法：迪杰斯特拉算法

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流